Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 1_COD_SB

Attempt : 2
Total Mark : 50
Marks Obtained : 27

Section 1 : Coding

1. Problem Statement

Arjun is working on a project where he needs to analyze a series of integers within a specified range [A, B]. The project involves identifying certain numbers that are not divisible by any smaller positive integers other than 1 and themselves. These numbers are crucial for the next step of his analysis, as they have unique mathematical properties.

Arjun has been asked to implement an efficient way to find all such numbers in a given range. Given that the range can be large, he needs to ensure the solution is optimized.

Can you help him implement this task using the Simple Sieve of Eratosthenes?

Input Format

vtu27499

The first line of input consists of an integer A, representing the starting range.

The second line of input consists of an integer B, representing the ending range.

Output Format

The output prints a space-separated list of numbers that are not divisible by any smaller positive integers other than 1 and themselves. within the range [A, B].

Refer to the sample output for formatting specifications

Sample Test Case

Input: 2
10
Output: 2 3 5 7

Answer

```
#include<stdio.h>
#include<stdbool.h>
int main(){
   int A, B;
   scanf("%d", &A);
   scanf("%d", &B);
   bool isPrime[B+1];
   for (int i = 0; i <= B; i++) isPrime[i] = true;
   isPrime[0] = isPrime[1] = false;
   for (int i = 2; i * i <= B; i++) {
      if (isPrime[i]) {
         for (int j = i * i; j <= B; j += i)
         isPrime[j] = false;
      }
   }
   for (int i = A; i <= B; i++){
      if (isPrime[i]) printf("%d", i);
   }
   printf("\n");
   return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Pradeep Narwal is analyzing multiple numerical ranges to identify prime numbers within each range. He is given several ranges defined by a starting number A and an ending number B. Using the Sieve of Eratosthenes, Pradeep wants to efficiently determine all prime numbers that lie within each range.

Write a program to ensure that all prime numbers between A and B (inclusive) are printed for every given range.

Input Format

The first line of input consists of an integer Q, representing the number of ranges.

The next Q lines each contain two integers A and B , representing the start and end of each range.

Output Format

For each range, print all prime numbers between A and B (inclusive) in a single line.

Prime numbers must be printed in ascending order, separated by a single space.

If a range contains no prime numbers, print an empty line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2
1 5
10 15

Output: 2 3 5
11 13

Answer

```
#include<stdio.h>
```

```
#include<stdlib.h>
#include<math.h>
#include<stdbool.h>
void simpleSieve(int limit, int primes[], int *count){
    bool isPrime[limit+1];
    for(int i=0;i<=limit;i++) isPrime[i]=true;
    isPrime[0]=isPrime[1]=false;
    for(int i=2;i*i<=limit;i++)
    if(isPrime[i])
    for(int j=i*i;j<=limit;j+=i)
    isPrime[j]=false;
    *count=0;
    for(int i=2;i<=limit;i++)
    if(isPrime[i]) primes[(*count)++]=i;
}
void segmentedSieve(int A,int B,int primes[],int count){
    bool isPrime[B-A+1];
    for(int i=0;i<=B-A;i++) isPrime[i]=true;
    for(int i=0;i<count;i++){
        int p=primes[i];
        if((long long)p*p>B) break;
        int start=(A+p-1)/p*p;
        if(start<p*p) start=p*p;
        for (int j=start;j<=B;j+=p) isPrime[j-A]=false;
    }
    for (int i=A;i<=B;i++)
        if (i>=2 && isPrime[i-A]) printf("%d",i);
    printf("\n");
}
int main() {
    int Q; scanf("%d",&Q);
    int A[Q],B[Q],maxB=0;
    for (int i=0;i<Q;i++) {
        scanf("%d %d",&A[i],&B[i]);
        if (B[i]>maxB) maxB=B[i];
    }
    int primes[100000],count;
    simpleSieve((int)sqrt(maxB)+1,primes,&count);
    for (int i=0;i<Q;i++)
        segmentedSieve(A[i],B[i],primes,count);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Joanna is developing a utility tool for a math club's quiz event. One of the rounds involves finding the greatest common divisor (GCD) of a list of numbers. Instead of computing it manually for each pair, she wants to automate the task using code.

She decides to use Euclid's Algorithm, a well-known and efficient method to compute the GCD of two numbers. For a list of numbers, she will apply the algorithm iteratively across all elements in the array.

Help Joanna implement a program that takes an array of positive integers and computes the GCD of the entire array using Euclid's Algorithm.

Make sure the program also handles special cases:

If the array is empty, return -1.If the array has only one number, return that number as the GCD.

Example

Input:

3

12 18 24

Output:

6

Explanation:

GCD(12, 18) = 6GCD(6, 24) = 6Final result = 6

Input Format

The first line of input contains an integer n, representing the number of elements in the array.

The second line contains n space-separated positive integers, each representing

an element of the array.

Output Format

The output consists of a single line:

- If the array contains two or more elements, print a single integer representing the GCD of all array elements.
- If the array contains only one element, print that element as the GCD.
- If the array is empty, print -1 as a convention to indicate undefined GCD.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 2
8 12

Output: 4

Answer

```
#include<stdio.h>
int gcd(int a, int b) {
   while (b != 0) {
      int temp = b;
      b = a % b;
      a = temp;
   }
   return a;
}
int gcdArray(int arr[], int n) {
   if (n == 0) return -1;
   if (n == 1) return arr[0];
   int result = arr[0];
   for (int i = 1; i < n; i++) {
      result = gcd(result, arr[i]);
   }
   return result;
}
```

```
int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    int result = gcdArray(arr, n);
    printf("%dn", result);
    return 0;
}
```

Status : Wrong **Marks : 0/10**

4. Problem Statement

Raychel is working on a calendar synchronization tool where she needs to determine how often two repeating tasks will coincide. Each task occurs after a fixed number of days, and she must find the least number of days after which both tasks align again — in other words, the least common multiple (LCM) of the two intervals.

Raychel knows that the most efficient way to compute the LCM is by first calculating the greatest common divisor (GCD) using Euclid's Algorithm, and then using that result to derive the LCM. She also wants to make sure the program works for large inputs without running into overflow errors.

Help Raychel implement a program that calculates the LCM of two non-negative integers by first using Euclid's Algorithm to compute their GCD, and then deriving the LCM from it.

Example

Input:

15 20

Explanation:

GCD(15, 20) = 5LCM = (15 / 5) × 20 = 3 × 20 = 60

Output:

60

Input Format

The input consists of a single line containing two space-separated integers 'a' and 'b', representing the two input numbers.

Output Format

The output consists of a single line:

- If both 'a' and 'b' are non-zero, print a single integer — the least number that is divisible by both.
- If either 'a' or b is '0', print 0 as the result since no common multiple exists in such cases.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 15 20

Output: 60

Answer

```
#include<stdio.h>
long long gcd(long long a, long long b){
   while (b!=0){
      long long temp=b;
      b=a%b;
      a=temp;
   }
   return a;
}
long long lcm(long long a, long long b) {
```

```
    if (a==0 || b==0) return 0;
    long long g = gcd(a, b);
}
int main(){
    long long a, b;
    scanf("%lld %lld", &a, &b);
    long long result=lcm(a, b);
    printf("%lld\n", result);
    return 0;
}
```

Status : Partially correct ***Marks : 1/10***

5. Problem Statement:

A software company is developing a mathematical tool for students to help them understand factorization and modular arithmetic. As part of the tool, they need a function that calculates the product of all positive divisors (factors) of a given integer, with the result taken modulo 1000000007. The tool is intended to assist in solving mathematical problems and providing insights into number theory.

Your task is to implement a function that computes the product of all factors of a given integer n, and then display the result modulo 1000000007.

Examples :

Input: 12

Output: 1728

1 * 2 * 3 * 4 * 6 * 12 = 1728

Input Format

The input consists of a single integer n.

Output Format

The output displays a single integer which is the product of all factors of n

modulo 1000000007.

Refer to the sample output for better understanding.

Sample Test Case

Input: 12

Output: 1728

Answer

```
#include<stdio.h>
#include<math.h>
#define MOD 1000000007
long long modPow(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) result = (result * base) % mod;
        base = (base * base) % mod;
        exp >>= 1;
    }
    return result;
}
int main() {
    int n;
    scanf("%d", &n);
    int count = 0;
    for (int i = 1; i <= sqrt(n); i++) {
        if (n % i == 0) {
            if (i == n / i) count++;
            else count += 2;
        }
    }
    long long result = modPow(n, count / 2, MOD);
    printf("%lld\n", result);
    return 0;
}
```

Status : Partially correct ***Marks : 6/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 1_COD_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Bella is developing a number puzzle game where each level involves simplifying equations using number theory. In one level, she comes across two large positive integers and is asked to reduce them using their greatest common divisor (GCD).

However, the puzzle has an extra twist:

She must express the GCD as a linear combination of the two numbers – that is, she needs to find two integers x and y such that: GCD(a, b) = a * x + b * y

Help Bella write a program that, given two integers a and b, finds:

The GCD of a and bThe coefficients 'x' and y" that satisfy the linear combination condition

Example

Input

48 18

Output:

6 -1 3

Explanation:

The GCD of 48 and 18 is 6.

Using the Extended Euclidean Algorithm, we can find integers x = -1 and y = 3 such that:

48 × (-1) + 18 × 3 = -48 + 54 = 6

This satisfies Bézout's Identity: GCD(48, 18) = 48x + 18y = 6.

Input Format

- The input consists of a single line with two space-separated integers 'a' and 'b'

Output Format

The output prints three space-separated integers in a single line:

- The first integer is the GCD of a and b.
- The second integer is the coefficient x such that a * x + b * y = GCD.
- The third integer is the coefficient y such that a * x + b * y = GCD.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 45 21
Output: 3 1 -2

Answer

```
#include<stdio.h>
long long extendedGCD(long long a, long long b, long long *x, long long *y){
```

```
    if(b == 0){
        *x = 1;
        *y = 0;
        return a;
    }
long long x1, y1;
long long gcd = extendedGCD(b, a % b, &x1, &y1);
*x = y1;
*y = x1 - (a / b) *y1;
return gcd;
}
int main() {
    long long a, b;
    scanf("%lld %lld", &a, &b);
    long long x, y;
    long long gcd = extendedGCD(a, b, &x, &y);
    printf("%lld %lld %lld\n",gcd, x, y);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Arjun is analyzing a numerical range to find special pairs of prime numbers. He is given a range defined by two integers L and R, and a target value T. Within the range [L,R], Arjun wants to identify all pairs of prime numbers whose sum equals T.

Write a program to ensure that all such valid prime pairs are printed in ascending order.

Input Format

The first line contains three space-separated integers:

- L starting value of the range
- R ending value of the range
- T target sum

Output Format

The output prints each valid pair of prime numbers whose sum equals T, one pair per line.

Each line should contain two space-separated integers representing a valid prime pair.

If no such pairs exist, no output should be printed.

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 1 40 10
Output: 3 7
5 5

Answer

```
#include <stdio.h>
#include <stdbool.h>

// Sieve of Eratosthenes
void sieve(bool prime[], int max) {
    for(int i = 0; i <= max; i++) {
        prime[i] = true;
    }

    prime[0] = prime[1] = false;

    for(int i = 2; i * i <= max; i++) {
        if(prime[i]) {
            for(int j = i * i; j <= max; j += i) {
                prime[j] = false;
            }
        }
    }
}

int main() {
    int L, R, T;
    scanf("%d %d %d", &L, &R, &T);
```

```
    bool prime[R + 1];

    // Generate primes up to R
    sieve(prime, R);

    // Store primes within range [L, R]
    int primes[R], count = 0;
    for(int i = L; i <= R; i++) {
        if(prime[i]) {
            primes[count++] = i;
        }
    }

    // Two-pointer technique
    int left = 0, right = count - 1;

    while(left <= right) {
        int sum = primes[left] + primes[right];

        if(sum == T) {
            printf("%d %d\n", primes[left], primes[right]);
            left++;
            right--;
        }
        else if(sum < T) {
            left++;
        }
        else {
            right--;
        }
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Sai Kishore is working on a tool that identifies special numbers in a range.

These numbers must have only two divisors: 1 and themselves. His task is to count how many of these special numbers exist below a given threshold N. Since N can be as large as one million, Sai Kishore needs an efficient way to find these numbers in a given range.

Sai Kishore decides to use a technique called Sieve of Eratosthenes, which is well-suited for efficiently identifying these special numbers.

Can you help him implement a program that counts how many such numbers exist below N?

Example:

Input:

10

Output:

4

Explanation:

The numbers less than 10 that have only two divisors (1 and themselves) are: 2, 3, 5, and 7. Thus, the output is 4.

Input Format

The input consists of a single integer N, representing the upper bound for the search.

Output Format

The output should be a single integer, representing the count of special numbers less than N.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10
Output: 4

vtu27499

Answer

```c
#include <stdio.h>
#include <stdbool.h>

int main() {
    int N;
    scanf("%d", &N);

    bool prime[N];

    // Initialize all as true
    for(int i = 0; i < N; i++) {
        prime[i] = true;
    }

    // 0 and 1 are not prime
    if(N > 0) prime[0] = false;
    if(N > 1) prime[1] = false;

    // Sieve of Eratosthenes
    for(int i = 2; i * i < N; i++) {
        if(prime[i]) {
            for(int j = i * i; j < N; j += i) {
                prime[j] = false;
            }
        }
    }

    // Count primes less than N
    int count = 0;
    for(int i = 2; i < N; i++) {
        if(prime[i]) {
            count++;
        }
    }

    // Output
    printf("%d", count);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

In the field of cryptography, especially in public-key algorithms like RSA, it is crucial to identify pairs of numbers that are coprime — i.e., they do not share any common divisors other than 1.

Before generating secure keys, cryptographic systems often verify whether two large numbers are coprime. This process uses a classical method known as Euclid's Algorithm, which efficiently determines the greatest common divisor (GCD).

You are given a list of number pairs. Your task is to determine whether each pair of integers is coprime using Euclid's Algorithm. If the GCD of the two numbers is 1, output "Coprime". Otherwise, output "Not Coprime".

Example

Input:

3

15 28

12 35

8 8

Explanation:

GCD(15, 28) = 1 Coprime

GCD(12, 35) = 1 Coprime

GCD(8, 8) = 8 Not Coprime

Output:

Coprime

Coprime

Not Coprime

Input Format

- The first line contains an integer 't' — the number of pairs to check.
- Each of the next 't' lines contains two space-separated integers 'a' and 'b'.

Output Format

- For each pair, output a single line:
- "Coprime" if the two numbers are coprime
- "Not Coprime" otherwise

- Each output must appear on a new line, in the same order as the input pairs.

Refer to the sample output for formatting specifictions.

Sample Test Case

Input: 1
17 29
Output: Coprime

Answer

```
#include <stdio.h>

// Function to find GCD using Euclid's Algorithm
long long gcd(long long a, long long b) {
   while(b != 0) {
      long long temp = b;
      b = a % b;
      a = temp;
   }
   return a;
}
```

```
int main() {
    int t;
    scanf("%d", &t);

    while(t--) {
        long long a, b;
        scanf("%lld %lld", &a, &b);

        if(gcd(a, b) == 1) {
            printf("Coprime\n");
        } else {
            printf("Not Coprime\n");
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 1_COD_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Haritha is working on a project to identify pairs of numbers in a given range [L, R] where each number in the pair is divisible only by 1 and itself.

She needs to find pairs of such numbers that differ by exactly 2. These pairs of numbers should be displayed efficiently within the range.

Help her to implement the task.

Input Format

The input consists of two integers L and R separated by a new line.

Output Format

The output prints each pair of numbers within the range [L, R] on a new line, in the format: a b where a and b are numbers that differ by exactly 2 and are

divisible only by 1 and themselves.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 30
60

Output: 41 43

Answer

```
#include <stdio.h>
#include <stdbool.h>

// Sieve of Eratosthenes
void sieve(bool prime[], int max) {
    for(int i = 0; i <= max; i++) {
        prime[i] = true;
    }

    prime[0] = prime[1] = false;

    for(int i = 2; i * i <= max; i++) {
        if(prime[i]) {
            for(int j = i * i; j <= max; j += i) {
                prime[j] = false;
            }
        }
    }
}

int main() {
    int L, R;
    scanf("%d%d", &L, &R);

    bool prime[R + 1];

    // Generate primes up to R
    sieve(prime, R);
```

```
    // Find twin primes in range
    for(int i = L; i <= R - 2; i++) {
        if(prime[i] && prime[i + 2]) {
            printf("%d %d\n", i, i + 2);
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Neha is designing a secret code system for her cybersecurity class. She discovers that certain numbers with exactly two distinct divisors (1 and themselves) create the most secure keys when their reversed digit sum is divisible by a given validation number K.

Neha needs to find all such numbers up to a specified limit N that meet both conditions. These numbers are crucial for the security of her code system. Given the large range of numbers, she needs an efficient solution to identify them.

Help her write a program to find all such numbers ≤ N that meet this criteria.

Input Format

The input consists of two space separated integers N and K

Output Format

The output prints all numbers that meet both conditions. Each number should be printed space-separated.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10 1

Output: 2 3 5 7

Answer

```
#include <stdio.h>
#include <stdbool.h>

// Function to reverse a number
int reverse(int num) {
    int rev = 0;
    while(num > 0) {
        rev = rev * 10 + (num % 10);
        num /= 10;
    }
    return rev;
}

// Sieve of Eratosthenes
void sieve(bool prime[], int n) {
    for(int i = 0; i <= n; i++) {
        prime[i] = true;
    }

    prime[0] = prime[1] = false;

    for(int i = 2; i * i <= n; i++) {
        if(prime[i]) {
            for(int j = i * i; j <= n; j += i) {
                prime[j] = false;
            }
        }
    }
}

int main() {
    int N, K;
    scanf("%d %d", &N, &K);

    bool prime[N + 1];

    // Generate primes
    sieve(prime, N);
```

```
    // Check conditions
    for(int i = 2; i <= N; i++) {
        if(prime[i]) {
            int rev = reverse(i);
            int sum = i + rev;

            if(sum % K == 0) {
                printf("%d ", i);
            }
        }
    }

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

3. Problem Statement

Ashwin is working on a system to identify numbers within a given range [L, R] that have a special property: they are divisible only by 1 and themselves.

Ashwin needs to find pairs of such numbers where both numbers are rearrangements of each other's digits. Ashwin must efficiently identify all such pairs in the given range.

Help him to implement the task.

Input Format

The input consists of two space separated integers L and R

Output Format

The output prints each pair of numbers within the range [L, R] that satisfy both conditions: they are divisible only by 1 and themselves, and they are rearrangements of each other's digits. Each pair is printed in the format: p1 p2, where p1 and p2 are numbers that are rearrangements of each other's digits.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 10 100

Output: 13 31
17 71
37 73
79 97

Answer

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

// Sieve of Eratosthenes
void sieve(bool prime[], int n) {
    for(int i = 0; i <= n; i++) {
        prime[i] = true;
    }

    prime[0] = prime[1] = false;

    for(int i = 2; i * i <= n; i++) {
        if(prime[i]) {
            for(int j = i * i; j <= n; j += i) {
                prime[j] = false;
            }
        }
    }
}

// Sort digits of a number (convert to string and sort)
void sortDigits(int num, char str[]) {
    sprintf(str, "%d", num);
    int len = strlen(str);

    // simple bubble sort for digits
    for(int i = 0; i < len - 1; i++) {
        for(int j = 0; j < len - i - 1; j++) {
            if(str[j] > str[j + 1]) {
                char temp = str[j];
```

```
            str[j] = str[j + 1];
            str[j + 1] = temp;
        }
    }
  }
}

int main() {
  int L, R;
  scanf("%d %d", &L, &R);

  bool prime[R + 1];
  sieve(prime, R);

  // Store primes in range
  int primes[R], count = 0;
  for(int i = L; i <= R; i++) {
    if(prime[i]) {
      primes[count++] = i;
    }
  }

  char s1[10], s2[10];

  // Check all pairs
  for(int i = 0; i < count; i++) {
    for(int j = i + 1; j < count; j++) {
      sortDigits(primes[i], s1);
      sortDigits(primes[j], s2);

      if(strcmp(s1, s2) == 0) {
        printf("%d %d\n", primes[i], primes[j]);
      }
    }
  }

  return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Suresh is studying prime numbers and is interested in identifying a special category of primes. Given a number N, he wants to find all prime numbers less than or equal to N whose sum of digits is also a prime number. To solve this efficiently, Suresh decides to use the Sieve of Eratosthenes for prime number generation.

Write a program to ensure that all such prime numbers are printed in increasing order.

Input Format

The input consists of single integer N, representing the upper limit of the range.

Output Format

The output prints all prime numbers less than or equal to N whose digit sum is also a prime number.

The numbers must be printed in ascending order, separated by a single space.

If no such numbers exist, print nothing.

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 50
Output: 2 3 5 7 11 23 29 41 43 47

Answer

```
#include <stdio.h>
#include <stdbool.h>

// Function to calculate sum of digits
int digitSum(int num) {
   int sum = 0;
   while(num > 0) {
      sum += num % 10;
      num /= 10;
```

```
    }
    return sum;
}

// Sieve of Eratosthenes
void sieve(bool prime[], int n) {
    for(int i = 0; i <= n; i++) {
        prime[i] = true;
    }

    prime[0] = prime[1] = false;

    for(int i = 2; i * i <= n; i++) {
        if(prime[i]) {
            for(int j = i * i; j <= n; j += i) {
                prime[j] = false;
            }
        }
    }
}

int main() {
    int N;
    scanf("%d", &N);

    bool prime[N + 1];

    // Generate primes up to N
    sieve(prime, N);

    // Check each prime
    for(int i = 2; i <= N; i++) {
        if(prime[i]) {
            int sum = digitSum(i);

            // Check if digit sum is also prime
            if(sum >= 2 && prime[sum]) {
                printf("%d ", i);
            }
        }
    }
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu27499

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 2_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Neha is organizing a sequence of integers where some elements are zeroes. She wants to rearrange the sequence so that all zeroes are moved to the end while keeping the relative order of the non-zero elements intact. Help Neha perform this task efficiently using the two-pointer technique to modify the sequence in place.

Input Format

The first line of input consists of an integer n, representing the size of the array nums[].

The second line contains n integers, representing the elements of the array nums[].

Output Format

vtu27499

The output prints the modified array nums[], with all zeroes moved to the end and the relative order of the non-zero elements maintained.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
0 1 0 3 12

Output: 1 3 12 0 0

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int nums[n];

    // Input array
    for(int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int j = 0; // pointer for non-zero placement

    // Move non-zero elements forward
    for(int i = 0; i < n; i++) {
        if(nums[i] != 0) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
            j++;
        }
    }

    // Output array
    for(int i = 0; i < n; i++) {
        printf("%d ", nums[i]);
```

```
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Arjun has a list of integers representing a specific sequence, and he wants to find its next permutation in lexicographical order. If the sequence is the largest permutation possible, he should rearrange it into the smallest permutation. Help Arjun solve this problem efficiently using the two-pointer technique to rearrange the sequence in place with constant extra memory.

Input Format

The first line of input consists of an integer n, representing the size of the array nums[].

The second line contains n integers, representing the elements of the array nums[].

Output Format

The output prints "Next permutation:" followed by n space-separated integers on the next line, representing the next lexicographically greater permutation of the given sequence.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: Next permutation:
1 3 2

Answer

```
#include <stdio.h>
```

```
// Function to reverse array from index start to end
void reverse(int nums[], int start, int end) {
    while(start < end) {
        int temp = nums[start];
        nums[start] = nums[end];
        nums[end] = temp;
        start++;
        end--;
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int nums[n];

    // Input array
    for(int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int i = n - 2;

    // Step 1: Find first decreasing element from right
    while(i >= 0 && nums[i] >= nums[i + 1]) {
        i--;
    }

    // Step 2: If found, find just greater element and swap
    if(i >= 0) {
        int j = n - 1;
        while(nums[j] <= nums[i]) {
            j--;
        }
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }

    // Step 3: Reverse the remaining part
```

```
    reverse(nums, i + 1, n - 1);

    // Output
    printf("Next permutation:\n");
    for(int k = 0; k < n; k++) {
        printf("%d ", nums[k]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Priya is searching for a specific word (needle) in a large text (haystack). She needs to find the index of the first occurrence of the word in the text. If the word is not present, she should return -1. Help Priya solve this efficiently using the two-pointer technique.

Input Format

The first line of input consists of a string haystack, representing the text in which the word is to be searched.

The second line of input consists of a string needle, representing the word to search for in the text.

Output Format

The output prints "The index of the first occurrence of needle in haystack is: X", where X represents the index value.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: sadbutsad
sad
Output: The index of the first occurrence of needle in haystack is: 0

Answer

```c
#include <stdio.h>
#include <string.h>

int main() {
    char haystack[31], needle[31];

    // Input
    scanf("%s", haystack);
    scanf("%s", needle);

    int n = strlen(haystack);
    int m = strlen(needle);

    int index = -1;

    // Two-pointer approach
    for(int i = 0; i <= n - m; i++) {
        int j = 0;

        while(j < m && haystack[i + j] == needle[j]) {
            j++;
        }

        if(j == m) { // full match found
            index = i;
            break;
        }
    }

    // Output
    printf("The index of the first occurrence of needle in haystack is: %d", index);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Ramesh is working with an array of integers and needs to remove all occurrences of a specific value from it.

To accomplish this efficiently, he uses the Two-Pointer technique, where one pointer scans the array and another pointer tracks the position to place valid elements. Given an integer array nums and an integer val, Ramesh must modify the array in-place so that all elements equal to val are removed while preserving the order of the remaining elements.

Write a program to ensure that the number of remaining elements and the modified array are printed correctly.

Input Format

The first line contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers, representing the elements of the array.

The third line contains an integer val, representing the value to be removed from the array.

Output Format

The first line prints "Number of elements after removal: " followed by count of elements not equal to val.

The second line prints "Modified array:"

Then print the modified array elements (up to index k) separated by a single space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3 2 2 3
3
Output: Number of elements after removal: 2

vtu27499

Modified array:
2 2

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int nums[n];

    // Input array
    for(int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int val;
    scanf("%d", &val);

    int j = 0; // pointer for valid elements

    // Two-pointer logic
    for(int i = 0; i < n; i++) {
        if(nums[i] != val) {
            nums[j] = nums[i];
            j++;
        }
    }

    // Output
    printf("Number of elements after removal: %d\n", j);
    printf("Modified array:\n");

    for(int i = 0; i < j; i++) {
        printf("%d ", nums[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Anil is working with a sorted array of integers and wants to remove duplicate elements efficiently. To solve this problem, he uses the Two-Pointer technique, where one pointer scans the array and another pointer keeps track of the position to store unique elements. Given a sorted integer array nums, Anil must modify the array in-place so that each unique element appears only once.

Write a program to ensure that the number of unique elements and the modified array are printed correctly.

Input Format

The first line contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers, representing the elements of the sorted array.

Output Format

The first line prints "Number of unique elements: k"

On the next line, print "Modified array:"

Then, print the first k elements of the modified array, separated by a single space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8
-1 -1 0 0 1 1 2 2

Output: Number of unique elements: 4
Modified array:
-1 0 1 2

Answer

vtu27499

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int nums[n];

    // Input array
    for(int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    // Edge case
    if(n == 0) {
        printf("Number of unique elements: 0\n");
        printf("Modified array:\n");
        return 0;
    }

    int j = 0; // pointer for unique elements

    // Two-pointer logic
    for(int i = 1; i < n; i++) {
        if(nums[i] != nums[j]) {
            j++;
            nums[j] = nums[i];
        }
    }

    int k = j + 1;

    // Output
    printf("Number of unique elements: %d\n", k);
    printf("Modified array:\n");

    for(int i = 0; i < k; i++) {
        printf("%d ", nums[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu27499

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 2_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Karthik is organizing marbles of three different colors: red, white, and blue. Each color is represented by an integer—0 for red, 1 for white, and 2 for blue. Karthik wants to sort the marbles so that all marbles of the same color are adjacent, and they are arranged in the order red, white, and blue. Help Karthik solve this problem efficiently using the two-pointer technique.

Input Format

The first line of input consists of an integer n, representing the number of marbles in the array nums[].

The second line contains n integers, representing the marbles' colors, where each integer is either 0, 1, or 2.

Output Format

The output prints the sorted array, where all marbles are grouped by color in the order red (0), white (1), and blue (2).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
2 0 2 1 1 0

Output: 0 0 1 1 2 2

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int nums[n];

    // Input
    for(int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int low = 0, mid = 0, high = n - 1;

    // Dutch National Flag Algorithm
    while(mid <= high) {
        if(nums[mid] == 0) {
            int temp = nums[low];
            nums[low] = nums[mid];
            nums[mid] = temp;
            low++;
            mid++;
        }
        else if(nums[mid] == 1) {
            mid++;
        }
        else { // nums[mid] == 2
```

```
            int temp = nums[mid];
            nums[mid] = nums[high];
            nums[high] = temp;
            high--;
        }
    }

    // Output
    for(int i = 0; i < n; i++) {
        printf("%d ", nums[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Kavya is an avid reader who loves finding hidden patterns in texts. One day, she stumbled upon a string of characters and wondered about the longest palindromic substring it contains. Help Kavya identify this substring using an efficient algorithm based on the two-pointer technique.

Input Format

The input consists of a single string s.

Output Format

The output displays "Longest Palindromic Substring: " followed by the longest palindrome found.

Refer the sample output for format specifications.

Sample Test Case

Input: babad
Output: Longest Palindromic Substring: bab

Answer

```c
#include <stdio.h>
#include <string.h>

// Function to expand around center
void expand(char s[], int left, int right, int *start, int *maxLen) {
    while(left >= 0 && right < strlen(s) && s[left] == s[right]) {
        if(right - left + 1 > *maxLen) {
            *maxLen = right - left + 1;
            *start = left;
        }
        left--;
        right++;
    }
}

int main() {
    char s[25];
    scanf("%s", s);

    int start = 0, maxLen = 1;
    int n = strlen(s);

    for(int i = 0; i < n; i++) {
        // Odd length palindrome
        expand(s, i, i, &start, &maxLen);

        // Even length palindrome
        expand(s, i, i + 1, &start, &maxLen);
    }

    // Output
    printf("Longest Palindromic Substring: ");
    for(int i = start; i < start + maxLen; i++) {
        printf("%c", s[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Rohit is analyzing an array of integers to find special combinations of numbers. Using the Two-Pointer technique, he wants to identify all unique triplets in the array whose sum is equal to 0. Given an integer array, Rohit must find and print all such triplets, ensuring that no duplicate triplets are included in the result.

Write a program to ensure that all valid triplets with sum zero are printed correctly.

Example 1

Input:

6

-1 0 1 2 -1 -4

Output:

-1 -1 2

-1 0 1

Explanation:

nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.

nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.

nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.

The distinct triplets are [-1,0,1] and [-1,-1,2].

Notice that the order of the output and the order of the triplets does not matter.

Example 2

Input:

3

0 1 1

vtu27499

Output:

No triplets found with sum 0.

Explanation:

The only possible triplet does not sum up to 0.

Example 3

Input:

3

0 0 0

Output:

0 0 0

Explanation:

The only possible triplet sums up to 0.

Input Format

The first line contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints each unique triplet whose sum is 0 on a separate line.

Each triplet must be printed in ascending order (as they appear after sorting).

If no such triplets exist, print "No triplets found with sum 0."

Refer to the sample output for the formatting specifications.

vtu27499

Sample Test Case

Input: 6
-1 0 1 2 -1 -4

Output: -1 -1 2
-1 0 1

Answer

```
#include <stdio.h>

// Function to sort array (simple bubble sort)
void sort(int arr[], int n) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int nums[n];

    // Input
    for(int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    // Sort array
    sort(nums, n);

    int found = 0;

    // Fix first element
    for(int i = 0; i < n - 2; i++) {
        // Skip duplicates
```

```
        if(i > 0 && nums[i] == nums[i - 1]) continue;

        int left = i + 1;
        int right = n - 1;

        while(left < right) {
            int sum = nums[i] + nums[left] + nums[right];

            if(sum == 0) {
                printf("%d %d %d \n", nums[i], nums[left], nums[right]);
                found = 1;

                // Skip duplicates
                while(left < right && nums[left] == nums[left + 1]) left++;
                while(left < right && nums[right] == nums[right - 1]) right--;

                left++;
                right--;
            }
            else if(sum < 0) {
                left++;
            }
            else {
                right--;
            }
        }
    }

    if(!found) {
        printf("No triplets found with sum 0.");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Rahul is given an array of integers and a target value T. Using the Two-Pointer technique, he wants to find two distinct elements in the array such

that the sum of the pair is closest to the target value. To apply this technique efficiently, the array must first be sorted.

Write a program to ensure that the pair of numbers whose sum is closest to T is identified and printed correctly.

Input Format

The first line contains two space-separated integers n and T, where:

- n is the number of elements in the array
- T is the target sum

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints two space separated integers representing the pair whose sum is closest to the target.

The output pair must be printed in ascending order.

If multiple pairs have the same closest sum, print the first pair found by the two-pointer traversal.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 50
10 22 28 29 30 40
Output: 10 40

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Function to sort array using qsort
int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
```

```
}

int main() {
    int n, T;
    scanf("%d %d", &n, &T);

    int arr[n];

    // Input
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Sort array
    qsort(arr, n, sizeof(int), compare);

    int left = 0, right = n - 1;

    int minDiff = 1000000000;
    int ans1 = arr[0], ans2 = arr[1];

    while(left < right) {
        int sum = arr[left] + arr[right];
        int diff = abs(sum - T);

        // Update closest pair
        if(diff < minDiff) {
            minDiff = diff;
            ans1 = arr[left];
            ans2 = arr[right];
        }

        // Move pointers
        if(sum < T) {
            left++;
        } else {
            right--;
        }
    }

    // Output (already in ascending order)
    printf("%d %d", ans1, ans2);
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu27499

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 2_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

At AquaPark, the maintenance team has a row of vertical water tanks of varying heights arranged side by side along a riverbank.The team wants to identify two tanks that, together with the riverbank, can hold the maximum volume of water between them when the space between them is filled.Each tank's height represents its capacity, and the distance between tanks determines the width of the container formed.

Your task is to read the heights of the tanks and determine the maximum amount of water (volume) that can be contained between any two tanks.

Input Format

The first line of input contains an integer n representing the number of water tanks.

The second line of input contains n space-separated integers representing the heights of the tanks.

Output Format

The output prints a single integer representing the maximum volume of water that can be contained between two tanks.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
Output: 6

Answer

```
#include <stdio.h>

// Function to find minimum of two numbers
int min(int a, int b) {
    return (a < b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int height[n];

    // Input
    for(int i = 0; i < n; i++) {
        scanf("%d", &height[i]);
    }

    int left = 0, right = n - 1;
    int maxArea = 0;

    // Two-pointer approach
    while(left < right) {
```

```
        int h = min(height[left], height[right]);
        int width = right - left;
        int area = h * width;

        if(area > maxArea) {
            maxArea = area;
        }

        // Move pointer with smaller height
        if(height[left] < height[right]) {
            left++;
        } else {
            right--;
        }
    }

    // Output
    printf("%d", maxArea);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Dhanya is tasked with writing a program to merge data from two sensors into a single unified dataset. Each sensor produces an array of data readings over time. The goal is to create a program that takes the data arrays from two sensors, merges them, and produces a single sorted array of merged data readings.

Can you assist Dhanya in this using two-pointer technique?

Input Format

The first line of input consists of an integer n, representing the number of readings produced by the first sensor.

The second line consists of an integer m, representing the number of readings produced by the second sensor.

The third line consists of n space-separated integers, representing the readings from the first sensor.

The fourth line consists of m space-separated integers, representing the readings from the second sensor.

Output Format

The output prints the merged and sorted sensor readings in ascending order, separated by space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
4
1 2 8
3 5 6 7

Output: 1 2 3 5 6 7 8

Answer

```
#include <stdio.h>

int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);

    int a[n], b[m];

    // Input arrays
    for(int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for(int i = 0; i < m; i++) {
        scanf("%d", &b[i]);
    }
```

```
    int i = 0, j = 0;

    // Merge using two pointers
    while(i < n && j < m) {
        if(a[i] <= b[j]) {
            printf("%d ", a[i]);
            i++;
        } else {
            printf("%d ", b[j]);
            j++;
        }
    }

    // Print remaining elements
    while(i < n) {
        printf("%d ", a[i]);
        i++;
    }

    while(j < m) {
        printf("%d ", b[j]);
        j++;
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

At ShopEase, two branches maintain sorted lists of product IDs available in their inventories.The regional manager wants to find the list of product IDs that are common to both branches to plan joint promotions.

Your task is to read the sorted lists of product IDs from both branches, find their intersection and print the common IDs in sorted order.If there are no common products, print "No Common Intersection".

Input Format

The first line of input contains an integer m representing the number of products in the first branch.

The second line of input contains m space-separated integers representing the sorted product IDs in the first branch.

The third line of input contains an integer n representing the number of products in the second branch.

The fourth line of input contains n space-separated integers representing the sorted product IDs in the second branch.

Output Format

The output prints the common product IDs in sorted order, space-separated.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
101 102 103 104 105
6
102 104 106 108 110 115

Output: 102 104

Answer

```
#include <stdio.h>

int main() {
    int m, n;
    scanf("%d", &m);

    int a[m];
    for(int i = 0; i < m; i++) {
        scanf("%d", &a[i]);
    }

    scanf("%d", &n);

    int b[n];
```

```
    for(int i = 0; i < n; i++) {
        scanf("%d", &b[i]);
    }

    int i = 0, j = 0;
    int found = 0;

    // Two-pointer technique
    while(i < m && j < n) {
        if(a[i] == b[j]) {
            printf("%d ", a[i]);
            found = 1;
            i++;
            j++;
        }
        else if(a[i] < b[j]) {
            i++;
        }
        else {
            j++;
        }
    }

    // If no common elements
    if(!found) {
        printf("No Common Intersection");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

At FreshMart Grocery, the cashier needs to quickly find pairs of products whose prices add up to a specific gift card value, so they can suggest bundles to customers using their gift cards efficiently.

You are given a sorted list of product prices. Your task is to find all unique

pairs of products whose prices sum exactly to the given gift card value.If no such pairs exist, print "No Pairs Found".

Input Format

The first line of input contains an integer n representing the number of products.

The second line of input contains n space-separated integers representing the sorted prices of the products.

The third line of input contains an integer k representing the gift card value.

Output Format

The output prints each pair of product prices that sum to k on a separate line, with the two prices separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
2 3 4 5 6 8
10

Output: 2 8
4 6

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    long long arr[n];

    // Input array (already sorted)
    for(int i = 0; i < n; i++) {
        scanf("%lld", &arr[i]);
    }
```

```
    long long k;
    scanf("%lld", &k);

    int left = 0, right = n - 1;
    int found = 0;

    // Two-pointer technique
    while(left < right) {
        long long sum = arr[left] + arr[right];

        if(sum == k) {
            printf("%lld %lld\n", arr[left], arr[right]);
            found = 1;

            // Skip duplicates (optional but safe)
            long long x = arr[left], y = arr[right];
            while(left < right && arr[left] == x) left++;
            while(left < right && arr[right] == y) right--;
        }
        else if(sum < k) {
            left++;
        }
        else {
            right--;
        }
    }

    if(!found) {
        printf("No Pairs Found");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 3_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

Section 1 : MCQ

1. The main idea behind the Knuth-Morris-Pratt algorithm is to:

Answer

Avoid redundant checking of characters that have already been matched

Status : Correct ***Marks : 1/1***

2. The Boyer-Moore algorithm uses which two rules to determine the shift amount?

Answer

Good suffix rule and bad character rule

Status : Correct ***Marks : 1/1***

vtu27499

3. The Boyer-Moore algorithm is particularly efficient when

Answer

The text contains repetitive elements

Status : Wrong ***Marks : 0/1***

4. In the KMP algorithm, if the prefix table at position ii has a value of kk, what does it imply?

Answer

The length of the longest proper prefix which is also a suffix up to position ii is kk.

Status : Correct ***Marks : 1/1***

5. The concept of prefix and suffix is used in which of the following algorithms?

Answer

Knuth-Morris-Pratt (KMP) algorithm

Status : Correct ***Marks : 1/1***

6. What is the main purpose of Manacher's Algorithm?

Answer

To find the longest common subsequence

Status : Wrong ***Marks : 0/1***

7. In the Z-algorithm, the Z-array is used to store:

Answer

The length of the longest substring starting at each position that matches the prefix

Status : Correct ***Marks : 1/1***

vtu27499

8. Which application is most efficiently solved using Z-algorithm?

Answer

Finding all occurrences of a pattern in a text

Status : Correct ***Marks : 1/1***

9. In Z-algorithm, when Z[k] is greater than 0, what does it imply?

Answer

Substring starting at k matches the prefix for length Z[k]

Status : Correct ***Marks : 1/1***

10. Which of the following best explains why Manacher's algorithm is preferable for palindrome problems over expand-around-center approaches?

Answer

Avoids recomputation of overlapping palindromes by mirroring results around the center

Status : Correct ***Marks : 1/1***

11. In bioinformatics applications, KMP is often used instead of Boyer-Moore. Why?

Answer

DNA has only 4 characters, reducing Boyer-Moore's skipping advantage

Status : Correct ***Marks : 1/1***

12. In KMP, the LPS array for pattern "ABABAC" is:

Answer

[0,0,1,2,3,0]

vtu27499

Status : Correct ***Marks : 1/1***

13. Which is true about Manacher's algorithm with respect to handling even-length palindromes?

Answer

Uses a modified string with separators between characters

Status : Correct ***Marks : 1/1***

14. How is Z-algorithm used for pattern search in text efficiently?

Answer

Concatenate P + "$" + T and compute Z-array

Status : Correct ***Marks : 1/1***

15. Which of the following strings has the largest number of palindromic substrings using Manacher’s algorithm?

Answer

"aaaaa"

Status : Correct ***Marks : 1/1***

vtu27499

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 3_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 46.5

Section 1 : Coding

1. Problem Statement

Anil is working on a project that involves counting occurrences of a specific pattern within a given text. He needs a program that efficiently finds and counts all occurrences of a pattern within a given text using the Knuth-Morris-Pratt (KMP) algorithm.

Your task is to help Anil by developing a program that takes a text and a pattern as input and counts the number of times the pattern appears in the text using the KMP algorithm.

Input Format

The first line contains a string txt, representing the text in which Anil wants to search for the pattern.

The second line contains a string pat, representing the pattern Anil wants to search for in the text.

Output Format

The output displays a single integer, representing the count of occurrences of the pattern pat in the text txt.

If the pattern is not found, nothing will be printed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: AABAACAADAABAABA
AABA

Output: 3

Answer

```
#include <stdio.h>
#include <string.h>

// Function to compute LPS array
void computeLPS(char pat[], int m, int lps[]) {
   int len = 0;
   lps[0] = 0;

   int i = 1;
   while(i < m) {
      if(pat[i] == pat[len]) {
         len++;
         lps[i] = len;
         i++;
      } else {
         if(len != 0) {
            len = lps[len - 1];
         } else {
            lps[i] = 0;
            i++;
         }
      }
```

```
    }
}

int main() {
    char txt[60], pat[30];
    scanf("%s", txt);
    scanf("%s", pat);

    int n = strlen(txt);
    int m = strlen(pat);

    int lps[m];
    computeLPS(pat, m, lps);

    int i = 0, j = 0;
    int count = 0;

    // KMP search
    while(i < n) {
        if(txt[i] == pat[j]) {
            i++;
            j++;
        }

        if(j == m) {
            count++;
            j = lps[j - 1]; // allow overlapping matches
        }
        else if(i < n && txt[i] != pat[j]) {
            if(j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }

    // Output
    if(count > 0) {
        printf("%d", count);
    }
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Rohan is developing a text analysis tool to efficiently find repeated patterns inside large documents. To speed up the searching process, he uses the Boyer–Moore string matching algorithm, specifically relying on the Bad Character Heuristic as the main data structure/technique.

Given a text string and a pattern string, write a program to ensure that the task is done by counting how many times the pattern occurs in the text.

Input Format

The first line contains a string representing the text (may contain spaces and special characters).

The second line contains a string representing the pattern to be searched.

Output Format

The output prints a single integer representing the number of occurrences of the pattern in the text.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: abababababa
ab

Output: 4

Answer

```
#include <stdio.h>
#include <string.h>
```

```
#define CHAR_SIZE 256

// Function to build bad character table
void badCharTable(char pat[], int m, int badChar[]) {
    for(int i = 0; i < CHAR_SIZE; i++) {
        badChar[i] = -1;
    }

    for(int i = 0; i < m; i++) {
        badChar[(int)pat[i]] = i;
    }
}

int main() {
    char text[200], pattern[100];

    // Read full line (including spaces)
    fgets(text, sizeof(text), stdin);
    fgets(pattern, sizeof(pattern), stdin);

    // Remove newline characters
    text[strcspn(text, "\n")] = '\0';
    pattern[strcspn(pattern, "\n")] = '\0';

    int n = strlen(text);
    int m = strlen(pattern);

    int badChar[CHAR_SIZE];
    badCharTable(pattern, m, badChar);

    int count = 0;
    int shift = 0;

    // Boyer-Moore search
    while(shift <= (n - m)) {
        int j = m - 1;

        // Match from right to left
        while(j >= 0 && pattern[j] == text[shift + j]) {
            j--;
        }
```

```
        // If pattern found
        if(j < 0) {
            count++;

            // Shift pattern
            if(shift + m < n) {
                shift += m - badChar[(int)text[shift + m]];
            } else {
                shift += 1;
            }
        }
        else {
            int move = j - badChar[(int)text[shift + j]];
            shift += (move > 1) ? move : 1;
        }
    }

    // Output
    printf("%d", count);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Kiran is working on a string analysis module where he needs to identify repeating patterns efficiently. To achieve this, he uses the LPS (Longest Prefix Suffix) array from the KMP string matching algorithm as the main data structure.

Given a string, write a program to ensure that the task is done by finding the length of the longest proper prefix that is also a suffix, ensuring the prefix and suffix do not overlap.

Example

Input:

aabcdaabc

Output:

4

Explanation:

The string "aabc" is the longest prefix which is also a suffix. So the length of the string "aabc" is printed as 4.

Input Format

The input consists of a string S, representing the input string.

Output Format

The output prints an integer representing the length of the longest proper prefix which is also a suffix.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: aabcdaabc

Output: 4

Answer

```
#include <stdio.h>
#include <string.h>

// Function to compute LPS array
void computeLPS(char pat[], int m, int lps[]) {
   int len = 0;
   lps[0] = 0;

   int i = 1;
   while(i < m) {
      if(pat[i] == pat[len]) {
         len++;
         lps[i] = len;
```

```
            i++;
        } else {
            if(len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
}

int main() {
    char s[110];
    scanf("%s", s);

    int n = strlen(s);
    int lps[n];

    // Compute LPS array
    computeLPS(s, n, lps);

    // The last value of LPS gives the answer
    printf("%d", lps[n - 1]);

    return 0;
}
```

Status : Partially correct ***Marks : 6.5/10***

4. Problem Statement

Liam is a software engineer who loves analyzing strings for patterns. One day, while reviewing usernames in a system, he noticed that some usernames read the same backward and forward (palindromes). He wants to count all palindromic sequences of characters in a username efficiently.

To accomplish this, Liam decides to use Manacher's Algorithm, which finds all palindromic substrings in linear time.

vtu27499

Given a string representing the username, help Liam count all the possible palindromic substrings using Manacher's Algorithm.

Input Format

The input consists of a single line containing the string S.

The string consists of lowercase English letters only.

Output Format

The output prints a single integer, representing the total number of palindromic substrings present in the username.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: abaa

Output: 6

Answer

```
#include <stdio.h>
#include <string.h>

// Transform string (add separators)
void preprocess(char *s, char *t) {
    int n = strlen(s);
    int j = 0;

    t[j++] = '^';  // starting sentinel
    for(int i = 0; i < n; i++) {
        t[j++] = '#';
        t[j++] = s[i];
    }
    t[j++] = '#';
    t[j++] = '$';  // ending sentinel
    t[j] = '\0';
}

int main() {
```

```
    char s[105], t[210];
    scanf("%s", s);

    preprocess(s, t);

    int n = strlen(t);
    int P[n];

    int center = 0, right = 0;
    int count = 0;

    for(int i = 1; i < n - 1; i++) {
        int mirror = 2 * center - i;

        if(i < right) {
            P[i] = (P[mirror] < (right - i)) ? P[mirror] : (right - i);
        } else {
            P[i] = 0;
        }

        // Expand palindrome
        while(t[i + 1 + P[i]] == t[i - 1 - P[i]]) {
            P[i]++;
        }

        // Update center and right boundary
        if(i + P[i] > right) {
            center = i;
            right = i + P[i];
        }

        // Count palindromes
        count += (P[i] + 1) / 2;
    }

    // Output
    printf("%d", count);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Aryan is working on a text processing project where he needs to analyze strings to find patterns efficiently. He decides to use the Z-algorithm, which for each position in a string, computes the length of the longest substring starting from that position which is also a prefix of the string.

Your task is to implement a program that computes the Z-array for a given string.

Input Format

The input consists of a single line containing the string S.

S may contain only uppercase and lowercase English letters.

Output Format

The output prints the Z-array of S as a sequence of integers separated by spaces.

The i-th element of the Z-array represents the length of the longest substring starting at position i which matches the prefix of S.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: aabxaab

Output: 0 1 0 0 3 1 0

Answer

```
#include <stdio.h>
#include <string.h>

int main() {
   char s[110];
   scanf("%s", s);

   int n = strlen(s);
```

```
    int Z[n];

    Z[0] = 0; // by definition

    int L = 0, R = 0;

    for(int i = 1; i < n; i++) {
        if(i <= R) {
            int k = i - L;
            Z[i] = (Z[k] < (R - i + 1)) ? Z[k] : (R - i + 1);
        } else {
            Z[i] = 0;
        }

        // Expand match
        while(i + Z[i] < n && s[Z[i]] == s[i + Z[i]]) {
            Z[i]++;
        }

        // Update window
        if(i + Z[i] - 1 > R) {
            L = i;
            R = i + Z[i] - 1;
        }
    }

    // Output Z-array
    for(int i = 0; i < n; i++) {
        printf("%d ", Z[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 3_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Dhiya is a data analyst working for a data compression company. One of her tasks is to analyze logs that are compressed using a simple format where each character is followed by the number of times it appears. For example, "A3B2" represents the string "AAABB".

Her goal is to identify whether a specific pattern exists in the original, uncompressed message and, if so, determine all starting and ending indices of the pattern's occurrence in the expanded string. To perform this efficiently, Dhiya decides to first expand the compressed string and then apply pattern matching.

Your task is to help Dhiya by building a program using Boyer-Moore pattern matching algorithm that performs this operation.

Input Format

The first line of input contains a string s consisting of characters followed by digits representing the compressed text.

The second line of input contains a string p representing the pattern string.

Output Format

The first line of output prints "Expanded Text: X" where X represents the expanded text.

The second line of output prints "Matches found (start_index end_index):" if match found.

For each match of the pattern in the expanded text, print the starting and ending indices (0-based) separated by a space (Print one pair per line).

If no match is found, print "No match found.".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: A3B2C4D1
CCC
Output: Expanded Text: AAABBCCCCD
Matches found (start_index end_index):
5 7
6 8

Answer

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 10000

// Function to expand compressed string
void expand(char *s, char *result) {
    int i = 0, k = 0;
```

```
    while (s[i] != '\0') {
        char ch = s[i++];

        int num = 0;
        while (isdigit(s[i])) {
            num = num * 10 + (s[i] - '0');
            i++;
        }

        for (int j = 0; j < num; j++) {
            result[k++] = ch;
        }
    }
    result[k] = '\0';
}

// Build bad character table
void badCharTable(char *pattern, int badChar[]) {
    for (int i = 0; i < 256; i++)
        badChar[i] = -1;

    for (int i = 0; pattern[i] != '\0'; i++)
        badChar[(int)pattern[i]] = i;
}

// Boyer-Moore search
void boyerMoore(char *text, char *pattern) {
    int n = strlen(text);
    int m = strlen(pattern);

    int badChar[256];
    badCharTable(pattern, badChar);

    int shift = 0;
    int found = 0;

    while (shift <= (n - m)) {
        int j = m - 1;

        while (j >= 0 && pattern[j] == text[shift + j])
            j--;
```

```
        if (j < 0) {
            if (!found) {
                printf("Matches found (start_index end_index):\n");
                found = 1;
            }
            printf("%d %d\n", shift, shift + m - 1);

            shift += (shift + m < n) ? m - badChar[text[shift + m]] : 1;
        } else {
            int move = j - badChar[text[shift + j]];
            shift += (move > 1) ? move : 1;
        }
    }

    if (!found) {
        printf("No match found.\n");
    }
}

int main() {
    char s[1000], p[1000];
    char expanded[MAX];

    scanf("%s", s);
    scanf("%s", p);

    expand(s, expanded);

    printf("Expanded Text: %s\n", expanded);

    boyerMoore(expanded, p);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Sanya is developing a text analysis tool and wants to identify the longest

palindromic sequence in a given string. A palindrome is a string that reads the same forwards and backwards.

Your task is to write a program that takes a string as input and returns the longest palindromic substring it contains. If there are multiple substrings of the same maximum length, returning any one of them is acceptable.

Input Format

The input consists of a single line containing the string S.

S may consist of uppercase and lowercase English letters.

Output Format

The output prints a single string representing the longest palindromic substring found in S.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: babad
Output: bab

Answer

```
#include <stdio.h>
#include <string.h>

// Function to expand around center
void expand(char *s, int left, int right, int *start, int *maxLen) {
   int n = strlen(s);

   while (left >= 0 && right < n && s[left] == s[right]) {
      if (right - left + 1 > *maxLen) {
         *maxLen = right - left + 1;
         *start = left;
      }
      left--;
      right++;
   }
```

```
}

int main() {
    char s[105];
    scanf("%s", s);

    int n = strlen(s);
    int start = 0, maxLen = 1;

    for (int i = 0; i < n; i++) {
        // Odd length palindrome
        expand(s, i, i, &start, &maxLen);

        // Even length palindrome
        expand(s, i, i + 1, &start, &maxLen);
    }

    // Print result
    for (int i = start; i < start + maxLen; i++) {
        printf("%c", s[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Rita is working on string pattern matching using the Z-Algorithm. She wants to find all starting indices where a given pattern string occurs in a text string efficiently.

Write a program that takes a text string and a pattern string as input and outputs a list of indices (0-based) where the pattern occurs. If the pattern does not occur in the text, output an empty list.

Input Format

The first line contains the text string txt.

The second line contains the pattern string pat.

Output Format

The output prints a list of integers representing the starting indices of all occurrences of the pattern in the text.

If no match is found, print [].

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abesdu
edu
Output: []

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX 2000

// Function to compute Z-array
void computeZ(char *str, int Z[]) {
   int n = strlen(str);
   int L = 0, R = 0;

   for (int i = 1; i < n; i++) {
      if (i <= R)
         Z[i] = (R - i + 1 < Z[i - L]) ? (R - i + 1) : Z[i - L];
      else
         Z[i] = 0;

      while (i + Z[i] < n && str[Z[i]] == str[i + Z[i]])
         Z[i]++;

      if (i + Z[i] - 1 > R) {
         L = i;
         R = i + Z[i] - 1;
      }
```

```
    }
}

int main() {
    char txt[1000], pat[100];
    char combined[MAX];

    scanf("%s", txt);
    scanf("%s", pat);

    // Create combined string: pat + "$" + txt
    sprintf(combined, "%s$%s", pat, txt);

    int Z[MAX] = {0};
    computeZ(combined, Z);

    int patLen = strlen(pat);
    int totalLen = strlen(combined);

    int found = 0;

    printf("[");

    for (int i = 0; i < totalLen; i++) {
        if (Z[i] == patLen) {
            if (found) printf(", ");
            printf("%d", i - patLen - 1);
            found = 1;
        }
    }

    printf("]\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Neha is building a plagiarism-detection feature where she must compare

two text strings to find similarities. To efficiently detect overlaps between the strings, she uses the KMP (Knuth–Morris–Pratt) string matching algorithm, relying on the LPS (Longest Prefix Suffix) array as the main data structure.

Given two strings, write a program to ensure that the task is done by finding the longest common substring present in both strings. If no common substring exists, the program should clearly indicate that result.

Input Format

The first line contains a string representing the first text.

The second line contains a string representing the second text.

Output Format

The output prints the longest common substring if it exists.

If there is no common substring, print "There is no common substring".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abcdef
abc
Output: abc

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX 105

// Build LPS array
void computeLPS(char *pat, int lps[]) {
   int len = 0;
   lps[0] = 0;
```

```
    int i = 1;
    while (pat[i] != '\0') {
        if (pat[i] == pat[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
}

// KMP search: returns 1 if pattern found
int KMPSearch(char *txt, char *pat) {
    int n = strlen(txt);
    int m = strlen(pat);

    int lps[MAX];
    computeLPS(pat, lps);

    int i = 0, j = 0;

    while (i < n) {
        if (txt[i] == pat[j]) {
            i++;
            j++;
        }

        if (j == m) {
            return 1; // found
        } else if (i < n && txt[i] != pat[j]) {
            if (j != 0)
                j = lps[j - 1];
            else
                i++;
        }
    }
```

```
    return 0;
}

int main() {
    char s1[MAX], s2[MAX];
    scanf("%s", s1);
    scanf("%s", s2);

    int len1 = strlen(s1);
    int len2 = strlen(s2);

    // Ensure s1 is shorter string
    if (len1 > len2) {
        char temp[MAX];
        strcpy(temp, s1);
        strcpy(s1, s2);
        strcpy(s2, temp);

        int t = len1;
        len1 = len2;
        len2 = t;
    }

    char result[MAX] = "";
    int found = 0;

    // Try all substring lengths (longest first)
    for (int len = len1; len > 0; len--) {
        for (int i = 0; i <= len1 - len; i++) {
            char sub[MAX];
            strncpy(sub, s1 + i, len);
            sub[len] = '\0';

            if (KMPSearch(s2, sub)) {
                printf("%s\n", sub);
                found = 1;
                return 0;
            }
        }
    }

    if (!found) {
```

```
        printf("There is no common substring\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 3_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 39

Section 1 : Coding

1. Problem Statement

Suresh is working on a text-processing system where he needs to efficiently count how many times a smaller pattern appears inside a larger string. To achieve fast pattern matching, he uses the Z-Algorithm, with the Z-array as the main data structure.

Given multiple test cases, each containing a text string and a pattern string, write a program to ensure that the task is done by computing the number of occurrences of the pattern in the text using the Z-algorithm approach.

For example:

If S = "ababa", and P = "ab", then "ab" occurs twice in "ababa".

Input Format

The first line of input contains T, representing the number of test cases.

For each test case:

Line 1 Contains two space separated integers N and M representing the length of the text and pattern respectively.

Line 2: Contains string S of length N representing the text.

Line 3: A string P of length M representing the pattern.

Output Format

For each test case, print a single integer on a new line indicating the number of times the pattern occurs in the text.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1
10 2
abababababa
ab
Output: 4

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX 205

// Compute Z-array
void computeZ(char *str, int Z[]) {
   int n = strlen(str);
   int L = 0, R = 0;

   for (int i = 1; i < n; i++) {
      if (i <= R)
```

```
            Z[i] = (R - i + 1 < Z[i - L]) ? (R - i + 1) : Z[i - L];
        else
            Z[i] = 0;

        while (i + Z[i] < n && str[Z[i]] == str[i + Z[i]])
            Z[i]++;

        if (i + Z[i] - 1 > R) {
            L = i;
            R = i + Z[i] - 1;
        }
    }
}

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int N, M;
        scanf("%d %d", &N, &M);

        char text[105], pattern[105];
        scanf("%s", text);
        scanf("%s", pattern);

        char combined[MAX];
        sprintf(combined, "%s$%s", pattern, text);

        int Z[MAX] = {0};
        computeZ(combined, Z);

        int count = 0;
        int patLen = strlen(pattern);
        int totalLen = strlen(combined);

        for (int i = 0; i < totalLen; i++) {
            if (Z[i] == patLen)
                count++;
        }

        printf("%d\n", count);
```

```
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Arjun is working as an NLP engineer to improve the smart assistant chatbot in his company. His team wants the chatbot to understand user emotions better, especially when users repeat or emphasize words.

They noticed that palindromic patterns (like "wow", "pop", "madam") in messages can help detect such emotions or expressions.

To help with this, Arjun is asked to write a program that finds out how many palindromic substrings are centered at each character in the user's message.

Since the message could be long, Arjun decides to use Manacher's Algorithm for fast and efficient processing.

Example:

Input

bananas

Output

1 1 3 5 3 1 1

Explanation

At the first character 'b', the only palindrome is "b" itself length 1At 'a', only "a" is a palindrome length 1At 'n', we get "ana" which is a palindrome centered at 'n' length 3At the middle 'a', we get "anana" which is a bigger palindrome length 5At the next 'n', again "ana" is a palindrome length 3At the second 'a', only "a" length 1Finally at 's', only "s" length 1

Input Format

The first line of input contains a string s, representing a message typed by a user in the chatbot conversation.

Output Format

The output prints the length of the longest palindromic substring centered at each character in the input string, separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: ab
Output: 1 1

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX 1005

int main() {
    char s[MAX];
    scanf("%s", s);

    int n = strlen(s);
    int d1[MAX]; // odd-length palindrome radius

    int l = 0, r = -1;

    for (int i = 0; i < n; i++) {
        int k;

        if (i > r)
            k = 1;
        else
            k = (d1[l + r - i] < (r - i + 1)) ? d1[l + r - i] : (r - i + 1);

        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k])
            k++;
```

```
        d1[i] = k;

        if (i + k - 1 > r) {
            l = i - k + 1;
            r = i + k - 1;
        }
    }

    // Output result: length = 2*d1[i] - 1
    for (int i = 0; i < n; i++) {
        printf("%d ", 2 * d1[i] - 1);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Given a text txt [0 . . . N-1] and a pattern pat [0 . . . M-1], write a function search (char pat [], char txt []) that prints all occurrences of pat [] in txt []. You may assume that N > M. Use zero-based indexing while returning the indices.

Example:

Input: txt [] = “THIS IS A TEST TEXT”, pat [] = “TEST”

Output: Pattern found at index 10

Input: txt [] = “AABAACAADAABAABA”, pat [] = “AABA”

Output: Pattern found at index 0, Pattern found at index 9, Pattern found at index 12

Input Format

The first line of input consists of a string txt, in which the pattern is to be searched.

The second line of input consists of a string pat, the pattern that needs to be found in the text.

Output Format

For each occurrence of the pattern in the text, print "Pattern found at index X" in a new line, where X the indices where the pattern is found within the text string.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: THIS IS A TEST TEXT
TEST
Output: Pattern found at index 10

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX 100

// Build LPS array
void computeLPS(char *pat, int lps[]) {
    int len = 0;
    lps[0] = 0;

    int i = 1;
    int m = strlen(pat);

    while (i < m) {
        if (pat[i] == pat[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
```

```
            i++;
        }
    }
  }
}

// KMP search function
void search(char *pat, char *txt) {
    int m = strlen(pat);
    int n = strlen(txt);

    int lps[MAX];
    computeLPS(pat, lps);

    int i = 0, j = 0;

    while (i < n) {
        if (pat[j] == txt[i]) {
            i++;
            j++;
        }

        if (j == m) {
            printf("Pattern found at index %d\n", i - j);
            j = lps[j - 1];
        } else if (i < n && pat[j] != txt[i]) {
            if (j != 0)
                j = lps[j - 1];
            else
                i++;
        }
    }
}

int main() {
    char txt[100], pat[20];

    // Read full line (including spaces)
    fgets(txt, sizeof(txt), stdin);
    fgets(pat, sizeof(pat), stdin);

    // Remove newline characters
```

```
    txt[strcspn(txt, "\n")] = 0;
    pat[strcspn(pat, "\n")] = 0;

    search(pat, txt);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Ritika is a software engineer working on an AI-based spam detection system for a popular email client. To efficiently scan email content, she uses the Boyer–Moore string matching algorithm, specifically the Bad Character Heuristic as the main data structure/technique.

Given an email message and a list of spam keywords, write a program to ensure that the task is done by detecting all keywords present in the email (case-insensitive), reporting their first occurrence positions, and computing a spam score based on the number of detected keywords.

Input Format

The first line of input contains a string representing the email content (may contain spaces and special characters).

The second line contains an integer n , representing the number of spam keywords.

The next n lines each contain a spam keyword.

Output Format

The first line prints a string "Detected Spam Keywords:"

For each detected keyword, print "<index>: "<keyword>" at position <starting_index>" where indexing starts from 0.

The final line prints "Spam Score: x/y" where x is the number of detected keywords and y is the total number of keywords.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: Buy this book now and get another free!
3
buy
now
free

Output: Detected Spam Keywords:
1: "buy" at position 0
2: "now" at position 14
3: "free" at position 34
Spam Score: 3/3

Answer

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 1005

// Convert string to lowercase
void toLower(char *s) {
    for (int i = 0; s[i]; i++)
        s[i] = tolower(s[i]);
}

// Build bad character table
void badCharTable(char *pat, int badChar[]) {
    for (int i = 0; i < 256; i++)
        badChar[i] = -1;

    for (int i = 0; pat[i]; i++)
        badChar[(int)pat[i]] = i;
}

// Boyer-Moore search (first occurrence)
int boyerMoore(char *txt, char *pat) {
    int n = strlen(txt);
```

```
    int m = strlen(pat);

    int badChar[256];
    badCharTable(pat, badChar);

    int shift = 0;

    while (shift <= (n - m)) {
        int j = m - 1;

        while (j >= 0 && pat[j] == txt[shift + j])
            j--;

        if (j < 0) {
            return shift; // first occurrence
        } else {
            int move = j - badChar[(int)txt[shift + j]];
            shift += (move > 1) ? move : 1;
        }
    }

    return -1; // not found
}

int main() {
    char text[MAX];
    fgets(text, sizeof(text), stdin);
    text[strcspn(text, "\n")] = 0;

    int n;
    scanf("%d", &n);
    getchar(); // consume newline

    char originalText[MAX];
    strcpy(originalText, text);

    toLower(text);

    char keywords[100][100];
    int foundCount = 0;
    int positions[100];
```

```
    for (int i = 0; i < n; i++) {
        fgets(keywords[i], sizeof(keywords[i]), stdin);
        keywords[i][strcspn(keywords[i], "\n")] = 0;
    }

    printf("Detected Spam Keywords:\n");

    for (int i = 0; i < n; i++) {
        char temp[100];
        strcpy(temp, keywords[i]);
        toLower(temp);

        int pos = boyerMoore(text, temp);

        if (pos != -1) {
            printf("%d: \"%s\" at position %d\n", i + 1, keywords[i], pos);
            foundCount++;
        }
    }

    if (foundCount == 0) {
        printf("No spam keywords detected.\n");
    }

    printf("Spam Score: %d/%d\n", foundCount, n);

    return 0;
}
```

Status : Partially correct ***Marks : 9/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 4_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 10

Section 1 : MCQ

1. The user performs the following operations on a stack of size 5 then, which of the following is the correct statement for the 1stack?

push(1);
pop();
push(2);
push(3);
pop();
push(2);
pop();
pop();
push(4);
pop();
pop();
push(5);

vtu27499

Answer

Overflow occurs

Status : Wrong ***Marks : 0/1***

2. If the sequence of operations - push (1), push (2), pop, push (1), push (2), pop, pop, pop, push (2), pop are performed on a stack, the sequence of popped out values

Answer

2,2,1,1,2

Status : Correct ***Marks : 1/1***

3. What is the result of the following operation?

Top (Push (S, X))

Answer

X

Status : Correct ***Marks : 1/1***

4. Stack A has the entries a, b, c (with a on top). Stack B is empty. An entry popped out of stack A can be printed immediately or pushed to stack B. An entry popped out of the stack B can only be printed. In this arrangement, which of the following permutations of a, b, c are not possible?

Answer

c a b

Status : Correct ***Marks : 1/1***

5. Pushing an element into the stack already has five elements. The stack size is 5, then the stack becomes

Answer

Overflow

Status : Correct ***Marks : 1/1***

6. In a Stack, if a user tries to remove an Element from Empty stack it is called

Answer

Underflow

Status : Correct ***Marks : 1/1***

7. The user performs the following operations on the stack of size 5 then at the end of the last operation, the total number of elements present in the stack is

```
push(1);
pop();
push(2);
push(3);
pop();
push(4);
pop();
pop();
push(5);
```

Answer

1

Status : Correct ***Marks : 1/1***

8. What will be the output after performing these sequence of operations

```
push(20);
push(4);
top();
pop();
```

```
pop();
pop();
push(5);
top();
```

Answer

20

Status : Wrong ***Marks : 0/1***

9. A normal queue, if implemented using an array of size MAX_SIZE, gets full when _____.

Answer

rear = MAX_SIZE – 1

Status : Correct ***Marks : 1/1***

10. If the elements 'A', 'B', 'C', and 'D' are placed in a queue and are deleted one at a time, in what order will they be removed?

Answer

ABCD

Status : Correct ***Marks : 1/1***

11. After performing this set of operations, what does the final queue contain?

```
InsertFront(10);
InsertFront(20);
InsertRear(30);
DeleteFront();
InsertRear(40);
InsertRear(10);
DeleteRear();
InsertRear(15);
display();
```

Answer

20 30 40 15

Status : Wrong ***Marks : 0/1***

12. What is the purpose of defining the constant SIZE in a queue implementation using an array?

Answer

To limit the maximum number of elements in the queue

Status : Correct ***Marks : 1/1***

13. What does the condition front = NULL indicate in inserting an element into a queue?

Answer

The queue is empty

Status : Correct ***Marks : 1/1***

14. Which of the following is a valid condition for a successful dequeue operation in the array implementation of a queue?

Answer

Rear > 0

Status : Wrong ***Marks : 0/1***

15. The essential condition that is checked before insertion in a queue is?

Answer

Underflow

Status : Wrong ***Marks : 0/1***

vtu27499

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 4_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Neo is experimenting with text transformations and decides to use a stack to reverse the sequence of words in his sentence. He provides a line of text and expects the program to utilize a stack to reorder the words so that the last word appears first and the first word appears last.

Write a program to ensure this is done correctly using a stack-based approach.

Example

Input: I am Neo

Output: Neo am I

vtu27499

Input Format

The input consists of a single line of input containing a sentence with words separated by spaces.

Output Format

The output prints a single line of output containing the words of the sentence reversed in order, separated by a single space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: Hello World
Output: World Hello

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX 100
#define MAX_WORDS 50

int main() {
    char sentence[MAX];
    char *stack[MAX_WORDS];
    int top = -1;

    // Read input sentence
    fgets(sentence, sizeof(sentence), stdin);
    sentence[strcspn(sentence, "\n")] = 0; // remove newline

    // Split sentence into words
    char *word = strtok(sentence, " ");
    while (word != NULL) {
        stack[++top] = word;  // push word onto stack
        word = strtok(NULL, " ");
    }

    // Pop words and print in reverse order
```

```
    while (top >= 0) {
        printf("%s", stack[top--]);
        if (top >= 0) printf(" ");
    }
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Give a string, remove duplicate letters from the string s such that each letter appears just once using Stack. The lexicographical order of your result must be the least of all feasible outcomes.

Input Format

The input consists of a string.

Output Format

The output displays the string in the lexicographical order with unique characters.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: bcabc
Output: abc

Answer

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

#define MAX 27
```

```
int main() {
    char s[MAX];
    fgets(s, sizeof(s), stdin);
    s[strcspn(s, "\n")] = 0;  // remove newline

    int last[26] = {0};    // store last occurrence index
    bool seen[26] = {0};   // track if character is in stack
    int len = strlen(s);

    for (int i = 0; i < len; i++) {
        last[s[i] - 'a'] = i;
    }

    char stack[MAX];
    int top = -1;

    for (int i = 0; i < len; i++) {
        int c = s[i] - 'a';
        if (seen[c]) continue;  // skip if already in stack

        // maintain lexicographical order
        while (top >= 0 && stack[top] > s[i] && last[stack[top] - 'a'] > i) {
            seen[stack[top] - 'a'] = false;  // remove from seen
            top--;                           // pop stack
        }

        stack[++top] = s[i];
        seen[c] = true;
    }

    stack[top + 1] = '\0';
    printf("%s\n", stack);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Mira is learning about palindromes and wants to build a program that can

check whether a given string is a palindrome using a stack-based approach. A palindrome reads the same backward as forward.

Write a program to ensure that the task is done by pushing characters to a stack and comparing them in reverse order to determine if the string is a palindrome.

Input Format

The input contains a single line containing a string with space-separated characters.

Output Format

The output consists of ,

If the string is a palindrome, print "It's a palindrome!"

Otherwise, print "Not a palindrome"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: madam
Output: It's a palindrome!

Answer

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

int main() {
    char str[MAX];
    char stack[MAX];
    int top = -1;

    // Read input
    fgets(str, sizeof(str), stdin);
```

vtu27499

```
    str[strcspn(str, "\n")] = 0; // remove newline

    // Normalize string: remove spaces and convert to lowercase
    char normalized[MAX];
    int n = 0;
    for (int i = 0; str[i]; i++) {
        if (isalpha(str[i])) {
            normalized[n++] = tolower(str[i]);
        }
    }
    normalized[n] = '\0';

    // Push all characters into stack
    for (int i = 0; i < n; i++) {
        stack[++top] = normalized[i];
    }

    // Compare by popping from stack
    int isPalindrome = 1;
    for (int i = 0; i < n; i++) {
        if (stack[top--] != normalized[i]) {
            isPalindrome = 0;
            break;
        }
    }

    if (isPalindrome) {
        printf("It's a palindrome!\n");
    } else {
        printf("Not a palindrome\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Gokul is working on a program to rearrange elements in a queue using an array. Given a queue of integers, he needs to modify it by interleaving its

first half with the second half. Write a program to help Gokul achieve this task.

Note: Interleaving involves merging elements from two separate halves in an alternating fashion to create a new sequence.

Example

Input

6

1 2 3 7 9 5

Output

1 7 2 9 3 5

Explanation

The interleave rearranges elements in a queue by alternating between two halves. For instance, given input [1, 2, 3, 7, 9, 5], it splits into [1, 2, 3] and [7, 9, 5], then interleaves them as [1, 7, 2, 9, 3, 5]. It achieves this by splitting the queue into two halves and then merging them interleaved.

Input Format

The first line of the input consists of an integer n, representing the number of elements in the queue.

The second line consists of n space-separated integers, representing the queue elements.

Output Format

The output prints n space-separated integers, representing the rearranged queue after interleaving its first half with the second half.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

vtu27499

1 2 3 7 9 5

Output: 1 7 2 9 3 5

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int queue[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &queue[i]);
    }

    int half = n / 2;

    // Interleave the two halves
    for (int i = 0; i < half; i++) {
        printf("%d %d", queue[i], queue[i + half]);
        if (i != half - 1) printf(" ");
    }
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Sanjay is learning about queues in his data structures class and he wants to write a program that counts the number of distinct elements in the queue. Your task is to guide him in this.

Example

Input:

7

45 33 29 45 45 46 39

Output:

5

Explanation:

The distinct elements in the queue are 45, 33, 29, 46, 39. So, the count is 5.

Input Format

The first line of input consists of an integer M, representing the number of elements in the queue.

The second line consists of M space-separated integers, representing the queue elements.

Output Format

The output prints an integer, representing the number of distinct elements in the queue.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
30 24 43 31 30
Output: 4

Answer

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int M;
    scanf("%d", &M);

    int queue[100];
    for (int i = 0; i < M; i++) {
        scanf("%d", &queue[i]);
    }
```

```
    int distinctCount = 0;

    for (int i = 0; i < M; i++) {
        bool alreadySeen = false;
        for (int j = 0; j < i; j++) {
            if (queue[i] == queue[j]) {
                alreadySeen = true;
                break;
            }
        }
        if (!alreadySeen) {
            distinctCount++;
        }
    }

    printf("%d\n", distinctCount);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 4_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

A financial analytics company is developing a tool to analyze stock price trends. The tool needs to identify the Next Greater Price for each stock price in a given list of daily prices. For a given stock price, the Next Greater Price is defined as the first price appearing after it in the list that is greater. If no such price exists, the result should be -1.

Note: The Next greater Element for an element x is the first greater element on the right side of x in the array. Elements for which no greater element exist, consider the next greater element as -1.

Examples:

vtu27499

Input Format

The first line of input consists of an integer n, representing the size of the array.

The second line consists of n space-separated stack of integers.

Output Format

The output prints "Next Greater Elements: " followed by an array of integers, representing the next greater elements in the stack.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
4 5 2 25

Output: Next Greater Elements: 5 25 25 -1

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[12], nge[12];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        nge[i] = -1;  // initialize all as -1
    }

    // Simple stack-based approach
    int stack[12], top = -1;

    for (int i = 0; i < n; i++) {
        while (top != -1 && arr[i] > arr[stack[top]]) {
            nge[stack[top--]] = arr[i];
        }
        stack[++top] = i;
    }
```

```
    printf("Next Greater Elements: ");
    for (int i = 0; i < n; i++) {
        printf("%d", nge[i]);
        if (i != n - 1) printf(" ");
    }
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement:

Riya is working on a text-cleaning utility where certain unwanted patterns must be removed from user input. She uses a stack data structure to process the string character by character.

Given a string containing alphabets, digits, and special characters, write a program to ensure that the task is done by removing every digit along with the closest non-digit character immediately to its left. The remaining characters should be preserved in their original order and printed as the final cleaned string.

Example:

Input:

s = "cb34"

Output:

""

Explanation:

Start with an empty stack.

Push c stack: ['c']

Push b stack: ['c','b']

Encounter 3 pop 'b' stack: ['c']

Encounter 4 pop 'c' stack: []

Result: empty string

Input Format

The input consists of a string s without spaces, consisting of letters, digits, and special characters.

Output Format

The output prints the resulting string after performing the required removals.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abc

Output: abc

Answer

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main() {
    char s[101];
    scanf("%s", s);

    char stack[101];
    int top = -1;

    for (int i = 0; s[i]; i++) {
        if (isdigit(s[i])) {
            if (top != -1) top--;  // remove closest non-digit character
            // digit itself is ignored
        } else {
            stack[++top] = s[i];  // push non-digit character
        }
    }
```

```
    stack[top + 1] = '\0';  // terminate string
    printf("%s\n", stack);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Kavya is developing a real-time analytics system that processes a continuous stream of numerical data. To efficiently track trends, she uses a Queue data structure to maintain incoming values and compute statistics incrementally.

Given a sequence of integers arriving one by one, write a program to ensure that the task is done by calculating and printing the moving average of all elements received so far after each new input. The average should be displayed up to two decimal places.

Example

Input

3

1 2 3

Output

1.00 1.50 2.00

Explanation

The moving average of the first element is 1.00.

The next elements are 1 and 2, the moving average is (1 + 2) / 2 = 1.50.

The next elements are 1, 2, and 3, the moving average is (1 + 2 + 3) / 3 = 2.00.

Input Format

The first line contains an integer n, representing the number of elements in the data stream.

The second line contains n space-separated integers representing the incoming values.

Output Format

The output prints n floating-point numbers, each representing the moving average after processing the corresponding element.

Each value should be printed with two decimal places, separated by spaces.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3

Output: 1.00 1.50 2.00

Answer

```
#include <stdio.h>

int main() {
   int n;
   scanf("%d", &n);

   int arr[15];
   double sum = 0.0;

   for (int i = 0; i < n; i++) {
      scanf("%d", &arr[i]);
      sum += arr[i];
      printf("%.2lf", sum / (i + 1));
      if (i != n - 1) printf(" ");
   }
   printf("\n");

   return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

In a small business, Jane is tasked with organizing customer feedback scores for a new product. Each customer has rated the product, and Jane needs to arrange these ratings in a specific order to analyze the feedback more effectively.

Specifically, she wants to rearrange the ratings such that the most frequent ratings appear first. If two ratings have the same frequency, the smaller rating should come first. To accomplish this, assist Jane with a program to process a queue of customer ratings and rearrange them based on their frequencies.

Input Format

The first line contains an integer n, which represents the number of customer ratings.

The second line contains n integers, each representing a customer rating.

Output Format

The output displays the rearranged ratings in the order of their frequency (most frequent first).

If frequencies are the same, the smaller rating should come first.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7
2 3 2 3 2 3 2

Output: 2 2 2 2 3 3 3

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>

typedef struct {
    int rating;
    int freq;
} Rating;

int cmp(const void *a, const void *b) {
    Rating *r1 = (Rating *)a;
    Rating *r2 = (Rating *)b;
    if (r1->freq != r2->freq)
        return r2->freq - r1->freq; // descending frequency
    return r1->rating - r2->rating; // ascending rating if freq same
}

int main() {
    int n;
    scanf("%d", &n);
    int arr[20];
    int counts[100] = {0};

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        counts[arr[i]]++;
    }

    Rating ratings[100];
    int k = 0;
    for (int i = 1; i <= 99; i++) {
        if (counts[i] > 0) {
            ratings[k].rating = i;
            ratings[k].freq = counts[i];
            k++;
        }
    }

    qsort(ratings, k, sizeof(Rating), cmp);

    for (int i = 0; i < k; i++) {
        for (int j = 0; j < ratings[i].freq; j++) {
            printf("%d ", ratings[i].rating);
        }
```

```
    }
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 4_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Rohit is learning combinatorics and wants to generate all possible selections from a given set of numbers. To systematically explore every combination, he uses a Stack data structure along with a bitwise technique.

Given a set of integers, write a program to ensure that the task is done by generating and printing all possible subsets (the power set) of the given set, including the empty set. Each subset must be printed in set notation.

Example

Input:

3

vtu27499

1 2 3

Output:

{}

{1}

{2}

{1, 2}

{3}

{1, 3}

{2, 3}

{1, 2, 3}

Input Format

The first line of input consists of an integer n, representing the number of elements in the set.

The second line consists of n space-separated integers representing the elements of the set.

Output Format

The output prints all possible subsets of the given set.

Each subset should be printed on a new line in the format {element1, element2, ...}.

The empty subset should be printed as {}.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: {}

vtu27499

{1}
{2}
{1, 2}
{3}
{1, 3}
{2, 3}
{1, 2, 3}

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    int arr[10];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int total = 1 << n; // 2^n subsets

    for (int mask = 0; mask < total; mask++) {
        printf("{");
        int first = 1; // to handle commas
        for (int j = 0; j < n; j++) {
            if (mask & (1 << j)) {
                if (!first) printf(", ");
                printf("%d", arr[j]);
                first = 0;
            }
        }
        printf("}\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given an array of integers, the task is to find the maximum absolute

difference between the nearest left and the right smaller element of every element in the array.

Note: If there is no smaller element on right side or left side of any element then we take zero as the smaller element. For example for the leftmost element, the nearest smaller element on the left side is considered as 0. Similarly, for rightmost elements, the smaller element on the right side is considered as 0.

Examples:

Input: arr[] = {2, 1, 8}

Output: 1

Left smaller LS[] {0, 0, 1}

Right smaller RS[] {1, 0, 0}

Maximum Diff of abs(LS[i] - RS[i]) = 1

Input: arr[] = {2, 4, 8, 7, 7, 9, 3}

Output: 4

Left smaller LS[] = {0, 2, 4, 4, 4, 7, 2}

Right smaller RS[] = {0, 3, 7, 3, 3, 3, 0}

Maximum Diff of abs(LS[i] - RS[i]) = 7 - 3 = 4

Input: arr[] = {5, 1, 9, 2, 5, 1, 7}

Output: 1

Input Format

The first line contains an integer n, representing the size of the array.

The second line contains n space-separated integers, representing the array elements.

Output Format

The output prints a single integer, representing the maximum absolute difference between the left and right smaller elements for any element in the array.

vtu27499

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
2 1 8

Output: 1

Answer

```
#include <stdio.h>
#include <stdlib.h>

int maxAbsDiff(int arr[], int n) {
    int LS[10], RS[10];
    int stack[10], top;

    // Nearest smaller to left
    top = -1;
    for(int i = 0; i < n; i++) {
        while(top >= 0 && stack[top] >= arr[i]) top--;
        LS[i] = (top == -1) ? 0 : stack[top];
        stack[++top] = arr[i];
    }

    // Nearest smaller to right
    top = -1;
    for(int i = n-1; i >= 0; i--) {
        while(top >= 0 && stack[top] >= arr[i]) top--;
        RS[i] = (top == -1) ? 0 : stack[top];
        stack[++top] = arr[i];
    }

    // Maximum absolute difference
    int maxDiff = 0;
    for(int i = 0; i < n; i++) {
        int diff = abs(LS[i] - RS[i]);
        if(diff > maxDiff) maxDiff = diff;
    }
```

```
    return maxDiff;
}

int main() {
    int n;
    scanf("%d", &n);
    int arr[10];
    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("%d\n", maxAbsDiff(arr, n));
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

A logistics company uses a queue data structure to manage the order of processing packages at their sorting center. Each package is identified by a unique code represented as an integer.

Due to a system update, the company needs to reverse the order in which the packages are processed without changing the sequence of processing for any new packages that arrive after the reversal.

The task is to implement a system that enqueues package codes, prints the original processing order, reverse the queue using a stack, and then prints the reversed processing order.

Input Format

The first line contains an integer capacity, the number of packages initially in the queue.

The second line contains capacity space-separated integers representing the package codes.

Output Format

The first line prints "Original queue:" followed by the package codes in their

original order.

The second line prints "Reversed queue:" followed by the package codes in their reversed order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
Output: Original queue: 1 2 3 4 5
Reversed queue: 5 4 3 2 1

Answer

```
#include <stdio.h>

int main() {
    int capacity;
    scanf("%d", &capacity);
    int queue[100]; // queue array
    int stack[100]; // stack array
    int top = -1;

    // Read package codes
    for(int i = 0; i < capacity; i++) {
        scanf("%d", &queue[i]);
    }

    // Print original queue
    printf("Original queue:");
    for(int i = 0; i < capacity; i++) {
        printf(" %d", queue[i]);
    }
    printf(" \n");

    // Push all elements to stack
    for(int i = 0; i < capacity; i++) {
        stack[++top] = queue[i];
    }
```

```
    // Pop from stack to get reversed queue
    printf("Reversed queue:");
    while(top >= 0) {
        printf(" %d", stack[top--]);
    }
    printf(" \n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Alex is working on a customer support system that handles a queue of incoming service requests. The system needs to manage the requests efficiently by enqueue, ensuring that requests are processed in the order they arrive.

To improve processing efficiency, Alex wants to implement a feature that allows moving the last request to the front of the queue when necessary. This operation helps prioritize the most recent request for urgent handling.

Your task is to implement this feature and ensure that the queue is updated accordingly.

Input Format

The first line of input contains a single integer n, representing the number of service requests.

The second line contains n integers, each representing a service request ID, separated by spaces.

Output Format

The output displays the queue after moving the last request to the front with request IDs separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
62 85 47 98

Output: 98 62 85 47

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    int queue[50];

    // Read the service requests
    for(int i = 0; i < n; i++) {
        scanf("%d", &queue[i]);
    }

    // Store the last request
    int last = queue[n-1];

    // Shift all other elements one step to the right
    for(int i = n-1; i > 0; i--) {
        queue[i] = queue[i-1];
    }

    // Place the last request at the front
    queue[0] = last;

    // Print the updated queue
    for(int i = 0; i < n; i++) {
        printf("%d ", queue[i]);
    }
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu27499

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 5_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

Section 1 : MCQ

1. After adding a new node at the end of a linked list, what does the current last node become?

Answer

The second last node

Status : Correct ***Marks : 1/1***

2. When inserting a new element at the beginning of a linked list, what happens to the current head of the list?

Answer

It becomes the second element

Status : Correct ***Marks : 1/1***

vtu27499

3. Which of the following are true about a singly linked list?

Answer

It allows random access to elements.
Each node has a reference to both the previous and next node.
It can grow dynamically during runtime.
The last node's "next" pointer points to NULL in a non-circular list.

Status : Wrong ***Marks : 0/1***

4. When inserting a new node into a singly linked list, which of the following are the possible cases for insertion?

Answer

 Inserting a new node at the middle of the list (random location)

Status : Wrong ***Marks : 0/1***

5. Given the linked list: 5 -> 10 -> 15 -> 20 -> 25 -> NULL. What will be the output of traversing the list and printing each node's data?

Answer

5 10 15 20 25

Status : Correct ***Marks : 1/1***

6. Which of the following is the process of finding information in a linked list?

Answer

Start from the head of the list and traverse sequentially until finding (or not finding) the node

Status : Correct ***Marks : 1/1***

7. Which of the following actions needs to be taken when the last item in a linked list is deleted?

Answer

Change the reference of the second last item to NULL reference

Status : Correct ***Marks : 1/1***

8. Which pointer helps in traversing a doubly linked list in reverse order?

Answer

prev

Status : Correct ***Marks : 1/1***

9. Which of the following is true about the last node in a doubly linked list?

Answer

Its next pointer is NULL

Status : Correct ***Marks : 1/1***

10. What will be the effect of setting the prev pointer of a node to NULL in a doubly linked list?

Answer

The node will become the new head

Status : Correct ***Marks : 1/1***

11. How do you delete a node from the middle of a doubly linked list?

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

12. What happens if we insert a node at the beginning of a doubly linked list?

Answer

The previous pointer of the new node is NULL

Status : Correct ***Marks : 1/1***

13. How many pointers does a node in a doubly linked list have?

Answer

2

Status : Correct ***Marks : 1/1***

14. In deletion from the end of a doubly linked list, which pointer needs to be updated?

Answer

prev->next = NULL

Status : Correct ***Marks : 1/1***

15. Which of the following is true about a Doubly Linked List?

Answer

Last node's next pointer is always NULL

Status : Correct ***Marks : 1/1***

vtu27499

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 5_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Arun is tasked with implementing a program to reverse a singly linked list. The program should populate the list elements by inserting them at the beginning, traverse the list and reverse the order of its elements.

Since Arun is new to programming, you have to guide him through the task.

Input Format

The first line of input consists of an integer N, representing the linked list size.

The second line consists of N space-separated list elements.

Output Format

The first line of output prints the list elements after inserting them at the beginning.

The second line prints the reversed linked list.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
23 85 47 62 31

Output: 31 62 47 85 23
23 85 47 62 31

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Define a Node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at the beginning
struct Node* insert_at_beginning(struct Node* head, int data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = data;
    new_node->next = head;
    head = new_node;
    return head;
}

// Function to print the linked list
void print_list(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

```
// Function to reverse the linked list
struct Node* reverse_list(struct Node* head) {
    struct Node* prev = NULL;
    struct Node* current = head;
    struct Node* next = NULL;

    while (current != NULL) {
        next = current->next;    // store next node
        current->next = prev;    // reverse the link
        prev = current;          // move prev forward
        current = next;          // move current forward
    }
    head = prev;
    return head;
}

int main() {
    int N;
    scanf("%d", &N);

    struct Node* head = NULL;
    int val;

    // Insert elements at the beginning
    for (int i = 0; i < N; i++) {
        scanf("%d", &val);
        head = insert_at_beginning(head, val);
    }

    // Print list after inserting at beginning
    print_list(head);

    // Reverse the linked list
    head = reverse_list(head);

    // Print the reversed list
    print_list(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

In a computer science course project, students are required to write a program to insert values at the end of a singly linked list and remove the last node from the list. The program should traverse the list and delete the last node, adjusting pointers accordingly.

Assist students in the task.

Input Format

The first line of input consists of an integer N, representing the list size.

The second line consists of N space-separated integers, representing the list elements.

Output Format

The output prints space-separated integers, representing the list after the deletion of the last element.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8
54 25 14 38 55 73 95 82
Output: 54 25 14 38 55 73 95

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Define Node structure
struct Node {
   int data;
   struct Node* next;
```

```
};

// Function to insert at the end
struct Node* insert_at_end(struct Node* head, int data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = data;
    new_node->next = NULL;

    if (head == NULL) {
        return new_node; // If list is empty
    }

    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = new_node;
    return head;
}

// Function to delete last node
struct Node* delete_last_node(struct Node* head) {
    if (head == NULL) return NULL;        // Empty list
    if (head->next == NULL) {             // Only one node
        free(head);
        return NULL;
    }

    struct Node* temp = head;
    while (temp->next->next != NULL) {    // Traverse to second last node
        temp = temp->next;
    }
    free(temp->next);                     // Delete last node
    temp->next = NULL;                    // Update second last node
    return head;
}

// Function to print the list
void print_list(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
```

```
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int N, val;
    scanf("%d", &N);

    struct Node* head = NULL;

    // Insert elements at the end
    for (int i = 0; i < N; i++) {
        scanf("%d", &val);
        head = insert_at_end(head, val);
    }

    // Delete the last node
    head = delete_last_node(head);

    // Print the resulting list
    print_list(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Dev is tasked with creating a program that efficiently finds the middle element of a linked list. The program should take user input to populate the linked list by inserting each element into the front of the list and then determining the middle element.

Assist Dev, as he needs to ensure that the middle element is accurately identified from the constructed singly linked list:

If it's an odd-length linked list, return the middle element.If it's an even-length linked list, return the second middle element of the two elements.

Input Format

The first line of input consists of an integer n, representing the number of elements in the linked list.

The second line consists of n space-separated integers, representing the elements of the list.

Output Format

The first line of output displays the linked list after inserting elements at the front.

The second line displays "Middle Element: " followed by the middle element of the linked list.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50

Output: 50 40 30 20 10
Middle Element: 30

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Define Node structure
struct Node {
   int data;
   struct Node* next;
};

// Insert at the front of the list
struct Node* insert_front(struct Node* head, int data) {
   struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
   new_node->data = data;
   new_node->next = head;
   return new_node;
```

```
}

// Print the linked list
void print_list(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

// Find middle element (second middle if even)
int find_middle(struct Node* head) {
    struct Node* slow = head;
    struct Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow->data;
}

int main() {
    int n, val;
    scanf("%d", &n);

    struct Node* head = NULL;

    // Insert elements at the front
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        head = insert_front(head, val);
    }

    // Print the linked list
    print_list(head);

    // Find and print middle element
    printf("Middle Element: %d\n", find_middle(head));
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Saran is tasked with implementing a doubly linked list in data structures, allowing users to append elements to the end of the list, and remove the last element. He needs to ensure efficient memory management in the implementation.

Assist him with a suitable program.

Input Format

The first line of input consists of an integer n, representing the size of the list.

The second line consists of n space-separated integers, representing the elements of the list.

Output Format

The output displays integers, representing the list after removing the last element, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
48 22 5 62 87

Output: 48 22 5 62

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Define Node structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Append node at the end of the list
struct Node* append(struct Node* tail, int data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = data;
    new_node->next = NULL;

    if (tail == NULL) {  // First node
        new_node->prev = NULL;
        return new_node;
    }

    tail->next = new_node;
    new_node->prev = tail;
    return new_node;
}

// Print the list from head to tail
void print_list(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

// Delete the last node and return the new tail
struct Node* delete_last(struct Node* tail) {
    if (tail == NULL) return NULL; // Empty list
    struct Node* temp = tail;

    if (tail->prev != NULL) {
        tail = tail->prev;
        tail->next = NULL;
    } else {
```

```
        // Only one element
        tail = NULL;
    }
    free(temp);
    return tail;
}

int main() {
    int n, val;
    scanf("%d", &n);

    struct Node *head = NULL, *tail = NULL;

    // Append elements
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        if (head == NULL) {
            head = tail = append(NULL, val);
        } else {
            tail = append(tail, val);
        }
    }

    // Delete last element
    tail = delete_last(tail);
    if (tail == NULL) head = NULL; // List became empty

    // Print list from head
    struct Node* temp = head;
    print_list(temp);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement:

Ravi is building a simple application to manage a doubly linked list of integers. He wants to insert new integers into the list and delete a node at a specific position. After deleting it, he needs to display the updated list.

Write a program to ensure that the task is done by performing insertion, deletion at a given position, and then printing the modified list.

Input Format

The first line contains an integer n, representing the number of elements to be inserted into the list.

The second line contains n space-separated integers, representing the values to insert.

The third line contains an integer pos, representing the position of the node to be deleted from the list.

Output Format

If the given position is valid, print "Node at position X with value Y deleted successfully".

If the position is invalid, print "Position X not found in the list."

In both cases, print

"Data present in the list:"

followed by the updated list elements on the next line, space separated."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50
4

vtu27499

Output: Node at position 4 with value 40 deleted successfully.
Data present in the list:
10 20 30 50

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Define Node structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to append node at the end
struct Node* append(struct Node* tail, int data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = data;
    new_node->next = NULL;

    if (tail == NULL) { // First node
        new_node->prev = NULL;
        return new_node;
    }

    tail->next = new_node;
    new_node->prev = tail;
    return new_node;
}

// Function to print the list from head
void print_list(struct Node* head) {
    struct Node* temp = head;
    while(temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

// Delete node at given position
```

```
int delete_at_position(struct Node** head_ref, int pos) {
    if (*head_ref == NULL) return -1; // Empty list

    struct Node* temp = *head_ref;
    int count = 1;

    // Deleting head
    if (pos == 1) {
        int val = temp->data;
        *head_ref = temp->next;
        if (*head_ref != NULL)
            (*head_ref)->prev = NULL;
        free(temp);
        return val;
    }

    // Traverse to the position
    while (temp != NULL && count < pos) {
        temp = temp->next;
        count++;
    }

    if (temp == NULL) return -1; // Position not found

    // Adjust pointers
    if (temp->prev != NULL)
        temp->prev->next = temp->next;
    if (temp->next != NULL)
        temp->next->prev = temp->prev;

    int val = temp->data;
    free(temp);
    return val;
}

int main() {
    int n, val, pos;
    scanf("%d", &n);

    struct Node *head = NULL, *tail = NULL;

    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &val);
        if (head == NULL) {
            head = tail = append(NULL, val);
        } else {
            tail = append(tail, val);
        }
    }

    scanf("%d", &pos);

    int deleted_value = delete_at_position(&head, pos);
    if (deleted_value == -1) {
        printf("Position %d not found in the list.\n", pos);
    } else {
        printf("Node at position %d with value %d deleted successfully.\n", pos,
deleted_value);
    }

    printf("Data present in the list:\n");
    print_list(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 5_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given a singly linked list and two integers M and N, implement a program to traverse the linked list such that you retain every M node and then delete the next N nodes.

Continue this process until the end of the linked list.

Input Format

The first line consists of two integers M and N, separated by a space, representing the values to retain and delete.

The second line consists of a series of integers separated by spaces, representing the values of the linked list nodes.

The last input is -1 which will terminate the inputs.

vtu27499

Output Format

The output prints the linked list after applying the process of retaining every M node and deleting the next N nodes represented as a series of integers separated by "->".

Sample Test Case

Input: 2 2
1 2 3 4 5 6 7 8 -1

Output: 1->2->5->6

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to append node at end
struct Node* append(struct Node* tail, int data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = data;
    new_node->next = NULL;

    if (tail != NULL) {
        tail->next = new_node;
    }
    return new_node;
}

// Function to process the list: retain M nodes, delete N nodes
void retainMDeleteN(struct Node* head, int M, int N) {
    struct Node* current = head;
    struct Node* temp;

    while (current) {
        // Retain M nodes
        for (int i = 1; i < M && current != NULL; i++)
            current = current->next;
```

```
        if (current == NULL)
            break;

        // Delete next N nodes
        temp = current->next;
        for (int i = 0; i < N && temp != NULL; i++) {
            struct Node* node_to_delete = temp;
            temp = temp->next;
            free(node_to_delete);
        }

        // Connect retained part to remaining list
        current->next = temp;
        current = temp;
    }
}

// Function to print the list
void print_list(struct Node* head) {
    struct Node* temp = head;
    while (temp) {
        printf("%d", temp->data);
        if (temp->next)
            printf("->");
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int M, N, val;
    scanf("%d %d", &M, &N);

    struct Node* head = NULL;
    struct Node* tail = NULL;

    // Input linked list until -1
    while (scanf("%d", &val) && val != -1) {
        if (head == NULL) {
            head = tail = append(NULL, val);
        } else {
```

```
        tail = append(tail, val);
    }
    }

    // Process the list
    retainMDeleteN(head, M, N);

    // Print the final list
    print_list(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Imagine you are working on a text processing tool and need to implement a feature that allows users to insert characters at a specific position.

Implement a program that takes user inputs to create a singly linked list of characters and inserts a new character after a given index in the list.

Input Format

The first line of input consists of an integer N, representing the number of characters in the linked list.

The second line consists of a sequence of N characters, representing the linked list.

The third line consists of an integer index, representing the index(0-based) after which the new character node needs to be inserted.

The fourth line consists of a character value representing the character to be inserted after the given index.

Output Format

If the provided index is out of bounds (larger than the list size):

1. The first line prints "Invalid index".

2. The second line prints "Updated list: " followed by the unchanged linked list values.

Otherwise, prints "Updated list: " followed by the updated space separated linked list after inserting the new character after the given index.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
a b c d e
2
X

Output: Updated list: a b c X d e

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure for singly linked list
struct Node {
    char data;
    struct Node* next;
};

// Function to append a node at the end
struct Node* append(struct Node* tail, char val) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = val;
    new_node->next = NULL;
    if (tail != NULL) {
        tail->next = new_node;
    }
    return new_node;
}

// Function to print the linked list
```

```
void print_list(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%c", temp->data);
        if (temp->next != NULL)
            printf(" ");
        temp = temp->next;
    }
    printf("\n");
}

// Function to insert a character after a given index
int insert_after_index(struct Node* head, int index, char val, int size) {
    if (index >= size) // Invalid index
        return 0;

    struct Node* current = head;
    for (int i = 0; i < index; i++)
        current = current->next;

    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = val;
    new_node->next = current->next;
    current->next = new_node;
    return 1;
}

int main() {
    int N;
    scanf("%d", &N);

    struct Node* head = NULL;
    struct Node* tail = NULL;

    // Read initial linked list characters
    for (int i = 0; i < N; i++) {
        char ch;
        scanf(" %c", &ch); // space before %c to ignore whitespaces
        if (head == NULL) {
            head = tail = append(NULL, ch);
        } else {
            tail = append(tail, ch);
```

```
    }
  }

  int index;
  char val;
  scanf("%d", &index);
  scanf(" %c", &val);

  // Insert after index
  int success = insert_after_index(head, index, val, N);

  if (!success)
    printf("Invalid index\n");

  printf("Updated list: ");
  print_list(head);

  return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Imagine Anu is tasked with finding the middle element of a doubly linked list. Given a doubly linked list where each node contains an integer value and is inserted at the end, implement a program to find the middle element of the list. If the number of nodes is even, return the middle element pair.

Input Format

The first line of input consists of an integer N, representing the number of nodes in the doubly linked list.

The second line consists of N space-separated integers, representing the values of the nodes in the doubly linked list.

Output Format

The first line of output prints the space-separated elements of the doubly linked list.

The second line prints the middle element(s) of the doubly linked list, depending on whether the number of nodes is odd or even.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50

Output: 10 20 30 40 50
30

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure for doubly linked list
struct Node {
   int data;
   struct Node* prev;
   struct Node* next;
};

// Function to append a node at the end
struct Node* append(struct Node* tail, int val) {
   struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
   new_node->data = val;
   new_node->next = NULL;
   new_node->prev = tail;
   if (tail != NULL)
      tail->next = new_node;
   return new_node;
}

// Function to print the list
void print_list(struct Node* head) {
   struct Node* temp = head;
   while (temp != NULL) {
      printf("%d", temp->data);
      if (temp->next != NULL) printf(" ");
```

```
        temp = temp->next;
    }
    printf("\n");
}

// Function to find middle element(s)
void print_middle(struct Node* head, int n) {
    struct Node* temp = head;
    int mid = n / 2;

    if (n % 2 == 1) {
        // Odd number of nodes: move mid steps
        for (int i = 0; i < mid; i++)
            temp = temp->next;
        printf("%d\n", temp->data);
    } else {
        // Even number of nodes: print mid-1 and mid elements
        for (int i = 0; i < mid - 1; i++)
            temp = temp->next;
        printf("%d %d\n", temp->data, temp->next->data);
    }
}

int main() {
    int N;
    scanf("%d", &N);

    struct Node* head = NULL;
    struct Node* tail = NULL;

    for (int i = 0; i < N; i++) {
        int val;
        scanf("%d", &val);
        if (head == NULL) {
            head = tail = append(NULL, val);
        } else {
            tail = append(tail, val);
        }
    }

    // Print the doubly linked list
    print_list(head);
```

```
    // Print middle element(s)
    print_middle(head, N);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Pranav wants to clockwise rotate a doubly linked list by a specified number of positions. He needs your help to implement a program to achieve this. Given a doubly linked list and an integer representing the number of positions to rotate, write a program to rotate the list clockwise.

Input Format

The first line of input consists of an integer n, representing the number of elements in the linked list.

The second line consists of n space-separated linked list elements.

The third line consists of an integer k, representing the number of positions to rotate the list.

Output Format

The output displays the elements of the doubly linked list after rotating it by k positions.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
1
Output: 5 1 2 3 4

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure for doubly linked list
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to append a node at the end
struct Node* append(struct Node* tail, int val) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = val;
    new_node->next = NULL;
    new_node->prev = tail;
    if (tail != NULL)
        tail->next = new_node;
    return new_node;
}

// Function to print the list
void print_list(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) printf(" ");
        temp = temp->next;
    }
    printf("\n");
}

// Function to rotate list clockwise by k positions
struct Node* rotate_clockwise(struct Node* head, struct Node* tail, int n, int k) {
    if (k == 0 || n <= 1) return head;

    // Move tail k steps backward to find new tail
    struct Node* new_tail = tail;
    for (int i = 0; i < k; i++)
        new_tail = new_tail->prev;
```

```
    struct Node* new_head = new_tail->next;

    // Break the list at new_tail
    new_tail->next = NULL;
    new_head->prev = NULL;

    // Connect old tail to old head
    tail->next = head;
    head->prev = tail;

    return new_head;
}

int main() {
    int n;
    scanf("%d", &n);

    struct Node* head = NULL;
    struct Node* tail = NULL;

    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        if (head == NULL) {
            head = tail = append(NULL, val);
        } else {
            tail = append(tail, val);
        }
    }

    int k;
    scanf("%d", &k);

    head = rotate_clockwise(head, tail, n, k);

    print_list(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 5_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Akash wants to write a program that populates values in a singly linked list by inserting each element into the front of the list. After insertion, the program should delete a node immediately after a node with a specific value X.

Assist him with a suitable program.

Input Format

The first line of input consists of an integer N, representing the number of elements.

The second line consists of N space-separated integers, representing the elements of the linked list.

vtu27499

The third line consists of an integer X, representing the value after which the node has to be deleted.

Output Format

The output consists of two lines:

The first line shows the original linked list in the format: Original Linked List: <elements> -> NULL.

The second line shows the modified linked list after deleting the node following the node with value X: After deleting node after <X>: <modified_elements> -> NULL.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
15 28 36 43
36
Output: Original Linked List: 43 -> 36 -> 28 -> 15 -> NULL
After deleting node after 36: 43 -> 36 -> 15 -> NULL

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
   int data;
   struct Node* next;
};

// Function to insert a node at the front
struct Node* insert_front(struct Node* head, int val) {
   struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
   new_node->data = val;
   new_node->next = head;
   head = new_node;
   return head;
```

```
}

// Function to print the linked list
void print_list(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) printf(" -> ");
        temp = temp->next;
    }
    printf(" -> NULL\n");
}

// Function to delete node after a node with value X
struct Node* delete_after_value(struct Node* head, int X) {
    struct Node* temp = head;

    // Traverse to find the node with value X
    while (temp != NULL && temp->data != X)
        temp = temp->next;

    // If node X found and it has a next node
    if (temp != NULL && temp->next != NULL) {
        struct Node* del = temp->next;
        temp->next = del->next;
        free(del);
    }

    return head;
}

int main() {
    int N, X;
    scanf("%d", &N);

    struct Node* head = NULL;
    for (int i = 0; i < N; i++) {
        int val;
        scanf("%d", &val);
        head = insert_front(head, val);
    }
```

```
    scanf("%d", &X);

    printf("Original Linked List: ");
    print_list(head);

    head = delete_after_value(head, X);

    printf("After deleting node after %d: ", X);
    print_list(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Emma is managing her daily to-do list. Each task is represented by an integer ID. Due to a change in her schedule, Emma needs to update her to-do list by removing the first task, another task from the end of the list, and a third task from a specific position within the list.

Your task is to help Emma implement a function to perform these deletions and manage her to-do list efficiently.

Input Format

The first line of input contains an integer T, representing the number of tasks currently in the list.

The second line contains T integers, each representing the ID of a task.

The third line contains an integer p, representing the position (0-based index) of the task to be removed.

Output Format

The first line of output prints the linked list after deleting the first node, with each element separated by a space.

The second line of output prints the linked list after deleting the last node, with each element separated by a space.

The third line of output prints the linked list after deleting the node at the specified position, with each element separated by a space

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
11 22 33 44 55
2
Output: 22 33 44 55
22 33 44
22 33

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* create_node(int val) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = val;
    new_node->next = NULL;
    return new_node;
}

// Function to insert a node at the end
struct Node* insert_end(struct Node* head, int val) {
    struct Node* new_node = create_node(val);
    if (head == NULL) return new_node;
    struct Node* temp = head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = new_node;
```

```
    return head;
}

// Function to print the linked list
void print_list(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) printf(" ");
        temp = temp->next;
    }
    printf("\n");
}

// Delete first node
struct Node* delete_first(struct Node* head) {
    if (head == NULL) return NULL;
    struct Node* temp = head;
    head = head->next;
    free(temp);
    return head;
}

// Delete last node
struct Node* delete_last(struct Node* head) {
    if (head == NULL) return NULL;
    if (head->next == NULL) {
        free(head);
        return NULL;
    }
    struct Node* temp = head;
    while (temp->next->next != NULL)
        temp = temp->next;
    free(temp->next);
    temp->next = NULL;
    return head;
}

// Delete node at given position (0-based)
struct Node* delete_at_position(struct Node* head, int pos) {
    if (head == NULL) return NULL;
    if (pos == 0) return delete_first(head);
```

```
    struct Node* temp = head;
    for (int i = 0; i < pos - 1 && temp->next != NULL; i++)
        temp = temp->next;

    if (temp->next != NULL) {
        struct Node* del = temp->next;
        temp->next = del->next;
        free(del);
    }
    return head;
}

int main() {
    int T, pos;
    scanf("%d", &T);

    struct Node* head = NULL;
    for (int i = 0; i < T; i++) {
        int val;
        scanf("%d", &val);
        head = insert_end(head, val);
    }

    scanf("%d", &pos);

    // Delete first node
    head = delete_first(head);
    print_list(head);

    // Delete last node
    head = delete_last(head);
    print_list(head);

    // Delete node at position p
    head = delete_at_position(head, pos);
    print_list(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Emma is working with a doubly linked list and needs a program to delete a specific value from the list. Write a program that reads a list of integers, constructs a doubly linked list, and then deletes a given value if it exists. The program should print appropriate messages if the value is found and deleted or if it is not found in the list.

Input Format

The first line contains an integer n, representing the number of elements in the list.

The second line contains n space-separated integers, representing the elements of the doubly linked list.

The third line contains an integer x, representing the value to be deleted from the list.

Output Format

The first line prints the initial doubly linked list, with elements separated by a space.

The second line prints "Value x deleted from the list." on a new line if x is found and deleted, If x is not found, print "Value x not found in the list." on a new line.

The third line prints the updated doubly linked list on a new line after deletion, with elements separated by a space,

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 20 30
20

Output: 10 20 30
Value 20 deleted from the list.
10 30

vtu27499

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Doubly linked list node
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to create a new node
struct Node* create_node(int val) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = val;
    new_node->prev = NULL;
    new_node->next = NULL;
    return new_node;
}

// Function to append a node at the end
struct Node* append(struct Node* head, int val) {
    struct Node* new_node = create_node(val);
    if (head == NULL) return new_node;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = new_node;
    new_node->prev = temp;
    return head;
}

// Function to print the list
void print_list(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) printf(" ");
        temp = temp->next;
    }
```

```
    printf("\n");
}

// Function to delete a node with value x
struct Node* delete_value(struct Node* head, int x, int* found) {
    struct Node* temp = head;

    while (temp != NULL) {
        if (temp->data == x) {
            *found = 1;

            // If node is head
            if (temp->prev == NULL) {
                head = temp->next;
                if (head != NULL) head->prev = NULL;
            } else {
                temp->prev->next = temp->next;
                if (temp->next != NULL)
                    temp->next->prev = temp->prev;
            }

            free(temp);
            return head;
        }
        temp = temp->next;
    }

    *found = 0;
    return head;
}

int main() {
    int n, x;
    scanf("%d", &n);

    struct Node* head = NULL;
    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        head = append(head, val);
    }
```

```
    scanf("%d", &x);

    // Print original list
    print_list(head);

    // Delete value x
    int found;
    head = delete_value(head, x, &found);

    if (found)
        printf("Value %d deleted from the list.\n", x);
    else
        printf("Value %d not found in the list.\n", x);

    // Print updated list
    print_list(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Imagine you're managing a store's inventory list, and some products were accidentally entered multiple times. You need to remove the duplicate products from the list to ensure each product appears only once.

You have an unsorted doubly linked list of product IDs. Some of these product IDs may appear more than once, and your goal is to remove any duplicates.

Note: Insert elements at the front of the doubly linked list.

Input Format

The first line of input consists of an integer n, representing the number of elements in the list.

The second line of input consists of n space-separated integers representing the list elements.

Output Format

The output prints the final list after removing duplicate nodes, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10
12 12 10 4 8 4 6 4 4 8

Output: 8 4 6 10 12

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure for doubly linked list
struct Node {
   int data;
   struct Node* prev;
   struct Node* next;
};

// Function to create a new node
struct Node* create_node(int val) {
   struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
   new_node->data = val;
   new_node->prev = NULL;
   new_node->next = NULL;
   return new_node;
}

// Function to insert at front
struct Node* insert_front(struct Node* head, int val) {
   struct Node* new_node = create_node(val);
   if (head != NULL) {
      head->prev = new_node;
      new_node->next = head;
   }
```

```
    head = new_node;
    return head;
}

// Function to print the list
void print_list(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) printf(" ");
        temp = temp->next;
    }
    printf("\n");
}

// Function to remove duplicates
struct Node* remove_duplicates(struct Node* head) {
    struct Node *current, *compare, *next_node;
    current = head;
    while (current != NULL) {
        compare = current->next;
        while (compare != NULL) {
            next_node = compare->next;
            if (compare->data == current->data) {
                // Remove duplicate node
                if (compare->prev != NULL)
                    compare->prev->next = compare->next;
                if (compare->next != NULL)
                    compare->next->prev = compare->prev;
                free(compare);
            }
            compare = next_node;
        }
        current = current->next;
    }
    return head;
}

int main() {
    int n;
    scanf("%d", &n);
```

```
    struct Node* head = NULL;
    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        head = insert_front(head, val);
    }

    head = remove_duplicates(head);

    print_list(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 6_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 10

Section 1 : MCQ

1. In Radix Sort, what happens if the input array contains numbers with different lengths?

Answer

The numbers are converted to the same length.

Status : Wrong ***Marks : 0/1***

2. In heap sort, when does the actual sorting take place?

Answer

During the swap and heapify phase

Status : Correct ***Marks : 1/1***

vtu27499

3. In heap sort, which of the following heap operations is performed to build a valid heap from an unsorted array?

Answer

Heapify

Status : Correct ***Marks : 1/1***

4. In heap sort, the element that is swapped with the final element of the heap is _____, in the case of a max-heap, sorted in descending order.

Answer

Largest element

Status : Correct ***Marks : 1/1***

5. Bucket sort is most efficient in the case when _____

Answer

The input is uniformly distributed

Status : Correct ***Marks : 1/1***

6. Which of the following is not necessarily a stable sorting algorithm?

Answer

All of the mentioned options

Status : Wrong ***Marks : 0/1***

7. Which step is essential to implement Bucket Sort correctly?

Answer

Dividing the range of input into intervals (buckets)

Status : Correct ***Marks : 1/1***

vtu27499

8. Which of the following is a characteristic of Radix Sort?

Answer

It is a divide-and-conquer sorting algorithm.

Status : Wrong ***Marks : 0/1***

9. A database contains the following employee IDs: [1263, 2910, 8742, 3891, 2156]. Using Radix Sort, which of the following is the correct order of employee IDs after the first pass (sorting by the least significant digit)?

Answer

[1263, 2156, 2910, 3891, 8742]

Status : Wrong ***Marks : 0/1***

10. What is the purpose of the counting sort algorithm in Radix Sort?

Answer

To sort the elements based on the current digit.

Status : Correct ***Marks : 1/1***

11. When does Heap Sort perform at its worst?

Answer

When all elements of the input array are the same.

Status : Correct ***Marks : 1/1***

12. How does Counting Sort handle duplicate elements?

Answer

It preserves their relative order

Status : Correct ***Marks : 1/1***

vtu27499

13. What is the purpose of the count array in Counting Sort?

Answer

To store the count of each unique element of the input array at their respective indices

Status : Correct ***Marks : 1/1***

14. What is the purpose of calculating the prefix sum in the Counting Sort algorithm?

Answer

To determine the final positions of elements in the output array

Status : Correct ***Marks : 1/1***

15. What is the initial value of each element in the count array in Counting Sort?

Answer

1

Status : Wrong ***Marks : 0/1***

vtu27499

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 6_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Krish is participating in a coding competition and has been given an array of integers. His task is to sort the array using the Counting Sort algorithm.

Your goal is to help Krish by implementing the countSort method to sort the array in non-decreasing order.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line of input consists of N space-separated integers, representing the elements of the array.

Output Format

The output prints a single line containing N space-separated integers representing the sorted array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
4 2 2 8 3

Output: 2 2 3 4 8

Answer

```
#include <stdio.h>

void countSort(int arr[], int n) {
    int i, max = arr[0];

    // Find maximum element
    for(i = 1; i < n; i++) {
        if(arr[i] > max) {
            max = arr[i];
        }
    }

    // Create count array
    int count[max + 1];

    // Initialize count array to 0
    for(i = 0; i <= max; i++) {
        count[i] = 0;
    }

    // Count occurrences
    for(i = 0; i < n; i++) {
        count[arr[i]]++;
    }

    // Print sorted array
    for(i = 0; i <= max; i++) {
```

```
        while(count[i] > 0) {
            printf("%d ", i);
            count[i]--;
        }
    }
}

int main() {
    int n, i;

    scanf("%d", &n);
    int arr[n];

    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    countSort(arr, n);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Raj, a keen problem solver, has come across an intriguing numerical puzzle that involves sorting numbers in ascending order. Eager to crack the code, he decides to implement the Radix Sort algorithm.

Your task is to assist Raj in designing a program to solve this numerical puzzle.

Input Format

The first line contains an integer n, denoting the number of elements to be sorted.

The second line consists of n space-separated integers, representing the numbers to be sorted.

Output Format

The output prints the array after it has been sorted using the Radix Sort algorithm, separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
543 986 217 765 329

Output: 217 329 543 765 986

Answer

```
#include <stdio.h>

// Function to get maximum value in array
int getMax(int arr[], int n) {
    int max = arr[0];
    for(int i = 1; i < n; i++) {
        if(arr[i] > max)
            max = arr[i];
    }
    return max;
}

// Counting sort based on digit (exp = 1, 10, 100...)
void countSort(int arr[], int n, int exp) {
    int output[n];
    int count[10] = {0};

    // Count occurrences of digits
    for(int i = 0; i < n; i++) {
        count[(arr[i] / exp) % 10]++;
    }

    // Convert count[] to cumulative count[]
    for(int i = 1; i < 10; i++) {
        count[i] += count[i - 1];
```

```
    }

    // Build output array (stable sort)
    for(int i = n - 1; i >= 0; i--) {
        int index = (arr[i] / exp) % 10;
        output[count[index] - 1] = arr[i];
        count[index]--;
    }

    // Copy output to original array
    for(int i = 0; i < n; i++) {
        arr[i] = output[i];
    }
}

// Radix Sort function
void radixSort(int arr[], int n) {
    int max = getMax(arr, n);

    // Apply counting sort for every digit
    for(int exp = 1; max / exp > 0; exp *= 10) {
        countSort(arr, n, exp);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    radixSort(arr, n);

    // Print sorted array
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
```

}

Status : Correct ***Marks : 10/10***

3. Problem Statement

Amit, an e-commerce manager, is working on an inventory management system to track the top-selling products. Given the sales data of various products, he wants to quickly determine the Kth best-selling product to make informed restocking decisions.

Help Amit implement Heap Sort to efficiently find the Kth best-selling product from the given sales data.

Input Format

The first line of input consists of an integer n, representing the number of products.

The second line consists of n space-separated integers, representing the sales data of each product.

The third line contains an integer K, representing the rank of the product to retrieve.

Output Format

The output prints the Kth best-selling product based on the given sales data.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 15 5 25
2
Output: 20

Answer

```c
#include <stdio.h>

// Function to swap two elements
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify function (Max Heap)
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

// Function to find Kth largest using Heap Sort
int kthLargest(int arr[], int n, int k) {
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract max k-1 times
    for (int i = n - 1; i >= n - k + 1; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }

    // Root is Kth largest
    return arr[0];
}
```

```
int main() {
    int n, k;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &k);

    int result = kthLargest(arr, n, k);
    printf("%d", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Ragavi, a computer scientist, is working on efficient text processing for search engines. She has a list of words in random order and needs to sort them in lexicographic order using Heap Sort to improve search performance.

Help Ragavi implement Heap Sort to arrange the words in lexicographic order efficiently.

Input Format

The first line consists of an integer N denoting the number of strings.

The next line consists of N space-separated strings.

Output Format

The output displays N space-separated sorted strings in lexicographic order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10
banana pineapple mango chikoo jack star guava lemon tomato orange
Output: banana chikoo guava jack lemon mango orange pineapple star tomato

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX 100

// Swap two strings
void swap(char a[], char b[]) {
    char temp[MAX];
    strcpy(temp, a);
    strcpy(a, b);
    strcpy(b, temp);
}

// Heapify function for strings (Max Heap)
void heapify(char arr[][MAX], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && strcmp(arr[left], arr[largest]) > 0)
        largest = left;

    if (right < n && strcmp(arr[right], arr[largest]) > 0)
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

// Heap Sort for strings
void heapSort(char arr[][MAX], int n) {
```

```
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract elements one by one
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    char arr[n][MAX];

    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i]);
    }

    heapSort(arr, n);

    // Print sorted strings
    for (int i = 0; i < n; i++) {
        printf("%s ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

You're organizing a gift-giving event where participants exchange gifts of varying values. To ensure fairness, you use the Bucket Sort algorithm. You gather the participants' gift values and initialize a count array.

Then, you iterate through the values, updating the count array. Next, you

sort the values in descending order by iterating through the count array. Finally, you display the sorted values, ensuring fairness in the gift exchange.

Input Format

The first line contains an integer N, representing the number of gift values.

The second line contains N space-separated integers representing the gift values brought by the participants. Each gift value is an integer between 1 and 999, inclusive.

Output Format

The output displays a single line containing N space-separated integers representing the sorted gift values in descending order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
156 378 453 278 281 987
Output: 987 453 378 281 278 156

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    int count[1000] = {0};

    // Input values and update count array
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        count[arr[i]]++;
    }

    // Print in descending order
```

```
    for (int i = 999; i >= 1; i--) {
        while (count[i] > 0) {
            printf("%d ", i);
            count[i]--;
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 6_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Yuvana, an avid programmer, is keen on mastering various sorting algorithms and has recently delved into Counting Sort. As a practice exercise, she is given an array of N integers where each element ranges from 1 to N-1. Some numbers may appear multiple times in the array. Her task is to identify and print all numbers that appear more than once. If no such duplicates are found, she should print -1.

Help Yuvana by implementing a function that efficiently finds and prints duplicate numbers using the Counting Sort technique.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

vtu27499

The second line of input consists of N space-separated integers, representing the elements of the array.

Output Format

The output prints all the duplicate numbers that appear more than once in the array, each separated by a space.

If no duplicates are found, print -1.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 3 4 2 2
Output: 2

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    int count[n + 1];

    // Initialize count array
    for (int i = 0; i <= n; i++) {
        count[i] = 0;
    }

    // Input and count frequency
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        count[arr[i]]++;
    }

    int found = 0;
```

```
    // Find duplicates
    for (int i = 1; i <= n; i++) {
        if (count[i] > 1) {
            printf("%d ", i);
            found = 1;
        }
    }

    // If no duplicates
    if (!found) {
        printf("-1");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Anu is tasked with organizing a list of fruit names stored as strings. To ensure that the list is easy to navigate, she decides to sort the strings in ascending alphabetical order.

Being a fan of efficient sorting algorithms, Anu opts to use radix sort for this task. Help Anu with a program that can take a list of fruit names as input and output the sorted list.

Input Format

The first line of input consists of an integer n, representing the number of fruit names in the list.

The next n lines each consist of a string, representing the names of the fruits.

Output Format

The output prints n lines containing the sorted list of fruit names in ascending alphabetical order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
apple
banana
orange
grape

Output: apple
banana
grape
orange

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX 100
#define CHAR_RANGE 256

// Function to get maximum length
int getMaxLen(char arr[][MAX], int n) {
    int max = strlen(arr[0]);
    for (int i = 1; i < n; i++) {
        if (strlen(arr[i]) > max)
            max = strlen(arr[i]);
    }
    return max;
}

// Counting sort for a specific character position
void countSort(char arr[][MAX], int n, int pos) {
    char output[n][MAX];
    int count[CHAR_RANGE] = {0};

    // Count frequency
    for (int i = 0; i < n; i++) {
        int ch = (pos < strlen(arr[i])) ? arr[i][pos] : 0;
        count[ch]++;
    }
```

```
    // Cumulative count
    for (int i = 1; i < CHAR_RANGE; i++) {
        count[i] += count[i - 1];
    }

    // Build output (stable)
    for (int i = n - 1; i >= 0; i--) {
        int ch = (pos < strlen(arr[i])) ? arr[i][pos] : 0;
        strcpy(output[count[ch] - 1], arr[i]);
        count[ch]--;
    }

    // Copy back
    for (int i = 0; i < n; i++) {
        strcpy(arr[i], output[i]);
    }
}

// Radix Sort for strings
void radixSort(char arr[][MAX], int n) {
    int maxLen = getMaxLen(arr, n);

    // Sort from last character to first
    for (int pos = maxLen - 1; pos >= 0; pos--) {
        countSort(arr, n, pos);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    char arr[n][MAX];

    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i]);
    }

    radixSort(arr, n);

    // Output sorted strings
```

```
    for (int i = 0; i < n; i++) {
        printf("%s\n", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Riya, an HR manager, needs a quick way to identify the Kth highest salary among employees for bonus distribution. To efficiently retrieve this information, she wants to use the max-heap technique.

Help Riya implement a solution that finds the Kth highest salary from the given list of salaries.

Input Format

The first line of input consists of an integer N, representing the number of employees.

The second line consists of N space-separated integers, representing the salaries of N employees.

The third line consists of an integer K, indicating the rank of the desired Kth highest salary.

Output Format

The output prints: <k>th largest Salary: <result>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
30000 25000 40000 35000 20000 28000 29500
5

Output: 5th largest Salary: 28000

Answer

```
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify (Max Heap)
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

// Function to find Kth largest
int kthLargest(int arr[], int n, int k) {
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract max k-1 times
    for (int i = n - 1; i >= n - k + 1; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
```

```
    return arr[0];
}

int main() {
    int n, k;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &k);

    int result = kthLargest(arr, n, k);

    printf("%dth largest Salary: %d", k, result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Manage a warehouse inventory with unique barcode numbers utilizing a specialized sorting algorithm designed for barcodes with exactly 16 digits. Input the number of barcode numbers (N) and their corresponding values.

Employ the Bucket Sort algorithm to distribute and sort the numbers based on their digits. Display the sorted barcode numbers, ensuring an efficient and organized warehouse inventory management system tailored for barcode systems with 16 digits.

Input Format

The first line contains an integer N, representing the number of barcode numbers to be sorted.

The following N lines contain exactly 16-digit long long integers, each representing a unique barcode number.

Output Format

The output displays a single line containing "Sorted barcode numbers:" followed by the sorted barcode numbers in ascending order. Each barcode number is displayed on a separate line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
2345678901234567
1234567890123456
3456789012345678
5678901234567890
4567890123456789

Output: Sorted barcode numbers:
1234567890123456
2345678901234567
3456789012345678
4567890123456789
5678901234567890

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX 20

// Function to perform counting sort based on digit position
void countSort(char arr[][MAX], int n, int pos) {
    char output[n][MAX];
    int count[10] = {0};

    // Count occurrences of digits
    for (int i = 0; i < n; i++) {
        int digit = arr[i][pos] - '0';
        count[digit]++;
    }
```

```
    // Cumulative count
    for (int i = 1; i < 10; i++) {
        count[i] += count[i - 1];
    }

    // Build output array (stable)
    for (int i = n - 1; i >= 0; i--) {
        int digit = arr[i][pos] - '0';
        strcpy(output[count[digit] - 1], arr[i]);
        count[digit]--;
    }

    // Copy back
    for (int i = 0; i < n; i++) {
        strcpy(arr[i], output[i]);
    }
}

// Radix sort for 16-digit barcodes
void radixSort(char arr[][MAX], int n) {
    // Fixed length = 16 digits
    for (int pos = 15; pos >= 0; pos--) {
        countSort(arr, n, pos);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    char arr[n][MAX];

    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i]);
    }

    radixSort(arr, n);

    printf("Sorted barcode numbers:\n");
    for (int i = 0; i < n; i++) {
        printf("%s\n", arr[i]);
    }
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu27499

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 6_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Alex is working on a data analysis project and needs to find specific elements within a dataset. He has an array of integers where some numbers may be repeated. His goal is to identify the Kth smallest element in this array. The challenge is that the array can contain duplicate values, so finding the exact Kth smallest element requires careful consideration.

Alex has decided to use a counting sort approach to solve this problem. Given an array of integers and a number K, where K is smaller than the size of the array, he needs to find and print the Kth smallest element in the array.

Help Alex by implementing a function that efficiently finds and prints the Kth smallest element using the counting sort approach.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line of input consists of N space-separated integers, representing the elements of the array.

The third line of input consists of an integer K, representing the position of the smallest element to find.

Output Format

The output prints the Kth smallest element in the array as "Kth Smallest element is <smallest element>".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
7 10 4 3 20 15
3
Output: Kth Smallest element is 7

Answer

```
#include <stdio.h>

int main() {
    int n, k;
    scanf("%d", &n);

    int arr[n];
    int count[101] = {0};

    // Input and frequency count
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        count[arr[i]]++;
    }
```

```
    scanf("%d", &k);

    int cumulative = 0;
    int result = -1;

    // Find Kth smallest
    for (int i = 1; i <= 100; i++) {
        cumulative += count[i];
        if (cumulative >= k) {
            result = i;
            break;
        }
    }

    printf("Kth Smallest element is %d", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Rohan is developing a text analysis tool and wants to sort a list of words based on the count of vowels in each word. He believes that sorting the words based on vowel count will help him analyze the text more effectively.

To achieve this, he plans to use radix sort. Assist Rohan with a suitable program to implement this sorting method for his project.

Input Format

The first line of input consists of an integer n, representing the number of words.

The next n lines each consist of a string, representing the words.

Output Format

The output prints n lines containing the sorted list of words based on the count of vowels in each word.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
another
stuck
door
you

Output: stuck
door
you
another

Answer

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

// Function to count vowels
int countVowels(char str[]) {
    int count = 0;
    for (int i = 0; str[i]; i++) {
        char ch = tolower(str[i]);
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
            count++;
        }
    }
    return count;
}

// Stable counting sort based on vowel count
void countingSort(char arr[][MAX], int n) {
    char output[n][MAX];
    int count[101] = {0};  // max vowels possible ≤ 100
    int vowelCount[n];

    // Store vowel counts
```

```
    for (int i = 0; i < n; i++) {
        vowelCount[i] = countVowels(arr[i]);
        count[vowelCount[i]]++;
    }

    // Cumulative count
    for (int i = 1; i < 101; i++) {
        count[i] += count[i - 1];
    }

    // Build output (stable)
    for (int i = n - 1; i >= 0; i--) {
        int v = vowelCount[i];
        strcpy(output[count[v] - 1], arr[i]);
        count[v]--;
    }

    // Copy back
    for (int i = 0; i < n; i++) {
        strcpy(arr[i], output[i]);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    char arr[n][MAX];

    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i]);
    }

    countingSort(arr, n);

    // Print sorted words
    for (int i = 0; i < n; i++) {
        printf("%s\n", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

In a manufacturing plant, each product is assigned an integer production code. To streamline quality control, the products have to be sorted in ascending order based on the last digit of their production codes.

Develop a program that uses Radix Sort to sort the products according to their code's last digit.

Input Format

The first line consists of an integer n, representing the number of products.

The second line consists of n space-separated integers each representing a product code.

Output Format

The output prints the sorted product codes based on the last digit in ascending order, separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
24570 61064 65391
Output: 24570 65391 61064

Answer

```
#include <stdio.h>

// Function for counting sort based on last digit
void countSort(int arr[], int n) {
    int output[n];
    int count[10] = {0};
```

```
    // Count occurrences of last digits
    for (int i = 0; i < n; i++) {
        int digit = arr[i] % 10;
        count[digit]++;
    }

    // Convert to cumulative count
    for (int i = 1; i < 10; i++) {
        count[i] += count[i - 1];
    }

    // Build output array (stable)
    for (int i = n - 1; i >= 0; i--) {
        int digit = arr[i] % 10;
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }

    // Copy back to original array
    for (int i = 0; i < n; i++) {
        arr[i] = output[i];
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    countSort(arr, n);

    // Print sorted array
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Meera, a librarian, wants to organize a list of book titles based on how many words each title contains. If two titles have the same number of words, they should be arranged in lexicographical (dictionary) order.

Write a program to sort the given book titles using a bucket-based approach, where titles are first grouped by word count and then alphabetically sorted within each group.

Input Format

The first line contains an integer N, representing the number of book titles.

The next N lines: Each line contains a book title (may contain spaces and special characters).

Output Format

The output prints "Sorted book titles:"

Then print the sorted book titles, each on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
Harry Potter and the Philosopher's Stone
To Kill a Mockingbird
The Great Gatsby
1984

Output: Sorted book titles:
1984
The Great Gatsby
To Kill a Mockingbird
Harry Potter and the Philosopher's Stone

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX_TITLES 50
#define MAX_LEN 101

// Function to count words in a string
int countWords(char str[]) {
    int count = 0, inWord = 0;

    for (int i = 0; str[i]; i++) {
        if (str[i] != ' ' && inWord == 0) {
            count++;
            inWord = 1;
        } else if (str[i] == ' ') {
            inWord = 0;
        }
    }
    return count;
}

// Sort strings lexicographically
void sortBucket(char bucket[][MAX_LEN], int size) {
    char temp[MAX_LEN];
    for (int i = 0; i < size - 1; i++) {
        for (int j = i + 1; j < size; j++) {
            if (strcmp(bucket[i], bucket[j]) > 0) {
                strcpy(temp, bucket[i]);
                strcpy(bucket[i], bucket[j]);
                strcpy(bucket[j], temp);
            }
        }
    }
}

int main() {
    int n;
    scanf("%d\n", &n);

    char titles[MAX_TITLES][MAX_LEN];
```

```
    // Read titles (including spaces)
    for (int i = 0; i < n; i++) {
        fgets(titles[i], MAX_LEN, stdin);
        titles[i][strcspn(titles[i], "\n")] = '\0'; // remove newline
    }

    // Buckets (max words ≤ 100)
    char buckets[101][MAX_TITLES][MAX_LEN];
    int bucketSize[101] = {0};

    int maxWords = 0;

    // Distribute titles into buckets
    for (int i = 0; i < n; i++) {
        int wc = countWords(titles[i]);
        strcpy(buckets[wc][bucketSize[wc]], titles[i]);
        bucketSize[wc]++;
        if (wc > maxWords)
            maxWords = wc;
    }

    printf("Sorted book titles:\n");

    // Process buckets
    for (int i = 1; i <= maxWords; i++) {
        if (bucketSize[i] > 0) {
            sortBucket(buckets[i], bucketSize[i]);
            for (int j = 0; j < bucketSize[i]; j++) {
                printf("%s\n", buckets[i][j]);
            }
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 10_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 11

Section 1 : MCQ

1. Which of the following sequences of array elements forms a heap?

Answer

{23, 17, 14, 6, 13, 10, 1, 12, 7, 5}

Status : Wrong ***Marks : 0/1***

2. Consider a max heap, represented by the array: 23, 17, 14, 6, 13, 10, 1, 5. If a value 20 is inserted into this heap, then what will be the new max heap?

Answer

23, 20, 14, 17, 13, 10, 1, 5, 6

Status : Correct ***Marks : 1/1***

3. Given a binary max heap. The elements are sorted in an array as 25, 14, 16, 13, 10, 8, 12. What is the content of the array after two delete operations?

Answer

14, 13, 12, 10, 8

Status : Wrong ***Marks : 0/1***

4. In a Min Heap, what is the condition that must be satisfied between a parent and its children?

Answer

The parent must have a lower value than its children

Status : Correct ***Marks : 1/1***

5. When inserting an element with a value of 8 into a Min Heap, where would it be placed if the current heap contains the elements [10, 12, 15, 20, 17]?

Answer

As the left child of the root

Status : Wrong ***Marks : 0/1***

6. If you're inserting an element with a value of 5 into a Min Heap, and the current heap contains the elements [11, 16, 10, 13, 18], what will the Min Heap look like after the insertion?

Answer

5, 10, 11, 13, 16, 18

Status : Wrong ***Marks : 0/1***

7. When deleting the root element from a Min Heap, which of the following replaces it as the new root?

Answer

the last element from the tree

Status : Correct ***Marks : 1/1***

8. Tries are mainly used for:

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

9. In a trie, how is the end of a valid word usually represented?

Answer

By using an end-of-word (terminal) marker at a node

Status : Correct ***Marks : 1/1***

10. Which of the following is true about tries?

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

11. What is stored in a segment tree node for a Range Minimum Query (RMQ)?

Answer

Minimum element in range

Status : Correct ***Marks : 1/1***

12. In a Range Max Query tree, what is the internal node value?

Answer

Maximum of child values

Status : Correct ***Marks : 1/1***

13. Which segment tree operation is used for range max queries?

Answer

max() merge

Status : Correct ***Marks : 1/1***

14. What data structure is often used in segment tree nodes to count distinct elements?

Answer

Set

Status : Correct ***Marks : 1/1***

15. In range distinct count queries, what does each segment tree node contain?

Answer

Set of elements

Status : Correct ***Marks : 1/1***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 10_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Jovika is learning about heaps and wants to implement a program that constructs a max heap and inserts elements into it.

She wants to write a program that implements max heap insertion. The program defines functions to swap elements and perform maximum heapification. She wants to extend this program to take a list of integers, build a max heap, and insert a new element into the heap.

Help her in implementing the program.

Input Format

The first line of input consists of an integer N, representing the number of elements to be inserted into the max heap.

vtu27499

The second line consists of N space-separated integers, representing the elements to be inserted into the max heap.

The third line consists of an integer representing the new element to be inserted into the max heap.

Output Format

The output prints the space-separated integers, representing the elements in the max heap after inserting the new element in the order of insertion.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
3 1 4 2 5
6
Output: 6 3 5 2 1 4

Answer

```
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify (Max Heap)
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
```

```
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

// Build Max Heap
void buildHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

// Insert into Max Heap
void insert(int arr[], int *n, int key) {
    arr[*n] = key;
    int i = *n;
    (*n)++;

    // Heapify up
    while (i != 0 && arr[(i - 1) / 2] < arr[i]) {
        swap(&arr[i], &arr[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[20]; // max size

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Build initial heap
    buildHeap(arr, n);

    int key;
```

```
    scanf("%d", &key);

    // Insert new element
    insert(arr, &n, key);

    // Print heap
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Bob is fascinated by numbers and loves arranging them in various ways. His recent exploration led him to Max Heaps.

Bob likes odd numbers more than even numbers. After inserting elements into a Max Heap, he wants to remove all even numbers from it while keeping the Max Heap property intact.

Your task is to help Bob build a Max Heap from the given elements, display it, delete all the even numbers from the resultant array and then rearrange the elements in the form of Max Heap.

Input Format

The first line of input consists of an integer N, representing the number of elements initially to be inserted into the Max Heap.

The second line consists of N space-separated integers, representing the elements to be inserted into the heap.

Output Format

The first line of output prints the space-separated elements of the Max Heap after building it from the given N elements.

The second line prints the space-separated elements of the Max Heap after removing even elements.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
3 2 1

Output: 3 2 1
3 1

Answer

```
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify function (Max Heap)
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}
```

```
// Build Max Heap
void buildHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Step 1: Build heap
    buildHeap(arr, n);

    // Print initial heap
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Step 2: Remove even numbers
    int newArr[20];
    int j = 0;

    for (int i = 0; i < n; i++) {
        if (arr[i] % 2 != 0) { // keep odd
            newArr[j++] = arr[i];
        }
    }

    // Step 3: Rebuild heap with odd numbers
    buildHeap(newArr, j);

    // Print final heap
    for (int i = 0; i < j; i++) {
        printf("%d ", newArr[i]);
```

```
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Iniya is studying data structures and algorithms, and she is currently learning about heaps.

She wants to practice implementing Min-Heap and perform a deletion operation on it. A min-heap is a binary tree in which the value of each node is less than or equal to the values of its children.

Write a program to assist Iniya in building a min-heap and delete the root element.

Input Format

The first line of input consists of an integer N, representing the number of elements in the initial Min-Heap.

The second line consists of N space-separated integers, where each integer represents an element in the Min-Heap.

Output Format

The output prints the space-separated elements of the Min-Heap after performing the deletion operation of the root.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
80 40 30 10 50 33 66
Output: 30 40 33 80 50 66

Answer

```
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify for Min Heap
void heapify(int arr[], int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    if (right < n && arr[right] < arr[smallest])
        smallest = right;

    if (smallest != i) {
        swap(&arr[i], &arr[smallest]);
        heapify(arr, n, smallest);
    }
}

// Build Min Heap
void buildHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];

    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
    }

    // Step 1: Build Min Heap
    buildHeap(arr, n);

    // Step 2: Delete root
    arr[0] = arr[n - 1];  // replace root with last element
    n--;                  // reduce heap size

    // Step 3: Heapify from root
    heapify(arr, n, 0);

    // Print result
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

A trie (pronounced as "try") or prefix tree is a tree data structure used to store and retrieve keys in a dataset of strings efficiently. This data structure has various applications, such as autocomplete and spellchecker.

Implement the Trie class:

Trie() Initializes the trie object.void insert(String word): Inserts the string word into the trie.boolean search(String word) Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.boolean startsWith(String prefix) Returns true if a previously inserted string word has the prefix, and false otherwise.

Example

Input:

4

apple

cat

orange

mat

cat

ora

Output:

The word is present in the trie.

There is a word with the given prefix in the trie.

Input Format

The first line of the input consists of the number of words to insert, N.

The next N lines of input consist of the words in each line.

The next line of input consists of the word to search.

The last line of the input consists of the prefix to search.

Output Format

When searching for a word using the search method, the program will output one of the following messages:

- If the word is present in the trie, the output prints "The word is present in the trie."
- If the word is not present in the trie, the output prints "The word is not present in the trie."

When checking for a prefix using the startsWith method, the program will output one of the following messages:

- If there is a word in the trie with the given prefix, the output prints "There is a word with the given prefix in the trie."
- If there is no word in the trie with the given prefix, the output prints "There is no word with the given prefix in the trie."

Sample Test Case

Input: 4
apple
cat
orange
mat
catlyn
ora
Output: The word is not present in the trie.
There is a word with the given prefix in the trie.

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define ALPHABET_SIZE 26

// Trie Node
struct TrieNode {
   struct TrieNode *children[ALPHABET_SIZE];
   int isEnd;
};

// Create new node
struct TrieNode* createNode() {
   struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
   node->isEnd = 0;
   for (int i = 0; i < ALPHABET_SIZE; i++)
      node->children[i] = NULL;
   return node;
}

// Insert word
void insert(struct TrieNode* root, char* word) {
   struct TrieNode* curr = root;
```

```
    for (int i = 0; word[i]; i++) {
        int index = word[i] - 'a';

        if (curr->children[index] == NULL)
            curr->children[index] = createNode();

        curr = curr->children[index];
    }
    curr->isEnd = 1;
}

// Search word
int search(struct TrieNode* root, char* word) {
    struct TrieNode* curr = root;

    for (int i = 0; word[i]; i++) {
        int index = word[i] - 'a';

        if (curr->children[index] == NULL)
            return 0;

        curr = curr->children[index];
    }

    return curr->isEnd;
}

// Check prefix
int startsWith(struct TrieNode* root, char* prefix) {
    struct TrieNode* curr = root;

    for (int i = 0; prefix[i]; i++) {
        int index = prefix[i] - 'a';

        if (curr->children[index] == NULL)
            return 0;

        curr = curr->children[index];
    }
    return 1;
}
```

```
int main() {
    int n;
    scanf("%d", &n);

    struct TrieNode* root = createNode();

    char word[100];

    // Insert words
    for (int i = 0; i < n; i++) {
        scanf("%s", word);
        insert(root, word);
    }

    // Search word
    char searchWord[100];
    scanf("%s", searchWord);

    if (search(root, searchWord))
        printf("The word is present in the trie.\n");
    else
        printf("The word is not present in the trie.\n");

    // Prefix check
    char prefix[100];
    scanf("%s", prefix);

    if (startsWith(root, prefix))
        printf("There is a word with the given prefix in the trie.\n");
    else
        printf("There is no word with the given prefix in the trie.\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Given an array of integers representing the combat power of N players.

You need to efficiently perform the following operations:

Query (Q L R): Find the maximum combat power between ranks L and R (inclusive).Update (U i val): Update the power of the player at index i to a new value val.

To ensure efficiency, implement these operations using a Segment Tree.

Input Format

The first line contains two space-separated integers, N and Q, representing the number of players and the number of operations.

The second line contains N space-separated integers representing the initial combat power scores of all players, where index 0 is the top-ranked player.

The next Q lines describe the operations:

- "Q L R" Query represents the maximum combat power from ranks L to R (inclusive).
- "U i val" Updates the power score of the player at rank i to val.

All indices are 0-based.

Output Format

For each query Q L R, print the maximum combat power in that range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 5
4 7 2 9 5 1
Q 1 4
U 3 6
Q 1 4
U 0 10
Q 0 2
Output: 9

vtu27499

7
10

Answer

```
#include <stdio.h>

#define MAX 100

int arr[MAX], seg[4 * MAX];

// Build segment tree
void build(int node, int start, int end) {
    if (start == end) {
        seg[node] = arr[start];
    } else {
        int mid = (start + end) / 2;
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);

        seg[node] = (seg[2 * node] > seg[2 * node + 1])
                    ? seg[2 * node]
                    : seg[2 * node + 1];
    }
}

// Query maximum in range
int query(int node, int start, int end, int L, int R) {
    if (R < start || end < L)
        return -1; // no overlap

    if (L <= start && end <= R)
        return seg[node];

    int mid = (start + end) / 2;
    int left = query(2 * node, start, mid, L, R);
    int right = query(2 * node + 1, mid + 1, end, L, R);

    return (left > right) ? left : right;
}

// Update value at index
void update(int node, int start, int end, int idx, int val) {
```

```
    if (start == end) {
        arr[idx] = val;
        seg[node] = val;
    } else {
        int mid = (start + end) / 2;

        if (idx <= mid)
            update(2 * node, start, mid, idx, val);
        else
            update(2 * node + 1, mid + 1, end, idx, val);

        seg[node] = (seg[2 * node] > seg[2 * node + 1])
                ? seg[2 * node]
                : seg[2 * node + 1];
    }
}

int main() {
    int n, q;
    scanf("%d %d", &n, &q);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Build tree
    build(1, 0, n - 1);

    while (q--) {
        char type;
        scanf(" %c", &type);

        if (type == 'Q') {
            int L, R;
            scanf("%d %d", &L, &R);
            printf("%d\n", query(1, 0, n - 1, L, R));
        }
        else if (type == 'U') {
            int i, val;
            scanf("%d %d", &i, &val);
            update(1, 0, n - 1, i, val);
        }
```

```
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu27499

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 10_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Sai has been studying various algorithms related to heaps. Recently, he came across a problem where he needed to filter elements from a given array based on certain criteria related to heap properties.

He wants to build a max heap from a given array of integers, delete elements that fall outside the specified range, and then rearrange the elements in the form of a Max Heap.

Write a program to help Sai in implementing the program.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

vtu27499

The second line consists of N space-separated integers, representing the elements of the array.

The third line consists of two integers, a and b, separated by a space, representing the range to filter the elements.

Output Format

The output prints the space-separated integers, representing the elements that remain in the max heap after removing elements outside the given range.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
5 7 8 3 12
1 6

Output: 5 3

Answer

```
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify (Max Heap)
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
```

```
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

// Build Max Heap
void buildHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[30];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int a, b;
    scanf("%d %d", &a, &b);

    // Step 1: Build initial heap
    buildHeap(arr, n);

    // Step 2: Filter elements within range
    int filtered[30];
    int k = 0;

    for (int i = 0; i < n; i++) {
        if (arr[i] >= a && arr[i] <= b) {
            filtered[k++] = arr[i];
        }
    }

    // Step 3: Rebuild heap with filtered elements
```

```
    buildHeap(filtered, k);

    // Step 4: Print result
    for (int i = 0; i < k; i++) {
        printf("%d ", filtered[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Maxwell, an enthusiastic computer science student, is exploring the fascinating world of data structures. His task is to create a program that constructs a min heap from a list of integers and identifies negative numbers within it.

Help Maxwell in completing the program.

Input Format

The first line of input consists of an integer N, representing the number of integers in the array.

The second line consists of N space-separated integers, representing the elements of the list.

Output Format

The first line of output prints the min-heap as a space-separated list of integers.

The second line prints the negative numbers, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

15 -5 20 -19 23 42 29

Output: -19 -5 20 15 23 42 29
-19 -5

Answer

```
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify (Min Heap)
void heapify(int arr[], int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    if (right < n && arr[right] < arr[smallest])
        smallest = right;

    if (smallest != i) {
        swap(&arr[i], &arr[smallest]);
        heapify(arr, n, smallest);
    }
}

// Build Min Heap
void buildHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int main() {
    int n;
    scanf("%d", &n);
```

```
    int arr[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Step 1: Build Min Heap
    buildHeap(arr, n);

    // Step 2: Print Min Heap
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Step 3: Print negative numbers
    for (int i = 0; i < n; i++) {
        if (arr[i] < 0) {
            printf("%d ", arr[i]);
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

You are given an array of n integers. You need to perform q queries of two types:

Q l r — Find the minimum value in the subarray from index l to r (inclusive).U idx val — Update the element at index idx to val.

Implement an efficient solution using a Segment Tree to handle both query and update operations.

Input Format

The first line contains two space-separated integers n and q, representing the

size of the array and the number of queries

The second line contains n space-separated integers representing the elements of the array.

Next q lines: Each line contains a query in one of the following formats:

- Q l r
- U idx val

Output Format

For each query of type Q, print the minimum value in the specified range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3 2
4 2 5
Q 0 2
Q 1 1
Output: 2
2

Answer

```
#include <stdio.h>

#define MAX 100

int arr[MAX], seg[4 * MAX];

// Build Segment Tree
void build(int node, int start, int end) {
   if (start == end) {
      seg[node] = arr[start];
   } else {
      int mid = (start + end) / 2;
      build(2 * node, start, mid);
      build(2 * node + 1, mid + 1, end);
```

```
        seg[node] = (seg[2 * node] < seg[2 * node + 1])
                    ? seg[2 * node]
                    : seg[2 * node + 1];
    }
}

// Query Minimum
int query(int node, int start, int end, int L, int R) {
    if (R < start || end < L)
        return 100000; // large value (no overlap)

    if (L <= start && end <= R)
        return seg[node];

    int mid = (start + end) / 2;
    int left = query(2 * node, start, mid, L, R);
    int right = query(2 * node + 1, mid + 1, end, L, R);

    return (left < right) ? left : right;
}

// Update value
void update(int node, int start, int end, int idx, int val) {
    if (start == end) {
        arr[idx] = val;
        seg[node] = val;
    } else {
        int mid = (start + end) / 2;

        if (idx <= mid)
            update(2 * node, start, mid, idx, val);
        else
            update(2 * node + 1, mid + 1, end, idx, val);

        seg[node] = (seg[2 * node] < seg[2 * node + 1])
                    ? seg[2 * node]
                    : seg[2 * node + 1];
    }
}

int main() {
```

```
    int n, q;
    scanf("%d %d", &n, &q);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Build tree
    build(1, 0, n - 1);

    while (q--) {
        char type;
        scanf(" %c", &type);

        if (type == 'Q') {
            int l, r;
            scanf("%d %d", &l, &r);
            printf("%d\n", query(1, 0, n - 1, l, r));
        }
        else if (type == 'U') {
            int idx, val;
            scanf("%d %d", &idx, &val);
            update(1, 0, n - 1, idx, val);
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Meera is working on a text processing task where she wants to identify the longest common prefix among a given set of words. She decides to use a Trie structure to efficiently compute this prefix.

Write a program to help Meera insert the given words into a Trie and then determine the longest common prefix among them.

Input Format

The first line contains an integer n, representing the number of words.

The next n lines each contain a single word.

Output Format

The output prints "The longest common prefix is: <prefix>" if a common prefix exists.

Otherwise, print "There is no common prefix"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
aeroplane
aerospace
aerial
aesthetic
Output: The longest common prefix is: ae

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define ALPHABET_SIZE 26

// Trie Node
struct TrieNode {
   struct TrieNode *children[ALPHABET_SIZE];
   int isEnd;
};

// Create new node
struct TrieNode* createNode() {
   struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
   node->isEnd = 0;
```

```
    for (int i = 0; i < ALPHABET_SIZE; i++)
        node->children[i] = NULL;
    return node;
}

// Insert word into Trie
void insert(struct TrieNode* root, char* word) {
    struct TrieNode* curr = root;

    for (int i = 0; word[i]; i++) {
        int index = word[i] - 'a';

        if (curr->children[index] == NULL)
            curr->children[index] = createNode();

        curr = curr->children[index];
    }
    curr->isEnd = 1;
}

// Count children of a node
int countChildren(struct TrieNode* node, int *index) {
    int count = 0;
    for (int i = 0; i < ALPHABET_SIZE; i++) {
        if (node->children[i] != NULL) {
            count++;
            *index = i;
        }
    }
    return count;
}

// Find Longest Common Prefix
void findLCP(struct TrieNode* root, char *prefix) {
    struct TrieNode* curr = root;
    int index;

    while (countChildren(curr, &index) == 1 && curr->isEnd == 0) {
        prefix[strlen(prefix)] = index + 'a';
        prefix[strlen(prefix) + 1] = '\0';
        curr = curr->children[index];
    }
```

```
}

int main() {
    int n;
    scanf("%d", &n);

    struct TrieNode* root = createNode();

    char word[100];

    for (int i = 0; i < n; i++) {
        scanf("%s", word);
        insert(root, word);
    }

    char prefix[100] = "";

    findLCP(root, prefix);

    if (strlen(prefix) > 0)
        printf("The longest common prefix is: %s", prefix);
    else
        printf("There is no common prefix");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 10_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Aarav, a computer science student, is working on a heap-based data processing system. He needs to find the second-largest element from a collection of integers represented as a max-heap. Aarav first constructs the max-heap, then removes the largest element to extract the next largest value.

Write a program to help Aarav determine and print the second largest element, or display a message if it does not exist.

Input Format

The first line contains an integer n, representing the number of elements in the heap.

vtu27499

The second line contains n space-separated integers representing the elements of the heap.

Output Format

The output prints the second-largest element in the heap.

If there is no second-largest element, print the message:

"There is no second largest element in the heap."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
100 70 10 30
Output: 70

Answer

```
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify (Max Heap)
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;
```

```
    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

// Build Max Heap
void buildHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    if (n < 2) {
        printf("There is no second largest element in the heap.");
        return 0;
    }

    // Step 1: Build heap
    buildHeap(arr, n);

    // Step 2: Remove root (largest)
    arr[0] = arr[n - 1];
    n--;

    // Step 3: Heapify
    heapify(arr, n, 0);

    // Step 4: Root is second largest
    printf("%d", arr[0]);

    return 0;
```

}

Status : Correct ***Marks : 10/10***

2. Problem Statement

Elijah wants to verify if an array represents a valid min-heap. Implement a program to check whether a given array satisfies the min-heap property.

In a min-heap, for every node i, the value at that node must be less than or equal to the values of its child nodes (if they exist). Write a program to read an array and determine whether it maintains the min-heap structure.

Input Format

The first line contains an integer n, representing the number of elements.

The next line contains n space-separated integers representing the elements to be inserted into the min heap.

Output Format

The first line prints "The array represents a valid min-heap" if the array satisfies the min-heap property. Otherwise, print "The array does not represent a valid min-heap".

The second line prints the space-separated elements of the array as given in the input.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: The array represents a valid min-heap
1 2 3

Answer

```c
// You are using GCC
#include <stdio.h>

int isMinHeap(int arr[], int n) {
    for (int i = 0; i <= (n - 2) / 2; i++) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        // Check left child
        if (left < n && arr[i] > arr[left])
            return 0;

        // Check right child
        if (right < n && arr[i] > arr[right])
            return 0;
    }
    return 1;
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    if (isMinHeap(arr, n))
        printf("The array represents a valid min-heap\n");
    else
        printf("The array does not represent a valid min-heap\n");

    // Print array
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Given a list of daily transaction amounts over N days. Each transaction can be positive (deposit) or negative (withdrawal).

You must efficiently:

Find the sum of transactions between two given days.Update the transaction amount for any specific day.

To perform these operations efficiently, implement a Segment Tree.

Input Format

The first line contains two integers, N and Q, representing the number of days and number of operations.

The second line contains N space-separated integers representing the transaction amount for each day.

The next Q lines contain operations in one of the following formats:

- S L R represents Query sum from index L to R
- U i val Updates index i with new value val

All indices are 0-based.

Output Format

For each query S L R, print the sum of transactions in that range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 5
10 -5 20 15 -10 5
S 0 2

U 1 7
S 0 2
S 3 5
S 0 5

Output: 25
37
10
47

Answer

```
// You are using GCC
#include <stdio.h>

#define MAX 100

int segTree[4 * MAX];
int arr[MAX];

// Build Segment Tree
void build(int node, int start, int end) {
    if (start == end) {
        segTree[node] = arr[start];
    } else {
        int mid = (start + end) / 2;
        build(2 * node, start, mid);
        build(2 * node + 1, mid + 1, end);
        segTree[node] = segTree[2 * node] + segTree[2 * node + 1];
    }
}

// Range Sum Query
int query(int node, int start, int end, int L, int R) {
    if (R < start || end < L) {
        return 0; // No overlap
    }
    if (L <= start && end <= R) {
        return segTree[node]; // Total overlap
    }
    int mid = (start + end) / 2;
    int leftSum = query(2 * node, start, mid, L, R);
    int rightSum = query(2 * node + 1, mid + 1, end, L, R);
    return leftSum + rightSum;
```

```
}

// Update value at index idx
void update(int node, int start, int end, int idx, int val) {
    if (start == end) {
        arr[idx] = val;
        segTree[node] = val;
    } else {
        int mid = (start + end) / 2;
        if (idx <= mid)
            update(2 * node, start, mid, idx, val);
        else
            update(2 * node + 1, mid + 1, end, idx, val);

        segTree[node] = segTree[2 * node] + segTree[2 * node + 1];
    }
}

int main() {
    int N, Q;
    scanf("%d %d", &N, &Q);

    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }

    build(1, 0, N - 1);

    while (Q--) {
        char type;
        scanf(" %c", &type);

        if (type == 'S') {
            int L, R;
            scanf("%d %d", &L, &R);
            printf("%d\n", query(1, 0, N - 1, L, R));
        } else if (type == 'U') {
            int i, val;
            scanf("%d %d", &i, &val);
            update(1, 0, N - 1, i, val);
        }
    }
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Given a string s and a string array dictionary, return the longest string in the dictionary that can be formed by deleting some of the given string characters.

If there is more than one possible result, return the longest word with the smallest lexicographical order.

Example

Input:

s = "abpcplea"

dictionary = ["ale", "apple", "monkey", "plea"]

Output:

"apple"

Input Format

The first line of input consists of the string.

The second line of input consists of the array size.

The following lines of input consist of the array elements, each in a separate line.

Output Format

The output prints the longest string in the dictionary that can be formed by deleting some of the given string characters.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: abpcplea
4
ale
apple
monkey
plea

Output: apple

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 1005

// Check if word is a subsequence of s
int isSubsequence(char *s, char *word) {
    int i = 0, j = 0;

    while (s[i] && word[j]) {
        if (s[i] == word[j]) {
            j++;
        }
        i++;
    }

    return word[j] == '\0';
}

int main() {
    char s[MAX];
    int n;

    scanf("%s", s);
    scanf("%d", &n);

    char dict[n][MAX];
```

```
    for (int i = 0; i < n; i++) {
        scanf("%s", dict[i]);
    }

    char result[MAX] = "";

    for (int i = 0; i < n; i++) {
        if (isSubsequence(s, dict[i])) {

            if (strlen(dict[i]) > strlen(result) ||
                (strlen(dict[i]) == strlen(result) && strcmp(dict[i], result) < 0)) {

                strcpy(result, dict[i]);
            }
        }
    }

    printf("%s\n", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 11_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 12

Section 1 : MCQ

1. Consider the following two sequences:

X = < B, C, D, C, A, B, C >Y = < C, A, D, B, C, B >

The length of longest common subsequence of X and Y is:

Answer

4

Status : Correct ***Marks : 1/1***

2. Which of the following problems should be solved using dynamic programming?

Answer

Longest common subsequence

Status : Correct ***Marks : 1/1***

3. Which of the following problems is closely related to the Subsets Sum problem and can also be solved using dynamic programming?

Answer

Coin Change Problem

Status : Correct ***Marks : 1/1***

4. Consider two strings A = "qpqrr" and B = "pqprqrp". Let x be the length of the longest common subsequence (not necessarily contiguous) between A and B and let y be the number of such longest common subsequences between A and B.

Then x + 10y = _____.

Answer

34

Status : Correct ***Marks : 1/1***

5. Consider the strings "PQRSTPQRS" and "PRATPBRQRPS". What is the length of the longest common subsequence?

Answer

8

Status : Wrong ***Marks : 0/1***

6. Which of the following is/are property/properties of a dynamic programming problem?

Answer

Both optimal substructure and overlapping subproblems

Status : Correct ***Marks : 1/1***

7. If an optimal solution can be created for a problem by constructing optimal solutions for its subproblems, the problem possesses _____ property.

Answer

Optimal substructure

Status : Correct ***Marks : 1/1***

8. In dynamic programming, the technique of storing the previously calculated values is called _____.

Answer

Memoization

Status : Correct ***Marks : 1/1***

9. Which of the following is an example of a problem that can be solved using memoization in dynamic programming?

Answer

Computing the edit distance between two strings

Status : Correct ***Marks : 1/1***

10. When a top-down approach to dynamic programming is applied to a problem, it usually _____.

Answer

decreases the time complexity and increases the space complexity

Status : Correct ***Marks : 1/1***

11. Which of the following problems is NOT solved using dynamic programming?

Answer

vtu27499

Edit distance problem

Status : Wrong ***Marks : 0/1***

12. Which of the following is NOT a characteristic of dynamic programming?

Answer

Dynamic programming can be used for problems where the solution has an optimal substructure.

Status : Wrong ***Marks : 0/1***

13. What happens when a top-down approach of dynamic programming is applied to any problem?

Answer

It increases the space complexity and decreases the time complexity.

Status : Correct ***Marks : 1/1***

14. The Fibonacci sequence is often used to illustrate dynamic programming concepts. What is the time complexity of a naive recursive implementation of Fibonacci numbers?

Answer

O(2^n)

Status : Correct ***Marks : 1/1***

15. What are the different techniques to solve dynamic programming problems:

Answer

Memoization and Bottom-Up

Status : Correct ***Marks : 1/1***

vtu27499

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 11_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an integer n, representing the total number of stairs, the task is to determine the number of distinct ways to reach the top when at each move you can climb 1, 2, or 3 steps.

Write a program using the Dynamic Programming (Bottom-Up Approach) algorithm to ensure that the total number of possible ways to climb the stairs is correctly calculated and displayed.

Examples:

Input: 4

Output: 7

Explanation: There are seven ways: {1, 1, 1, 1}, {1, 2, 1}, {2, 1, 1}, {1, 1, 2}, {2,

2}, {3, 1}, {1, 3}.

Input: 3

Output: 4

Explanation: There are four ways: {1, 1, 1}, {1, 2}, {2, 1}, {3}.

Input Format

The input contains a single integer n, representing the total number of stairs.

Output Format

The output prints a single integer representing the total number of distinct ways to reach the top.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
Output: 7

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int dp[25];  // since n ≤ 20

    dp[0] = 1;

    if (n >= 1) dp[1] = 1;
    if (n >= 2) dp[2] = 2;

    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2] + dp[i - 3];
    }
```

```
    printf("%d\n", dp[n]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given a non-negative integer n, the task is to compute the n-th Lucas number using the Dynamic Programming (Bottom-Up Approach) algorithm. The Lucas sequence follows the recurrence relation:

$L(0) = 2$, $L(1) = 1$, and $L(n) = L(n-1) + L(n-2)$ for $n \geq 2$.

Write a program to ensure that the n-th Lucas number is correctly calculated and displayed.

Input Format

The input contains a single integer n, representing the position in the Lucas sequence.

Output Format

The output prints a single integer representing the n-th Lucas number.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
Output: 4

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int main() {
    int n;
    scanf("%d", &n);

    int dp[25];  // since n ≤ 20

    dp[0] = 2;

    if (n >= 1)
        dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    printf("%d\n", dp[n]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement:

Dr. Ananya is a computational biologist working on analyzing DNA sequences. She needs to identify the longest palindromic subsequence within a DNA string, which may represent certain symmetric patterns important for genetic research.

Given a string S representing a DNA sequence (composed of letters A, C, G, T), find the length of the longest subsequence of S that reads the same forwards and backwards.

Note:

A subsequence is formed by deleting zero or more characters without changing the order of the remaining characters.The DNA sequences can be long, and an efficient solution using dynamic programming is required to handle them within computational limits.

Input Format

The input displays a single line containing the DNA string S.

Output Format

The output displays a single integer representing the length of the longest palindromic subsequence in S.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: madam

Output: 5

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    char s[105];
    scanf("%s", s);

    int n = strlen(s);
    int dp[105][105];

    // Base case: single characters
    for (int i = 0; i < n; i++) {
        dp[i][i] = 1;
    }

    // Build table for lengths 2 to n
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i <= n - len; i++) {
            int j = i + len - 1;

            if (s[i] == s[j]) {
```

```
            if (len == 2)
                dp[i][j] = 2;
            else
                dp[i][j] = dp[i + 1][j - 1] + 2;
        } else {
            dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
        }
    }
  }

  printf("%d\n", dp[0][n - 1]);

  return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Arjun is tracking the stock prices for the last N days. He wants to invest in a way that he buys and sells stocks alternately to maximize his profit. He can either buy or skip on a day, but after buying he must sell before buying again. Find the maximum profit he can achieve from the stock prices. Help him to implement the task.

Formula

Let dp[i][0] = maximum profit on day i if not holding a stock

Let dp[i][1] = maximum profit on day i if holding a stock

Transition:

dp[i][0] = max(dp[i-1][0], dp[i-1][1] + prices[i]) either skip or sell today

dp[i][1] = max(dp[i-1][1], dp[i-1][0] - prices[i]) either keep holding or buy today

Base case:

dp[0][0] = 0

dp[0][1] = -prices[0]

Result:

max_profit = dp[N-1][0]

Input Format

The first line of input consists of Integer N representing the number of days.

The second line of input consists of N Integers prices[i] representing the stock price on day i.

Output Format

The output prints: "Maximum profit:" max_profit

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: Maximum profit: 2

Answer

```
#include <stdio.h>
#include <string.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    char s[105];
    scanf("%s", s);

    int n = strlen(s);
    int dp[105][105];

    // Base case: single characters
    for (int i = 0; i < n; i++) {
        dp[i][i] = 1;
```

```
    }

    // Build table for lengths 2 to n
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i <= n - len; i++) {
            int j = i + len - 1;

            if (s[i] == s[j]) {
                if (len == 2)
                    dp[i][j] = 2;
                else
                    dp[i][j] = dp[i + 1][j - 1] + 2;
            } else {
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
            }
        }
    }

    printf("%d\n", dp[0][n - 1]);

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

5. Problem Statment

To celebrate Diwali, Dhoni wants to buy a variety of crackers. He noticed four different types of crackers on the menu list, each with a price. He's now trying to figure out how many different ways he can get the crackers for Rs.N.

Rockets - 1 RsSparklers - 2 RsChakkars - 3 RsFlower pots - 5 Rs

Example

Input

4

Output

The total number of ways to get crackers is 4

Explanation

1st way -> 4 rockets

2nd way -> 2 Sparklers

3rd way -> 1 Sparkler, 2 rockets

4th way -> 1 Chakkar, 1 rocket

Input Format

The input consists of a single integer N, representing the total amount (in rupees) Dhoni wants to spend on crackers.

Output Format

The output prints "The total number of ways to get crackers is X" Where X representing the total number of different ways Dhoni can buy crackers such that the total cost equals N.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

Output: The total number of ways to get crackers is 6

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    int coins[] = {1, 2, 3, 5};
    int dp[N + 1];

    for (int i = 0; i <= N; i++)
        dp[i] = 0;

    dp[0] = 1;
```

```
    // Process each coin
    for (int c = 0; c < 4; c++) {
        int coin = coins[c];
        for (int i = coin; i <= N; i++) {
            dp[i] += dp[i - coin];
        }
    }

    printf("The total number of ways to get crackers is %d\n", dp[N]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 11_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Given a set of coin denominations and a target value V, determine the minimum number of coins required to make the value V.

If it's not possible to make the value with the given coins, return -1.

Input Format

The first line contains an integer n, representing the number of coin denominations.

The second line contains n space-separated integers, representing the coin denominations.

The third line contains an integer amount, representing the target value V.

Output Format

vtu27499

The output prints the minimum number of coins required to make the target value.

Print -1 if the value cannot be formed using the given denominations.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 5
11
Output: 3

Answer

```
// You are using GCC
#include <stdio.h>

#define INF 100000

int min(int a, int b) {
    return (a < b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int coins[10];
    for (int i = 0; i < n; i++) {
        scanf("%d", &coins[i]);
    }

    int V;
    scanf("%d", &V);

    int dp[V + 1];

    // Initialize DP array
    dp[0] = 0;
```

```
    for (int i = 1; i <= V; i++) {
        dp[i] = INF;
    }

    // Bottom-up DP
    for (int i = 1; i <= V; i++) {
        for (int j = 0; j < n; j++) {
            if (coins[j] <= i && dp[i - coins[j]] != INF) {
                dp[i] = min(dp[i], dp[i - coins[j]] + 1);
            }
        }
    }

    if (dp[V] == INF)
        printf("-1\n");
    else
        printf("%d\n", dp[V]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement:

Rajesh runs a delivery company and needs to deliver n packages to different cities. Each package i has:

A weight wt[i]A profit val[i] if delivered on timeA delivery deadline deadline[i] (in hours from now)Rajesh has a truck with maximum weight capacity W and can only deliver one package at a time. He can choose the order of deliveries, but if a package is delivered after its deadline, he earns zero profit for it.

Task: Determine the maximum total profit Rajesh can earn by selecting which packages to deliver and in what order, without exceeding the truck's capacity at any point.

Input Format

First line: integer n — number of packages

Second line: n integers — values of each package (profit)

Third line: n integers — weights of each package

Fourth line: n integers — deadlines of each package

Fifth line: integer W — truck capacity

Output Format

The output prints the single integer maximum total profit.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
100 200 150 120
10 20 15 15
5 3 4 2
30

Output: 470

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 20

typedef struct {
    int val;
    int wt;
    int dl;
} Package;

Package p[MAX];

int compare(const void *a, const void *b) {
    Package *p1 = (Package *)a;
```

```
    Package *p2 = (Package *)b;
    return p1->dl - p2->dl;
}

// Check if subset is feasible
int isValid(int mask, int n) {
    Package temp[MAX];
    int k = 0;
    int totalWt = 0;

    for (int i = 0; i < n; i++) {
        if (mask & (1 << i)) {
            temp[k++] = p[i];
            totalWt += p[i].wt;
        }
    }

    if (totalWt > 100000) return 0; // safe guard (not needed really)

    // Sort selected jobs by deadline
    for (int i = 0; i < k - 1; i++) {
        for (int j = i + 1; j < k; j++) {
            if (temp[i].dl > temp[j].dl) {
                Package t = temp[i];
                temp[i] = temp[j];
                temp[j] = t;
            }
        }
    }

    int time = 0;
    for (int i = 0; i < k; i++) {
        time++;
        if (time > temp[i].dl)
            return 0;
    }

    return 1;
}

int main() {
    int n;
```

```
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d", &p[i].val);

    for (int i = 0; i < n; i++)
        scanf("%d", &p[i].wt);

    for (int i = 0; i < n; i++)
        scanf("%d", &p[i].dl);

    int W;
    scanf("%d", &W);

    int maxProfit = 0;

    // Try all subsets
    for (int mask = 0; mask < (1 << n); mask++) {
        int totalWt = 0;
        int totalVal = 0;

        for (int i = 0; i < n; i++) {
            if (mask & (1 << i)) {
                totalWt += p[i].wt;
                totalVal += p[i].val;
            }
        }

        if (totalWt <= W && isValid(mask, n)) {
            if (totalVal > maxProfit)
                maxProfit = totalVal;
        }
    }

    printf("%d\n", maxProfit);

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

3. Problem Statement

You are given n items, each with a certain weight and value. Your task is to select items to put in a knapsack of capacity W so that the total value is maximized. You cannot break an item; you must either take the whole item or leave it.

Formally, given two integer arrays, val[0..n-1] and wt[0..n-1], representing the values and weights of n items respectively, and an integer W representing the knapsack capacity, determine the maximum value subset of items such that the total weight does not exceed W.

Solve this problem using dynamic programming.

Input Format

The first line contains an integer n, the number of items.

The second line contains n integers, the values of each item separated by spaces.

The third line contains n integers, the weights of each item separated by spaces.

The fourth line contains an integer W, the maximum capacity of the knapsack.

Output Format

Print a single integer representing the maximum value that can be carried in the knapsack.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2
50 60
5 10
10
Output: 60

Answer

vtu27499

```c
// You are using GCC
#include <stdio.h>

#define MAXN 105
#define MAXW 1005

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;
    scanf("%d", &n);

    int val[MAXN], wt[MAXN];

    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    scanf("%d", &W);

    int dp[MAXN][MAXW] = {0};

    // Build DP table
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {

            if (wt[i - 1] <= w) {
                dp[i][w] = max(
                    dp[i - 1][w],
                    val[i - 1] + dp[i - 1][w - wt[i - 1]]
                );
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    printf("%d\n", dp[n][W]);
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Imagine you are a tourist in Moscow and have booked a bus tour to see some amazing attractions just outside of town. The bus first drives around town for a while (a long while, since Moscow is a big city) picking up people at their respective hotels. It then proceeds to the amazing attraction, and after a few hours go back into the city, again driving to each hotel, this time to drop people off.

For some reason, whenever you do this, your hotel is always the first to be visited for pickup and the last to be visited for drop-off, meaning that you have to suffer through two not-so-amazing sightseeing tours of all the local hotels.

This is clearly not what you want to do (unless for some reason you are really into hotels), so let's fix it. We will develop some software to enable the sightseeing company to route its bus tours more fairly—though it may sometimes mean a longer total distance for everyone, fair is fair, right?

For this problem, there is a starting location (the sightseeing company headquarters), h hotels that need to be visited for pickups and dropoffs, and a destination location (the amazing attraction). We need to find a route that goes from the headquarters, through all the hotels, to the attraction, then back through all the hotels again (possibly in a different order), and finally back to the headquarters.

To guarantee that none of the tourists (and, in particular, you) is forced to suffer through two full tours of the hotels, we require that every hotel that is visited among the first $h/2$ hotels on the way to the attraction is also visited among the first $h/2$ hotels on the way back.

Subject to these restrictions, we would like to make the complete bus tour

as short as possible. Note that these restrictions may force the bus to drive past a hotel without stopping there (this is not considered visiting) and then visit it later, as illustrated in the sample input.

Input Format

The first line of each test case consists of two integers n and m, where n is the number of locations (hotels, headquarters, attraction) and m is the number of pairs of locations between which the bus can travel.

The n different locations are numbered from 0 to n−1, where 0 is the headquarters, 1 through n−2 are the hotels, and n−1 is the attraction. Assume that there is at most one direct connection between any pair of locations and it is possible to travel from any location to any other location (but not necessarily directly).

Following the first line are m lines, each containing three integers u, v, and t, indicating that the bus can go directly between locations u and v in t seconds (in either direction).

Output Format

For each test case, print the case number and the minimum total travel time of the complete tour.

The output should follow this format: "Case X: Y"

where:

X is the test case number starting from 1.

Y is the shortest possible total time (in seconds) for the bus tour.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 4
0 1 10
1 2 20
2 3 30
3 4 40

4 6
0 1 1
0 2 1
0 3 1
1 2 1
1 3 1
2 3 1

Output: Case 1: 300
Case 2: 6

Answer

```
#include <stdio.h>

#define INF 100000000
#define MAXN 20

int dist[MAXN][MAXN];
int dp[1 << MAXN][MAXN];

int min(int a, int b) {
    return (a < b) ? a : b;
}

int main() {
    int n, m;
    int tc = 1;

    while (scanf("%d %d", &n, &m) != EOF) {

        // Initialize distances
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (i == j) dist[i][j] = 0;
                else dist[i][j] = INF;
            }
        }

        for (int i = 0; i < m; i++) {
            int u, v, t;
            scanf("%d %d %d", &u, &v, &t);
            if (t < dist[u][v]) {
                dist[u][v] = dist[v][u] = t;
```

```
    }
}

// Floyd-Warshall
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}

int FULL = (1 << n);

for (int i = 0; i < FULL; i++) {
    for (int j = 0; j < n; j++) {
        dp[i][j] = INF;
    }
}

dp[1 << 0][0] = 0;

// DP over subsets
for (int mask = 0; mask < FULL; mask++) {
    if (!(mask & 1)) continue; // must include headquarters (0)

    for (int u = 0; u < n; u++) {
        if (!(mask & (1 << u))) continue;
        if (dp[mask][u] == INF) continue;

        for (int v = 0; v < n; v++) {
            if (mask & (1 << v)) continue;

            if (dist[u][v] == INF) continue;

            int nmask = mask | (1 << v);

            dp[nmask][v] = min(dp[nmask][v],
                               dp[mask][u] + dist[u][v]);
        }
```

```
        }
    }

    int ALL = (1 << n) - 1;
    int ans = INF;

    for (int i = 0; i < n; i++) {
        if (dp[ALL][i] < INF && dist[i][0] < INF) {
            ans = min(ans, dp[ALL][i] + dist[i][0]);
        }
    }

    printf("Case %d: %d\n", tc++, ans);
  }

  return 0;
}
```

Status : Wrong ***Marks : 0/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 11_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 31

Section 1 : Coding

1. Problem Statement

Write a program to implement an optimal binary search tree using dynamic programming.

Given a sorted array key [0... n-1] and an array freq [0... n-1] of frequency counts, where freq [i] is the number of searches for key [i], Construct a binary search tree of all keys such that the total cost of all the searches is as small as possible.

Note: Cost of a BST node: level of that node * frequency. The level of the root is 1.

Input Format

The first line of the input is the value of n.

The second line of input is the keys separated by a single space.

The last line of input is the frequency separated by a single space.

Output Format

The output prints the cost of the optimal binary search tree.

Sample Test Case

Input: 3
10 12 20
34 8 50

Output: Cost of Optimal BST is 142

Answer

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

// Function to calculate sum of frequencies from i to j
int sumFreq(int freq[], int i, int j) {
    int sum = 0;
    for (int k = i; k <= j; k++) {
        sum += freq[k];
    }
    return sum;
}

// Function to calculate cost of Optimal BST
int optimalBST(int keys[], int freq[], int n) {
    int dp[n][n];  // dp[i][j] will store cost of optimal BST from keys[i..j]

    // Initialize for single keys
    for (int i = 0; i < n; i++) {
        dp[i][i] = freq[i];
    }

    // Fill dp table for chains of length 2 to n
    for (int length = 2; length <= n; length++) {
        for (int i = 0; i <= n - length; i++) {
            int j = i + length - 1;
            dp[i][j] = INT_MAX;
```

```
            // Try making each key in keys[i..j] as root
            for (int r = i; r <= j; r++) {
                int leftCost = (r > i) ? dp[i][r - 1] : 0;
                int rightCost = (r < j) ? dp[r + 1][j] : 0;
                int cost = leftCost + rightCost + sumFreq(freq, i, j);

                if (cost < dp[i][j]) {
                    dp[i][j] = cost;
                }
            }
        }
    }

    return dp[0][n - 1];
}

int main() {
    int n;
    scanf("%d", &n);

    int keys[n], freq[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &keys[i]);
    }
    for (int i = 0; i < n; i++) {
        scanf("%d", &freq[i]);
    }

    int result = optimalBST(keys, freq, n);
    printf("Cost of Optimal BST is %d\n", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given the weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack. In other words,

given two integer arrays, val[0..n-1] and wt[0..n-1] which represent values and weights associated with n items, respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of val[] such that the sum of the weights of this subset is smaller than or equal to W. You cannot break an item, either pick the complete item or don't pick it.

Solve this problem using dynamic programming.

Input Format

The first line of input represents the number of items.

The second line of input consists of the values of each item separated by spaces.

The third line of input consists of the weights of each item separated by spaces.

The fourth line of input consists of an integer, which represents the maximum capacity of the knapsack.

Output Format

The output prints the maximum value of items that can be held by the knapsack.

Sample Test Case

Input: 3
60 100 120
10 20 30
50
Output: 220

Answer

```
// You are using GCC
#include <stdio.h>

// Function to return maximum of two integers
int max(int a, int b) {
   return (a > b) ? a : b;
}

// Function to solve 0/1 Knapsack problem using DP
```

```
int knapsack(int val[], int wt[], int n, int W) {
    int dp[n + 1][W + 1];

    // Build table dp[][] in bottom-up manner
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i - 1] <= w)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    return dp[n][W];
}

int main() {
    int n;
    scanf("%d", &n);

    int val[n], wt[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
    }
    for (int i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }

    int W;
    scanf("%d", &W);

    printf("%d\n", knapsack(val, wt, n, W));

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Given an array, the task is to find the sum of the maximum sum by alternating subsequences starting with the first element. Here alternating sequence means first decreasing, then increasing, then decreasing,... For example, 10, 5, 14, and 3 are alternating sequences.

Note: The reverse sequence type (increasing—decreasing—increasing—...) is not considered alternating here.

Example

Input:

6

4 3 8 5 3 8

Output:

28

Explanation:

The alternating subsequence (starting with the first element) that has the maximum sum is {4, 3, 8, 5, 8}

Input Format

The first line contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output displays a single integer representing the maximum sum of an alternating subsequence from the given array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6

vtu27499

4 3 8 5 3 8

Output: 28

Answer

```
// You are using GCC
#include <stdio.h>

// Function to find maximum alternating subsequence sum
int maxAlternatingSum(int arr[], int n) {
    int sum = arr[0];   // Always start with the first element
    int i = 0;

    while (i < n - 1) {
        // Step 1: find next smaller element (decreasing)
        while (i < n - 1 && arr[i + 1] <= arr[i]) {
            i++;
        }
        sum += arr[i];  // add local minimum

        // Step 2: find next larger element (increasing)
        while (i < n - 1 && arr[i + 1] >= arr[i]) {
            i++;
        }
        sum += arr[i];  // add local maximum
    }

    return sum;
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int result = maxAlternatingSum(arr, n);
    printf("%d\n", result);

    return 0;
```

}

Status : Partially correct ***Marks : 1/10***

4. Problem Statement

A taxi driver wants to maximize the cab fare by choosing the longest possible travel path through a city represented as a square matrix.

Each cell in the matrix represents the kilometers travelled at that location.

The taxi can move only in two directions:

Right (i, j+1)Down (i+1, j)

The driver will move to the next cell only if the difference between the current value and the next value is exactly +1 or -1.

Rules:

Movement is allowed only right or down.The absolute difference between the current cell value and the next cell value must be 1.If both right and down moves satisfy the condition, the driver always prefers the right move.The trip continues until no valid move is possible.

Your task is to determine and print the longest trip path starting from the first cell (0,0).

Input Format

The first line contains an integer N, representing the size of the square matrix.

The next N lines contain N space-separated integers, representing the matrix values.

Output Format

The output prints the longest trip path starting from the first element of the matrix in the format "Longest trip path a->b->c->d" where a, b, c, d are the values visited in the path.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
2 3 1
0 4 5
8 7 6

Output: Longest trip path 2->3->4->5->6

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Function to check if move is valid
int isValid(int current, int next) {
    return abs(current - next) == 1;
}

// Function to find longest trip path
void longestTripPath(int N, int matrix[N][N]) {
    int i = 0, j = 0;
    printf("Longest trip path %d", matrix[i][j]);

    while (1) {
        // Prefer right move if valid
        if (j + 1 < N && isValid(matrix[i][j], matrix[i][j + 1])) {
            j++;
            printf("->%d", matrix[i][j]);
        }
        // Else try down move
        else if (i + 1 < N && isValid(matrix[i][j], matrix[i + 1][j])) {
            i++;
            printf("->%d", matrix[i][j]);
        }
        else {
            break; // No valid move
        }
    }
    printf("\n");
```

```
}

int main() {
    int N;
    scanf("%d", &N);

    int matrix[N][N];
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    longestTripPath(N, matrix);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 12_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

Section 1 : MCQ

1. The first step in the naive greedy algorithm is _____.

Answer

analysing the zero flow

Status : Correct ***Marks : 1/1***

2. Suppose you have coins of denominations 1,3 and 4. You use a greedy algorithm, in which you choose the largest denomination coin which is not greater than the remaining sum. For which of the following sums, will the algorithm doesn't produce an optimal answer?

Answer

10

vtu27499

Status : Correct ***Marks : 1/1***

3. Which data structure is most commonly used to efficiently implement a greedy algorithm that repeatedly selects the minimum (or maximum) element at each step?

Answer

Heap

Status : Correct ***Marks : 1/1***

4. What is a greedy algorithm?

Answer

An algorithm that makes the best choice at each step, without considering the overall impact

Status : Correct ***Marks : 1/1***

5. Which of the following problems is typically solved using a greedy algorithm?

Answer

Coin change problem.

Status : Wrong ***Marks : 0/1***

6. What is the primary advantage of greedy algorithms over other approaches for certain problems?

Answer

Greedy algorithms are easy to implement

Status : Correct ***Marks : 1/1***

7. Which of the following is a disadvantage of greedy algorithm?

Answer

It may not always find the globally optimal solution

Status : Correct ***Marks : 1/1***

8. Which of the following approaches should be used to find the solution to the activity selection problem?

Answer

Greedy approach

Status : Correct ***Marks : 1/1***

9. Suppose two activities A and B, have start and finish times as SA, FA and SB, FB respectively. Both activities are said to be compatible, under which of the following conditions?

Answer

SA >= FB or SB >= FA

Status : Correct ***Marks : 1/1***

10. In the activity selection problem, if two activities have the same finish time, which one is selected?

Answer

The one with the earlier start time

Status : Correct ***Marks : 1/1***

11. Consider the following number of activities with their start and finish time given below. In which sequence will the activity be selected in order to maximize the number of activities, without any conflicts?

Answer

A1, A2, A3, A4

Status : Wrong ***Marks : 0/1***

12. Which of the following is a characteristic of the greedy algorithm?

Answer

It makes the best choice at each step by considering only the local problem state.

Status : Correct ***Marks : 1/1***

13. What is the primary reason why the greedy algorithm does not work for some optimization problems?

Answer

It does not reconsider previous decisions and makes choices based only on the current state.

Status : Correct ***Marks : 1/1***

14. In the greedy approach using a stack, what condition do we check before popping from the stack

Answer

If the top is greater than current digit and k > 0

Status : Correct ***Marks : 1/1***

15. Why is sorting not used in this greedy solution?

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

vtu27499

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 12_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 38.5

Section 1 : Coding

1. Problem Statement

Given an array of non-negative integers, the task is to find the minimum number of elements such that their sum should be greater than the sum of the rest of the elements of the array.

Example:

Input: arr[] = [3 , 1 , 7, 1]

Output: 1

Explanation: Smallest subset is {7}. Sum of this subset is greater than the sum of all other elements left after removing subset {7} from the array

Input: arr[] = [2 , 1 , 2]

Output: 2

Explanation: Smallest subset is {2 , 1}. Sum of this subset is greater than the sum of all other elements left after removing subset {2 , 1} from the array

Input Format

The first line contains an Integer n, representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints a single integer representing the minimum number of elements whose sum is greater than the sum of the remaining elements.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3 1 7 1

Output: 1

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Comparator for descending sort
int cmp(const void *a, const void *b) {
    return (*(int*)b - *(int*)a);
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[25]; // n ≤ 20
    int total = 0;
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
        total += arr[i];
    }

    // Sort in descending order
    qsort(arr, n, sizeof(int), cmp);

    int chosen_sum = 0, count = 0;
    for (int i = 0; i < n; i++) {
        chosen_sum += arr[i];
        count++;
        if (chosen_sum > total - chosen_sum) {
            printf("%d\n", count);
            return 0;
        }
    }
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given an array prices[] of n different candies, where prices[i] represents the price of the ith candy and all prices are distinct. The store has an offer where for every candy you buy, you can get up to k other different candies for free. Find the minimum and maximum amount of money needed to buy all the candies.

Note: In both cases, you must take the maximum number of free candies possible during each purchase.

Examples:

Input: prices[] = [3, 2, 1, 4], k = 2

Output: [3, 7]

Explanation: For minimum cost, buy the candies costing 1 and 2. for maximum cost, buy the candies costing 4 and 3.

Input: prices[] = [3, 2, 1, 4, 5], k = 4

Output: [1, 5]

Explanation: For minimum cost, buy the candy costing 1. for maximum cost, buy the candy costing 5.

Input Format

The first line contains an Integer n, representing the number of candies.

The second line contains n space-separated integers, representing the prices of the candies.

The third line contains an integer k, representing the number of free candies per purchase.

Output Format

The output prints two integers separated by a space:

minCost maxCost

representing the minimum and maximum money needed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3 2 1 4
2

Output: 3 7

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Comparator for ascending sort
int cmpAsc(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
```

```
}

// Comparator for descending sort
int cmpDesc(const void *a, const void *b) {
    return (*(int*)b - *(int*)a);
}

int main() {
    int n, k;
    scanf("%d", &n);

    int prices[25];
    for (int i = 0; i < n; i++) {
        scanf("%d", &prices[i]);
    }
    scanf("%d", &k);

    // Minimum cost
    qsort(prices, n, sizeof(int), cmpAsc);
    int minCost = 0;
    for (int i = 0; i < n; i += (k + 1)) {
        minCost += prices[i];
    }

    // Maximum cost
    qsort(prices, n, sizeof(int), cmpDesc);
    int maxCost = 0;
    for (int i = 0; i < n; i += (k + 1)) {
        maxCost += prices[i];
    }

    printf("%d %d\n", minCost, maxCost);
    return 0;
}
```

Status : Partially correct ***Marks : 2.5/10***

3. Problem Statement

Given a fraction represented by two integers — a numerator n and a denominator d — express the fraction as a sum of unique unit fractions

(fractions with numerator 1). This representation is known as the Egyptian Fraction.

Write a program to ensure that the given fraction is converted into its Egyptian Fraction form.

Input Format

The input contains two space-separated integers n and d, representing the numerator and denominator of the fraction.

Output Format

The output prints the Egyptian Fraction representation of the given fraction in the form of unit fractions separated by +.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 14

Output: 1/3 + 1/11 + 1/231

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n, d;
    scanf("%d %d", &n, &d);

    while (n != 0) {
        // Find the ceiling of d/n
        int x = (d + n - 1) / n;  // ceil(d/n)

        printf("1/%d", x);

        // Update fraction: n/d - 1/x = (n*x - d) / (d*x)
        n = n * x - d;
        d = d * x;
```

```
    if (n != 0) printf(" + ");
  }
  printf("\n");
  return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Emily is building a data compression feature in her file organizing system. She wants filenames to have no repeated characters and appear in the smallest possible order so users can locate files easily. For example, if a corrupted filename is "cbacdcbc", she wants it cleaned up as "acdb" — which preserves one instance of each letter, keeps relative order, and is the smallest alphabetically.

This function helps ensure filenames are clean, minimal, and alphabetically organized!

Example 1:

Input: s = "bcabc"

Output: "ABC"

Example 2:

Input: s = "cbacdcbc"

Output: "acdb"

Input Format

The first line of input contains a string num representing a non-negative integer.

The second line contains an integer k representing how many digits to remove.

Output Format

The output prints the string representing the smallest possible number after removing exactly k digits.

Leading zeros should be removed, except if the number is zero itself.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: bcabc

Output: abc

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

int main() {
    char s[10005];
    scanf("%s", s);

    int n = strlen(s);
    int freq[26] = {0};
    bool inStack[26] = {false};

    // Count frequency of each character
    for (int i = 0; i < n; i++) {
        freq[s[i] - 'a']++;
    }

    char stack[10005];
    int top = -1;

    for (int i = 0; i < n; i++) {
        int c = s[i] - 'a';
        freq[c]--;

        if (inStack[c]) continue; // already in stack

        // Greedy removal: pop larger chars if they still appear later
        while (top >= 0 && stack[top] > s[i] && freq[stack[top] - 'a'] > 0) {
            inStack[stack[top] - 'a'] = false;
```

```
        top--;
      }

      stack[++top] = s[i];
      inStack[c] = true;
    }

    stack[top + 1] = '\0';
    printf("%s\n", stack);

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

Given the arrival and departure times of all trains that reach a railway station, find the minimum number of platforms required for the station so that no train is kept waiting. Consider that all the trains arrive on the same day and leave on the same day.

Arrival and departure times can never be the same for a train, but we can have the arrival time of one train equal to the departure time of the other. At any given instance, the same platform cannot be used for both the departure of a train and the arrival of another train. In such cases, we need different platforms.

Example 1

Input:

N=6

arr[] = {0900, 0940, 0950, 1100, 1500, 1800}, dep[] = {0910, 1200, 1120, 1130, 1900, 2000}

Output: 3

Explanation: A minimum of three platforms are required to safely arrive

and depart all trains.

Input Format

The first line consists of an integer N, which represents the total number of trains.

The second line consists of N space-separated integers representing the arrival time.

The third line consists of N space-separated integers representing the departure time.

Output Format

The output displays an integer which is the minimum number of platforms required to safely arrive and depart the train.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
0900 0940 0950 1100 1500 1800
0910 1200 1120 1130 1900 2000

Output: 3

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Comparator for qsort
int cmp(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int main() {
    int N;
    scanf("%d", &N);
```

```
    int arr[15], dep[15];
    for (int i = 0; i < N; i++) scanf("%d", &arr[i]);
    for (int i = 0; i < N; i++) scanf("%d", &dep[i]);

    // Sort arrival and departure times
    qsort(arr, N, sizeof(int), cmp);
    qsort(dep, N, sizeof(int), cmp);

    int i = 0, j = 0;
    int platforms = 0, maxPlatforms = 0;

    while (i < N && j < N) {
        if (arr[i] < dep[j]) {
            // Train arrives before previous departs    need new platform
            platforms++;
            if (platforms > maxPlatforms) maxPlatforms = platforms;
            i++;
        } else {
            // Train departs    free a platform
            platforms--;
            j++;
        }
    }

    printf("%d\n", maxPlatforms);
    return 0;
}
```

Status : Partially correct ***Marks : 6/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 12_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement:

Raju has a hot dog of a certain length that he wishes to divide strategically to maximize the number of friends he can serve. The effectiveness of a given partition is determined by the product of its segment lengths. Raju's goal is to determine the optimal way to cut the hot dog such that this product is maximized, thereby maximizing the number of friends he can serve.

Example

Input:

4

Output:

Maximum friends served 4

Explanation:

For a hot dog of length 4, possible partitions and their corresponding products are:

1, 3 (1 × 3) = 3

2, 2 (2 × 2) = 4 (Maximum)

3, 1 (3 × 1) = 3

1, 1, 2 (1 × 1 × 2) = 2

1, 2, 1 (1 × 2 × 1) = 2

2, 1, 1 (2 × 1 × 1) = 2

1, 1, 1, 1 (1 × 1 × 1 × 1) = 1

Thus, the optimal partition results in a maximum product of 4, serving the highest number of friends.

Input Format

The input consists of a single integer n, representing the length of the hot dog.

Output Format

The output prints the maximum number of friends served.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
Output: Maximum friends served 4

Answer

```
// You are using GCC
#include <stdio.h>

// Function to compute maximum product partition
int maxProduct(int n) {
    if (n == 2) return 1;
```

```
    if (n == 3) return 2;

    int product = 1;
    // Cut into as many 3's as possible
    while (n > 4) {
        product *= 3;
        n -= 3;
    }
    product *= n; // multiply the remainder
    return product;
}

int main() {
    int n;
    scanf("%d", &n);

    int result = maxProduct(n);
    printf("Maximum friends served %d\n", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Imagine you are given a set of items, each with a weight, a value, and a penalty value. You need to solve the Fractional Knapsack problem with the objective of maximizing the total value while minimizing penalties. Implement a greedy algorithm to choose items that best meet this objective.

Example:

Input:

2

5 20 2

10 30 5

15

Output:

Maximum total value while minimizing penalties: 14.00

Explanation:

Sort the items based on the modified value-to-weight ratios (value - penalty) / weight:

Item 1: (7 - 1) / 2 = 3

Item 2: (12 - 4) / 3 = 2

Start filling the knapsack:

Add all of Item 1 (Weight: 2, Value: 7 - 1 = 6)

Add 1 unit of Item 2 (Weight: 3, Value: 12 - 4 = 8)

The total value is 6 + 8 = 14.

So, the correct output for the given input is 14.

Example:

Input:

2

5 20 2

10 30 5

15

Output:

Maximum total value while minimizing penalties: 43.00

Explanation:

Sort the items based on the modified value-to-weight ratios (value - penalty) / weight:

Item 2: (30 - 5) / 10 = 2.5

Item 1: (20 - 2) / 5 = 3.6

Start filling the knapsack:

Add all of Item 1 (Weight: 5, Value: 20 - 2 = 18)

Add 1 unit of Item 2 (Weight: 10, Value: 30 - 5 = 25)

Now, let's calculate the total value of the knapsack:

Total Weight = 5 + 10 = 15 (fits within the knapsack's capacity)

Total Value = 18 + 25 = 43

Input Format

The first line contains an integer 'N' representing the number of items.

The next 'N' lines contain information for each item:

Three integers 'weight,' 'value,' and 'penalty,' are separated by spaces, where 'weight' is the weight of the item, 'value' is the value of the item, and 'penalty' is the penalty for taking the item.

The last line contains an integer 'knapsackWeight' representing the weight limit of the knapsack.

Output Format

The output prints "Maximum total value while minimizing penalties: " followed by the maximum total value achievable while minimizing penalties, rounded to two decimal places.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 2
2 7 1
3 12 4

vtu27499

5
Output: Maximum total value while minimizing penalties: 14.00

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Structure for items
typedef struct {
    int weight;
    int value;
    int penalty;
    double ratio;
    double effectiveValue;
} Item;

// Comparator for sorting by ratio (descending)
int cmp(const void *a, const void *b) {
    double r1 = ((Item*)a)->ratio;
    double r2 = ((Item*)b)->ratio;
    if (r1 < r2) return 1;
    else if (r1 > r2) return -1;
    return 0;
}

int main() {
    int N;
    scanf("%d", &N);

    Item items[15];
    for (int i = 0; i < N; i++) {
        scanf("%d %d %d", &items[i].weight, &items[i].value, &items[i].penalty);
        items[i].effectiveValue = items[i].value - items[i].penalty;
        items[i].ratio = items[i].effectiveValue / items[i].weight;
    }

    int knapsackWeight;
    scanf("%d", &knapsackWeight);

    // Sort items by ratio
    qsort(items, N, sizeof(Item), cmp);
```

```
    double totalValue = 0.0;
    int remainingWeight = knapsackWeight;

    for (int i = 0; i < N && remainingWeight > 0; i++) {
        if (items[i].weight <= remainingWeight) {
            // Take whole item
            totalValue += items[i].effectiveValue;
            remainingWeight -= items[i].weight;
        } else {
            // Take fraction of item
            totalValue += items[i].ratio * remainingWeight;
            remainingWeight = 0;
        }
    }

    printf("Maximum total value while minimizing penalties: %.2f\n", totalValue);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

There are n participants in a race. Each participant is assigned a performance score given in the integer array performance.

You are responsible for awarding medals to these participants, subject to the following conditions:

Each participant must receive at least one medal.

Participants with a higher performance score than their neighbors must receive more medals than their neighbors.

Return the minimum number of medals you need to distribute to the participants.

Example 1:

Input: ratings = [1,0,2]

Output: 5

Explanation:

Participant 1 gets 1 medal.

Participant 2 gets 1 medal (since their performance is the lowest).

Participant 3 gets 2 medals (since their performance is higher than Participant 2).

The total number of medals required is 5.

Example 2:

Input: ratings = [1,2,2]

Output: 4

Explanation:

Participant 1 gets 1 medal.

Participant 2 gets 2 medals (since their performance is better than Participant 1).

Participant 3 gets 1 medal (because their performance is equal to Participant 2, and they don't need more medals).

The total number of medals required is 4.

Input Format

The first line consists of an integer n representing the number of participants.

The second line consists of n space-separated integers representing the performance scores of the participants.

Output Format

An integer representing the minimum number of medals required to satisfy the conditions.

Refer to the sample output for the formatting specifications.

vtu27499

Sample Test Case

Input: 3
1 0 2

Output: 5

Answer

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int ratings[20005];
    for (int i = 0; i < n; i++) {
        scanf("%d", &ratings[i]);
    }

    int medals[20005];
    for (int i = 0; i < n; i++) {
        medals[i] = 1; // everyone gets at least 1
    }

    // Left to right
    for (int i = 1; i < n; i++) {
        if (ratings[i] > ratings[i-1]) {
            medals[i] = medals[i-1] + 1;
        }
    }

    // Right to left
    for (int i = n-2; i >= 0; i--) {
        if (ratings[i] > ratings[i+1]) {
            medals[i] = max(medals[i], medals[i+1] + 1);
        }
    }
```

```
    int total = 0;
    for (int i = 0; i < n; i++) {
        total += medals[i];
    }

    printf("%d\n", total);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

You are planning a circular road trip that passes through several gas stations. Each station provides a certain amount of fuel, and traveling from one station to the next consumes a specific amount of fuel.

Your goal is to determine a starting gas station index from which you can begin your journey such that you complete the entire circuit in a clockwise direction without running out of fuel at any point.

Given two integer arrays:

gas[i] represents the fuel available at the i-th station,

cost[i] represents the fuel required to travel from station i to station i+1 (circularly),

Return the starting station index that allows completing the full trip.

If no such station exists, return -1.

Example 1

Input:

5

1 2 3 4 5

3 4 5 1 2

Output:

3

Explanation:

Start at station 3 (index 3) and fill up with 4 units of gas. Your tank = 0 + 4 = 4

Travel to station 4. Your tank = 4 - 1 + 5 = 8

Travel to station 0. Your tank = 8 - 2 + 1 = 7

Travel to station 1. Your tank = 7 - 3 + 2 = 6

Travel to station 2. Your tank = 6 - 4 + 3 = 5

Travel to station 3. The cost is 5. Your gas is just enough to travel back to station 3.

Therefore, return 3 as the starting index.

Example 2

Input

3

2 3 4

3 4 3

Output:

-1

Explanation:

You can't start at station 0 or 1, as there is not enough gas to travel to the next station.

Let's start at station 2 and fill up with 4 units of gas. Your tank = 0 + 4 = 4

Travel to station 0. Your tank = 4 - 3 + 2 = 3

Travel to station 1. Your tank = 3 - 3 + 3 = 3

You cannot travel back to station 2, as it requires 4 units of gas but you only have 3.

vtu27499

Therefore, you can't travel around the circuit once no matter where you start.

Input Format

The first line of input consists of an integer N, representing the number of gas stations.

The second line of input consists of N space-separated integers representing the amount of gas available at each station.

The third line of input consists of N space-separated integers representing the cost of traveling from each station to the next.

Output Format

The output prints an integer representing the index of the starting gas station that allows you to complete the circuit, or -1 if no such station exists.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
3 4 5 1 2
Output: 3

Answer

```
// You are using GCC
#include <stdio.h>

int canCompleteCircuit(int gas[], int cost[], int n) {
    int totalGas = 0, totalCost = 0;
    for (int i = 0; i < n; i++) {
        totalGas += gas[i];
        totalCost += cost[i];
    }
    if (totalGas < totalCost) return -1;

    int start = 0, tank = 0;
```

```
    for (int i = 0; i < n; i++) {
        tank += gas[i] - cost[i];
        if (tank < 0) {
            start = i + 1;
            tank = 0;
        }
    }
    return start;
}

int main() {
    int n;
    scanf("%d", &n);

    int gas[100005], cost[100005];
    for (int i = 0; i < n; i++) scanf("%d", &gas[i]);
    for (int i = 0; i < n; i++) scanf("%d", &cost[i]);

    int result = canCompleteCircuit(gas, cost, n);
    printf("%d\n", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: MALGIREDDY SAIKRUPA REDDY M
Email: vtu27499@veltech.edu.in
Roll no: vtu27499
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L1

VTU_Week 12_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an amount, find the minimum number of notes of different denominations that sum up to the given amount. Starting from the highest denomination note, try to accommodate as many notes as possible for a given amount.

We may assume that we have infinite supply of notes of values {2000, 500, 200, 100, 50, 20, 10, 5, 1}

Examples:

Input: 800

Output: Currency Count

500 : 1

200 : 1

100 : 1

Input: 2456

Output: Currency Count

2000 : 1

200 : 2

50 : 1

5 : 1

1 : 1

Input Format

The input contains an integer representing the amount to be broken down.

Output Format

The output prints "Currency Count" followed by each denomination used in the format:

"note_value : count"

(Only print denominations that are used.)

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 800
Output: Currency Count
500 : 1
200 : 1
100 : 1

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int main() {
    int amount;
    scanf("%d", &amount);

    int denominations[] = {2000, 500, 200, 100, 50, 20, 10, 5, 1};
    int n = sizeof(denominations) / sizeof(denominations[0]);

    printf("Currency Count\n");

    for (int i = 0; i < n; i++) {
        if (amount >= denominations[i]) {
            int count = amount / denominations[i];
            amount = amount % denominations[i];
            printf("%d : %d\n", denominations[i], count);
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given a lock which is made up of n-different circular rings and each ring has 0-9 digit printed serially on it. Initially all n-rings together show a n-digit integer but there is particular code only which can open the lock.

You can rotate each ring any number of time in either direction. You have to find the minimum number of rotation done on rings of lock to open the lock.

Examples:

Input: Input = 2345, Unlock code = 5432

Output: Rotations required = 8

Explanation: 1st ring is rotated thrice as 2->3->4->5

2nd ring is rotated once as 3->4

3rd ring is rotated once as 4->3

4th ring is rotated thrice as 5->4->3->2

Input: Input = 1919, Unlock code = 0000

Output: Rotations required = 4

Explanation: 1st ring is rotated once as 1->0

2nd ring is rotated once as 9->0

3rd ring is rotated once as 1->0

4th ring is rotated once as 9->0

Input Format

The first line contains an integer representing the current input on the lock.

The second line contains an integer representing the unlock code.

Output Format

The output prints "Minimum Rotation = X" where X is the minimum total number of rotations needed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 28756
98234

Output: Minimum Rotation = 12

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char current[100], target[100];
```

```
    scanf("%s", current);
    scanf("%s", target);

    int n = strlen(current);
    int rotations = 0;

    for (int i = 0; i < n; i++) {
        int c = current[i] - '0';
        int t = target[i] - '0';
        int diff = abs(c - t);
        rotations += (diff < 10 - diff) ? diff : (10 - diff);
    }

    printf("Minimum Rotation = %d\n", rotations);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement:

Sarah is organizing a race in her local community park, where participants must jump from one marker to the next. Each participant starts at the first marker and can make jumps to other markers based on the distance provided for each marker. Sarah wants to calculate the minimum number of jumps it would take for a participant to reach the final marker.

You are tasked with writing a program that helps Sarah determine the minimum number of jumps required for a participant to reach the last marker.

Each element in the array represents the maximum number of steps a participant can jump forward from that marker. For example, if you're at marker i, you can jump to any marker within the range [i+1, i + nums[i]].

Write a program to calculate the minimum number of jumps to reach the last marker.

Example 1:

Input: nums = 2 3 1 1 4

Output: 2

Explanation: The minimum number of jumps to reach the last index is 2. Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: nums = 2 3 0 1 4

Output: 2

Input Format

The input consists of space-separated integers representing the jump lengths from each index.

Output Format

The output prints an integer, representing the minimum number of jumps required to reach the last index.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2 3 1 1 4
Output: 2

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

int minJumps(int nums[], int n) {
    if (n <= 1) return 0;  // Already at the last marker
    if (nums[0] == 0) return -1; // Can't move anywhere

    int jumps = 0;
    int farthest = 0;
    int currentEnd = 0;
```

```
    for (int i = 0; i < n - 1; i++) {
        farthest = (farthest > i + nums[i]) ? farthest : (i + nums[i]);

        if (i == currentEnd) {
            jumps++;
            currentEnd = farthest;
        }
    }
    return jumps;
}

int main() {
    int nums[10000];
    int n = 0;

    // Read space-separated input until newline
    while (scanf("%d", &nums[n]) == 1) {
        n++;
        if (getchar() == '\n') break;
    }

    int result = minJumps(nums, n);
    printf("%d\n", result);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Shoot the Kuruvi (Balloons)

There are some spherical balloons taped onto a flat wall that represent the XY-plane. The balloons are represented as a 2D integer array points, where points[i] = [xstart, xend] denotes a balloon whose horizontal diameter stretches between xstart and xend. You do not know the exact y-coordinates of the balloons.

Arrows can be shot up directly vertically (in the positive y-direction) from different points along the x-axis. A balloon with xstart and xend is burst by

an arrow shot at x if xstart <= x <= xend. There is no limit to the number of arrows that can be shot. A shot arrow keeps traveling up infinitely, bursting any balloons in its path.

Given the number of array points, followed by the array points. Return the minimum number of arrows that must be shot to burst all balloons.

Example

Input

4

10 16

2 8

1 6

7 12

Output

2

Explanation

The balloons can be burst by 2 arrows:

Shoot an arrow at x = 6, bursting the balloons [2,8] and [1,6].

Shoot an arrow at x = 11, bursting the balloons [10,16] and [7,12].

Input

4

1 2

3 4

5 6

7 8

Output

4

vtu27499

Explanation

One arrow needs to be shot for each balloon for a total of 4 arrows.

Input

4

1 2

2 3

3 4

4 5

Output

2

Explanation

The balloons can be burst by 2 arrows:

Shoot an arrow at x = 2, bursting the balloons [1,2] and [2,3].

Shoot an arrow at x = 4, bursting the balloons [3,4] and [4,5].

Input Format

The first line of input consists of a single integer n, representing the Number of balloons

The next n lines contain two integers xstart and xend, representing the start and end positions of each balloon

Output Format

The output prints a single integer, which represents the minimum number of arrows needed to burst all the balloons.

Refer to the output for the formatting specifications.

Sample Test Case

Input: 4

vtu27499

1 2
2 3
3 4
4 5

Output: 2

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Comparator for qsort: sort by end coordinate
int compare(const void *a, const void *b) {
    int *balloonA = (int *)a;
    int *balloonB = (int *)b;
    return balloonA[1] - balloonB[1];
}

int main() {
    int n;
    scanf("%d", &n);

    int balloons[n][2];
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &balloons[i][0], &balloons[i][1]);
    }

    // Sort balloons by end coordinate
    qsort(balloons, n, sizeof(balloons[0]), compare);

    int arrows = 1; // At least one arrow needed
    int arrowPos = balloons[0][1]; // Shoot at end of first balloon

    for (int i = 1; i < n; i++) {
        if (balloons[i][0] > arrowPos) {
            // Need a new arrow
            arrows++;
            arrowPos = balloons[i][1];
        }
    }

    printf("%d\n", arrows);
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu27499